

Snap! のこと II

齋藤文康

2026 年 8 月 13 日

この文書は、容量の関係で、「Snap! のこと」に収まらなくなった項目を別文書にしたものです。私の理解不足で間違っているところがある可能性もあります。正しい内容になるように努めました、スクリプトを含め無保証です。

置き場所は、「Snap! プログラミング」[<https://fumiyasu-aizu.github.io/>] です。容量の関係で分割した部分「Snap! のこと I」もそこに置かれています。このサイトが検索エンジンに無視されて表示されないことがよくあります。間違いの修正や新たに分かったことを加筆していますので上記サイトの最新のものをご利用ください。

目次

1	行列の積	4
1.1	inner product	4
1.2	outer product	11
2	Continuation 継続	14
2.1	run と call の with continuation バージョン	14
2.2	catch と throw	17
2.3	map からの脱出	17
2.4	break と continue	19
2.5	thread	21
3	カスタムブロックの生成	25
4	this ()	35
5	再帰呼び出し	39
5.1	再帰呼び出しの例	39
5.1.1	階乗	39
5.1.2	ハノイの塔	39
5.2	再帰呼び出しの使用	40
5.2.1	繰り返し	40
5.2.2	my length	41
5.2.3	リストをリポート	43
5.2.4	take と drop	44
5.2.5	my contains	45
5.2.6	my uniques	45
5.2.7	my index of () in ()	46
5.2.8	リスト要素の巡回	48
5.2.9	リストのコピー	50
5.2.10	指定の要素に対する delete と replace	53
5.3	= と identical	55
5.3.1	クイックソート（整列／並べ替え）	57
5.3.2	フィボナッチ数列	60
5.3.3	末尾再帰	62
6	高階関数	64
6.1	高階関数型ブロックの基本	64
6.2	高階関数型の take と drop	66
6.3	操作を指定するリストの巡回	67
6.4	foldl, foldr	69

6.5	unfold	74
6.6	クロージャー	76

1 行列の積

1.1 inner product

配列要素同士の掛け算とは別に、代数学には行列と行列の積というものがあります。

2 行 2 列の行列同士では次のような計算方法になります。

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11} \times b_{11} + a_{12} \times b_{21} & a_{11} \times b_{12} + a_{12} \times b_{22} \\ a_{21} \times b_{11} + a_{22} \times b_{21} & a_{21} \times b_{12} + a_{22} \times b_{22} \end{pmatrix}$$

a 側の行列の列数と b 側の行列の行数は同じ必要があります。

定義ブロックを作成することは可能ですが、APL primitives ライブラリーに inner product ブロックがあるので利用できます。

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 1 \times 5 + 2 \times 6 \\ 3 \times 5 + 4 \times 6 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}$$

1	17
length: 1	
2	39
length: 1	
length: 2	



$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 4 & 5 \\ 6 & 7 \\ 8 & 9 \end{pmatrix} = \begin{pmatrix} 1 \times 4 + 2 \times 6 + 3 \times 8 & 1 \times 5 + 2 \times 7 + 3 \times 9 \end{pmatrix} = \begin{pmatrix} 40 & 46 \end{pmatrix}$$

1	A	B
1	40	46



$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$ は 1 行 3 列の行列です。list 1 2 3 としてしまうと 3 行 1 列の行列になってしまうので、

1	A	B	C
1	1	2	3

にする必要があります。

行列の積の利用例として座標変換があります。座標 (x, y) の点を原点 $(0, 0)$ を中心にして反時計回りに θ (シータ) 度回転させた座標 (x', y') を求めるものです。

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

web で「回転行列」を検索すると、大学系などのサイトで数学的な解説が見られます。

一番わかりやすい例として、 $(1, 0)$ の点を 90 度反時計回りに回転させると $(0, 1)$ の点になります。

左側の行列、回転させるための変換行列 R を作成するブロックです。



R を表示すると、次のような値の要素の行列になります。 $6.123233995736766e-17$ は 0 とみなします。

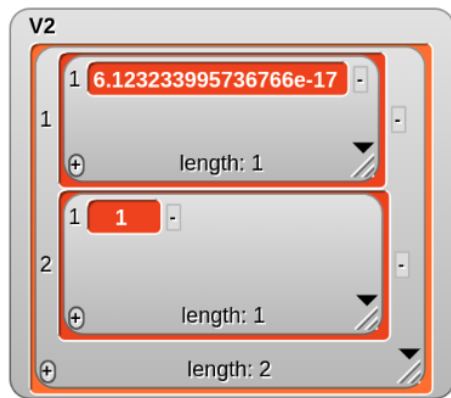


この行列 $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ と座標 (x, y) $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ の積

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 \times 1 + (-1) \times 0 \\ 1 \times 1 + 0 \times 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix} \text{ を求めます。}$$



$V0$ が変換前の座標で、 $V2$ が変換後の座標です。



結果として、座標 $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ を 90 度回転させた座標 $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ が求められました。

注意点として、(x, y) の座標の表し方が 2 行 1 列の形で指定する必要があります。

(x, y) 座標の対を並べたものを指定すればそのまま変換された座標の対が得られますが、普通の (x, y) 座標の指定の仕方ではありません。

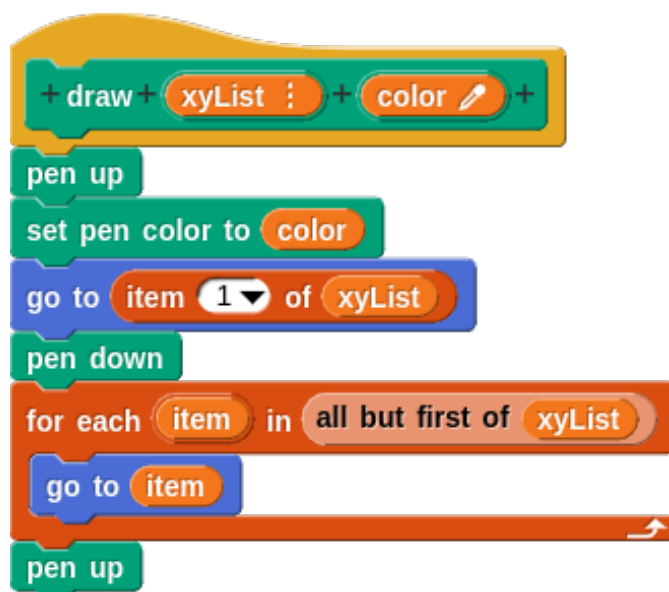


転置行列を使うと、普通の (x, y) 座標の指定の仕方ができます。



(x, y) 座標のリストで与えられた図形を一筆書きで描くブロックです。

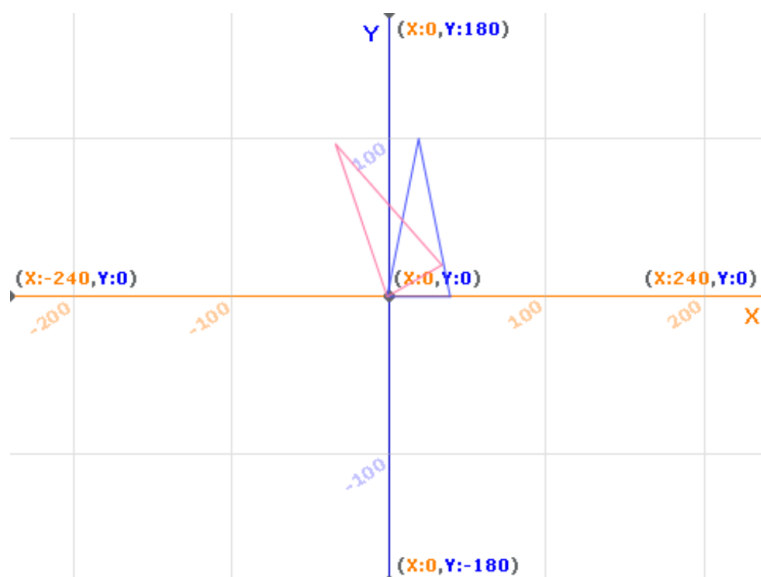
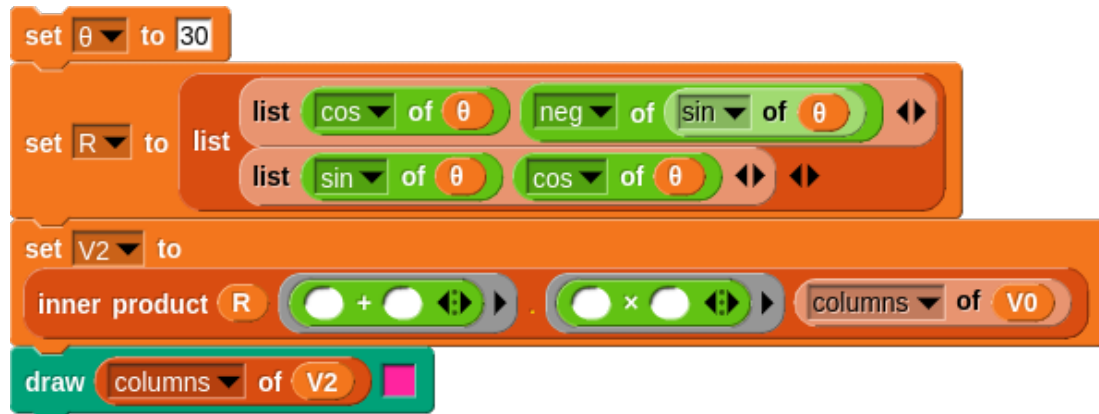
go to ブロックは **go to random position** にリストをドロップしたものです。



三角形  を描くスクリプトです。



それを半時計回りに 30 度回転させるスクリプトです。



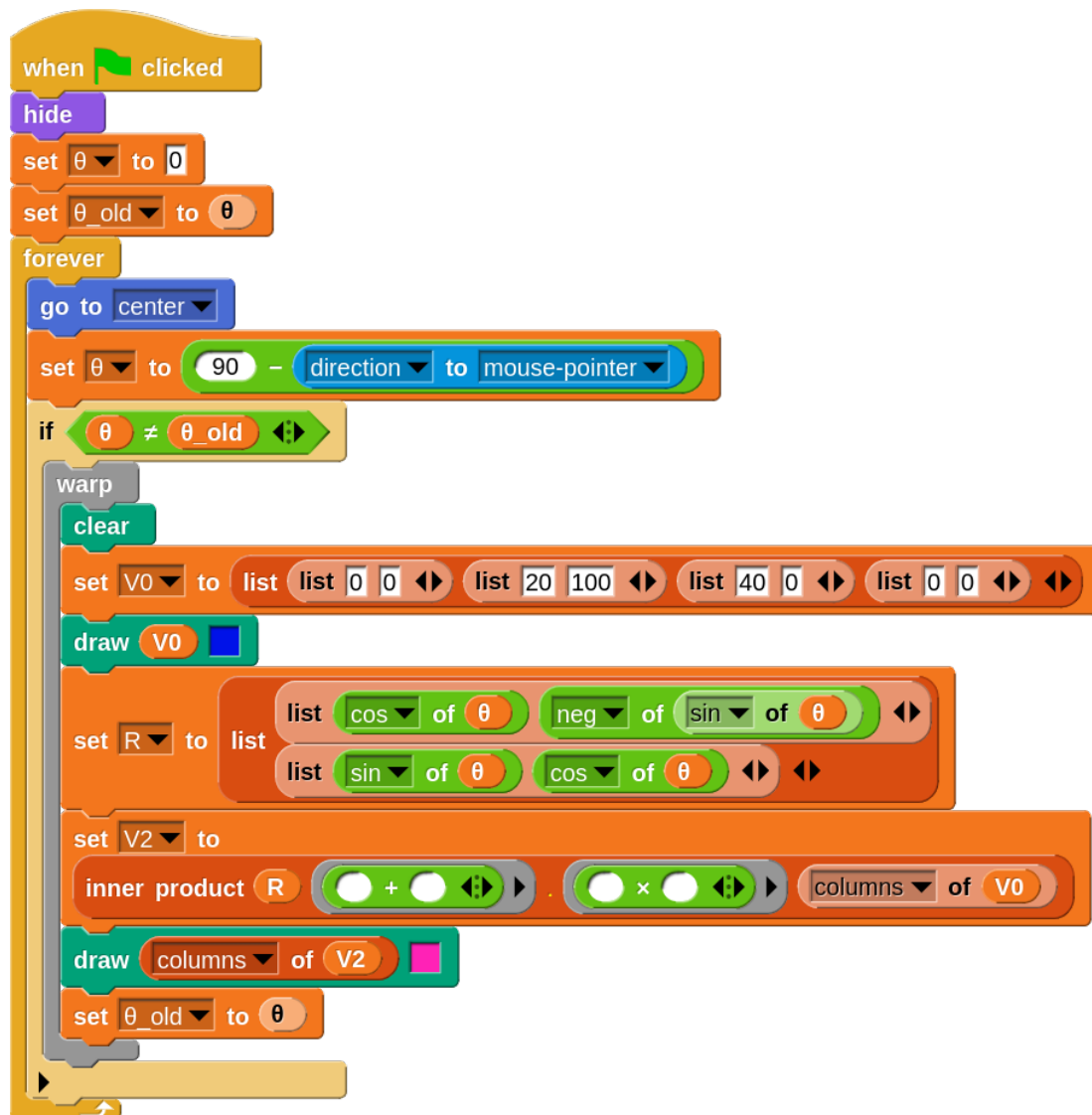
[参考文献]

『Python ではじめる数学の冒険

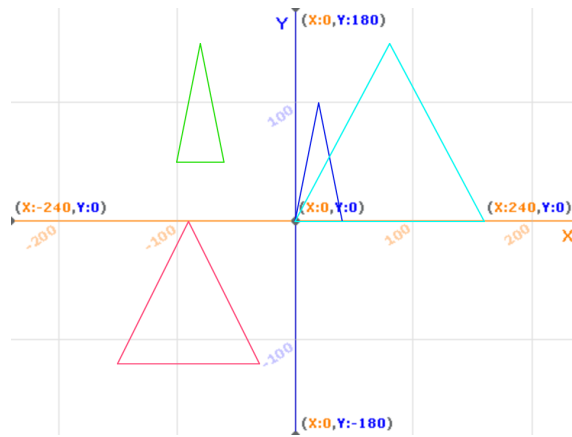
プログラミングで図解する代数、幾何学、三角関数』

Peter Farrell 著 鈴木幸敏 訳 オライリー・ジャパン 刊

原点座標から見たマウス位置の角度を θ として、マウスの動きに合わせて回転させてみます。なお、角度は水平方向を 0 度とする必要がありますので補正します。角度を記録して変化があった時だけ warp で描画しています。



行列の積の演算には加算と乗算が含まれるので、加算部分だけが有効になるようにすれば図形の平行移動が可能ですし、乗算部分だけが有効になるようにすれば図形の拡大縮小が可能です。



x 軸方向に 4 倍、y 軸方向に 1.5 倍します。

$$\begin{pmatrix} 4 & 0 \\ 0 & 1.5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \begin{pmatrix} 4 \times x + 0 \times y \\ 0 \times x + 1.5 \times y \end{pmatrix} \Rightarrow \begin{pmatrix} 4x \\ 1.5y \end{pmatrix}$$



平行移動の場合は 1（有効化）と 0（無効化）の要素を追加（指定）して、移動座標を求めます。

x 軸方向に -100、y 軸方向に 50 平行移動します。

$$\begin{pmatrix} 1 & 0 & -100 \\ 0 & 1 & 50 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \Rightarrow \begin{pmatrix} 1 \times x + 0 \times y + (-100) \times 1 \\ 0 \times x + 1 \times y + 50 \times 1 \end{pmatrix} \Rightarrow \begin{pmatrix} x - 100 \\ y + 50 \end{pmatrix}$$



Snap! ではリスト全体に対する演算ができるので、inner product を使わなくても次のように拡大縮小（x 軸方向に 3 倍、y 軸方向に 1.2 倍）と平行移動（-150, -120）が一度にできます。



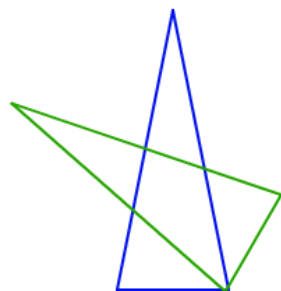
平行移動を利用すれば図形の任意の位置を中心として回転させることができます。三角形 V0 の重心の座標を中心にして 60 度回転してみます。

(Gx, Gy) を V0 の重心にし、原点に平行移動し、回転させ、元の位置に戻します。座標値を変換しているだけで図形をあちこち移動させているわけではありません。

```

hide
set V0 to list list 0 0 list 20 100 list 40 0 list 0 0
draw V0
set Gx to
  item 1 of item 1 of V0 + item 1 of item 2 of V0 +
  item 1 of item 3 of V0 / 3
set Gy to
  item 2 of item 1 of V0 + item 2 of item 2 of V0 +
  item 2 of item 3 of V0 / 3
set θ to 60
set R to list list cos of θ neg of sin of θ
  list sin of θ cos of θ
set V2 to
  pipe V0 - list Gx Gy
  columns of
  inner product R + ×
  columns of
  + list Gx Gy
draw V2

```



1.2 outer product

inner product に対して、**outer product**   というものもあります。

APL には、**numbers from 1 to X** の機能を持つ  X  があります。
この記号のようなものはギリシャ文字の ι イオタです。

次のようにすると、九九の表の一部ができます。

3	A	B	C
1	1	2	3
2	2	4	6
3	3	6	9

outer product  3     3 

計算方法を表示させてみます。

3	A	B	C
1	1x1	1x2	1x3
2	2x1	2x2	2x3
3	3x1	3x2	3x3

outer product  3  **join**    3 

単位行列なども作成できます。

4	A	B	C	D
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1

outer product  4  **if**   **then** 1 **else** 0   4 

4	A	B	C	D
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1

outer product  4  **if**   **then** 1 **else** 0   4 

[参考文献]

『基礎からの APL 解説と例題例解』

西川利男/日本アイビー・エム 共著 サイエンスハウス 刊

『基礎からの APL 解説と例題例解』では、素数の求め方が解説されています。

set N to 5

1 ~ 5 の整数のリスト N を作り、その中の素数を求めてみます。

まず、そのリストの各要素に対して、そのリストの各要素で割った余りが 0 のものを調べます。

(11 ページ、outer product の計算方法で $\times$ の代わりに mod になります。)

5	A	B	C	D	E
1	true	false	false	false	false
2	true	true	false	false	false
3	true	false	true	false	false
4	true	true	false	true	false
5	true	false	false	false	true

pipe N outer product mod = 0

A の列の意味は、1 ~ 5 の値を 1 で割った余りが 0 かどうかを表しています。これはすべて割り切れるので全部 true です。B の列の意味は、1 ~ 5 の値を 2 で割った余りが 0 かどうかを表しています。この場合、2 と 4 の時だけ true です。他の C, D, E ではどちらも同じ値の時だけ true です。

素数は 1 と自分自身でしか割り切れないものなので、その行の true の合計が 2 のものということになります。

配列で、各行の要素の合計を求めるのが combine in rows ブロックです。

combine in rows (reduce by column vectors) +

true と false についても、1 と 0 として合計してくれるようです。

1	1	-
2	2	-
3	2	-
4	3	-
5	2	-
+	length: 5	

pipe N outer product mod = 0 combine in rows (reduce by column vectors) +

行中の true の要素の合計が 2 のものを map を使って求めます。

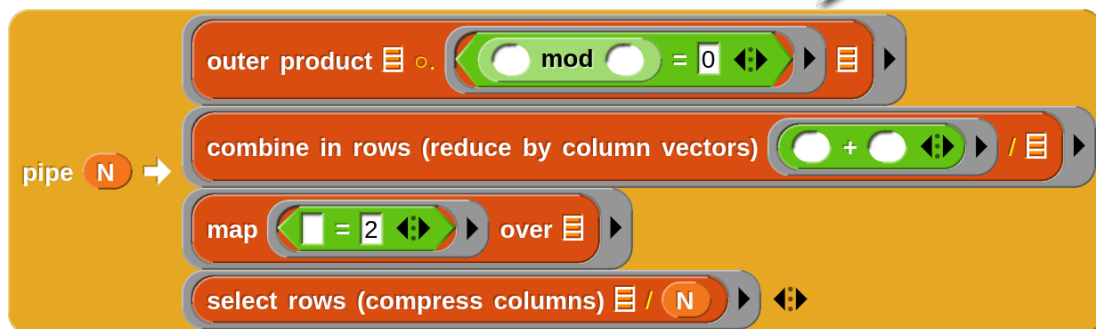
1	false
2	true
3	true
4	false
5	true
length: 5	



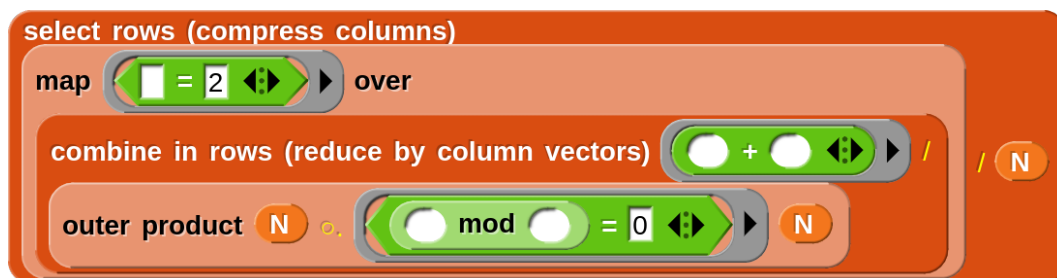
2, 3, 5 の位置が true になりました。その位置に対応する N の要素のリストを求めます。
select rows ブロックはリストの中から true で指定した位置の要素を集めてリストにします。
これにより、素数のリストが求められます。

select rows (compress columns) / N

1	2
2	3
3	5
length: 3	



次のようにまとめることもできます。



2 Continuation 継続

「Continuation 継続」は、プログラムの実行過程でその後に行われる処理と説明されます。Snap! では継続を目で見ることができるので、すこし理解しやすいかもしれません。

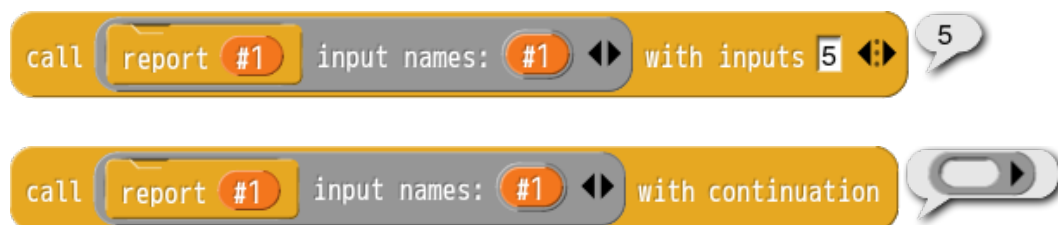
2.1 run と call の with continuation バージョン

Snap! の Continuations と Iteration, composition のライブラリーには、run with continuation と call with continuation があります。

report 1 を、ただの call と call with continuation ですと、結果は同じになります。

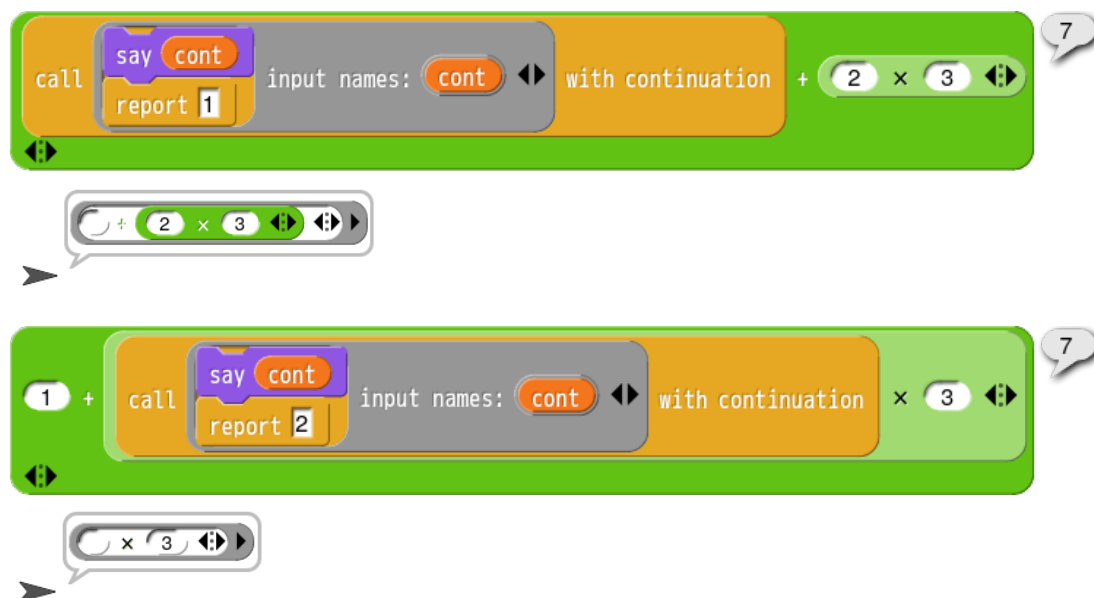


しかし、リング端の三角をクリックしてフォーマルパラメーターを使用してみると、call with continuation の方は空のリングが表示されます。



call with continuation のフォーマルパラメーターにはこのブロックに継続するスクリプトがセットされます。この場合はこの後に処理すべきものが何もないので空です。

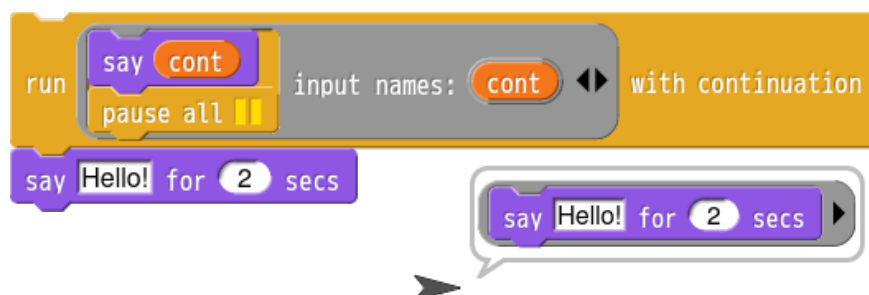
$1 + 2 \times 3$ の各スロットにこれをセットして、その位置での継続を表示させてみます。それぞれその時点での継続、その後に処理すべき内容がセットされます。(フォーマルパラメーターを cont にリネームしています。)



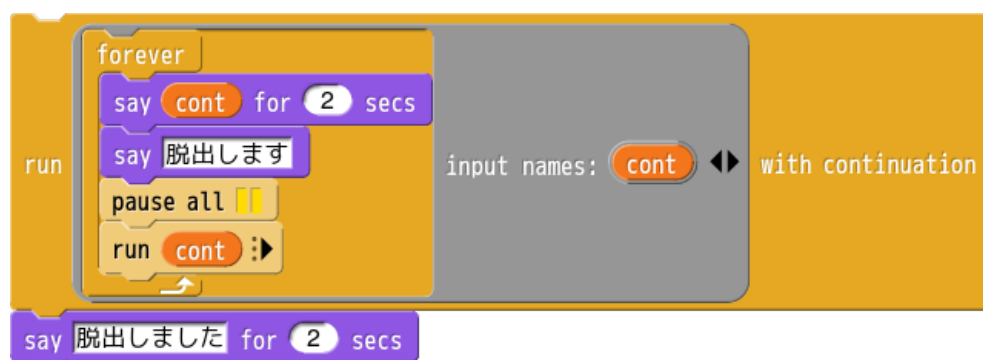


用途としては、call with continuation や run with continuation のリング内部にあるスクリプトで、ある条件の時にその外（継続）への脱出に使われます。

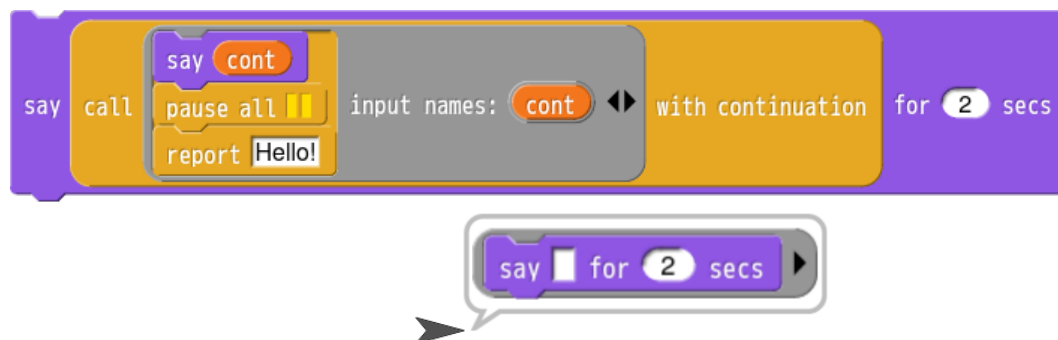
このスクリプトを実行すると、run with continuation の継続が表示されます。



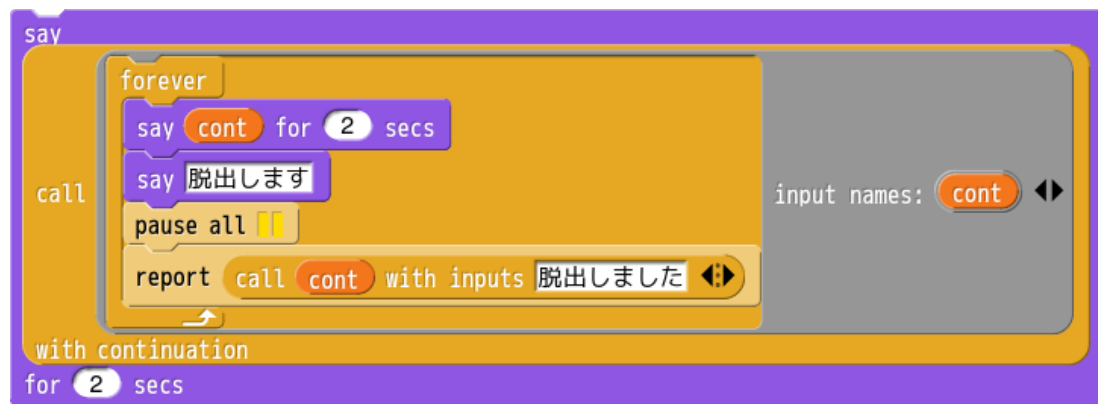
say や pause all が置かれているところにループを置き、その中で run cont とすると、run with continuation の中から外へジャンプすることができます。



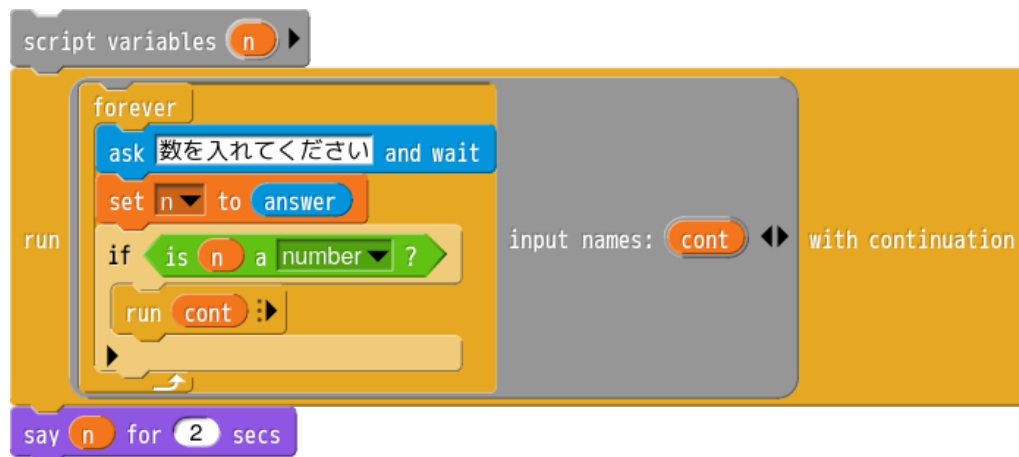
このスクリプトを実行すると、call with continuation の継続が表示されます。自分自身が入っているので、say の入力为空のブロックが表示されます。



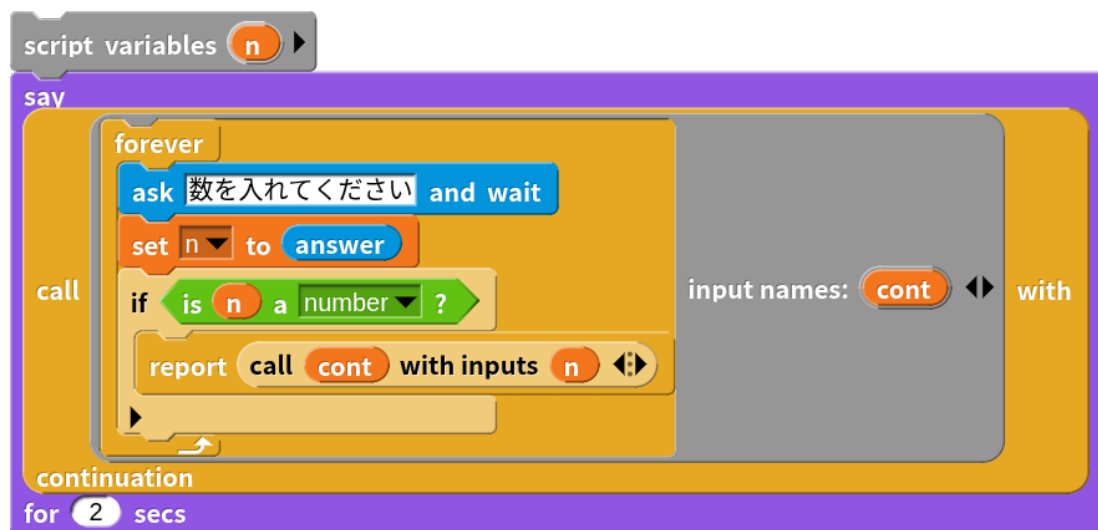
say や pause all が置かれているところにループを置き、その中で call cont とすると、call with continuation の中から外へジャンプすることができます。call with continuation はリポーターブロックなので値を返さなければなりません。call cont とすることで脱出になりますが、with inputs で値を指定することでその値「脱出しました」を返すことができます。



ask を使って数値のみを受け付けるスクリプトを作ってみます。run with continuation 版です。

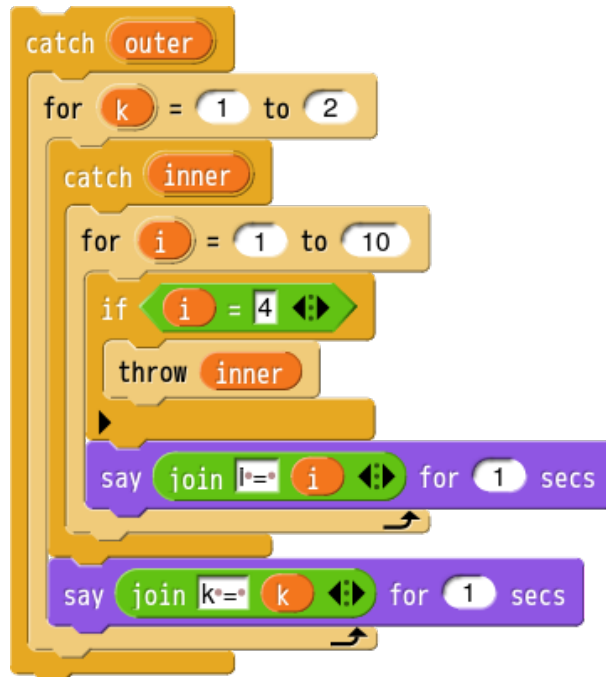


call with continuation 版です。



2.2 catch と throw

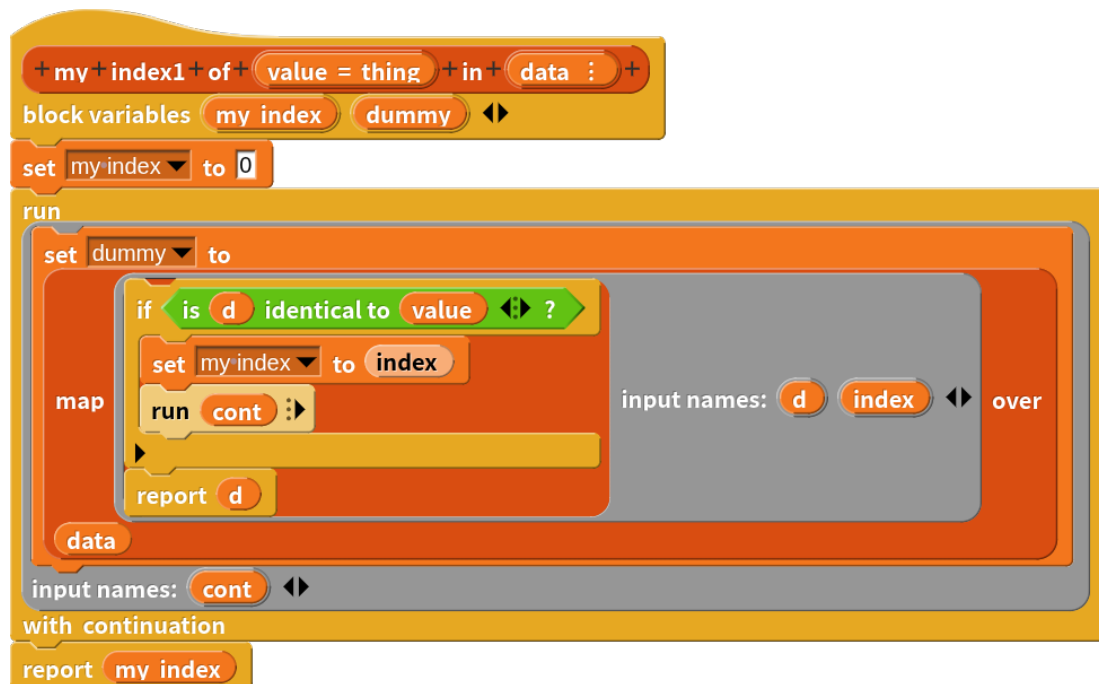
継続を意識することなくループなどからの脱出に使用できるものが、Libraries の Iteration, composition にある catch と throw です。



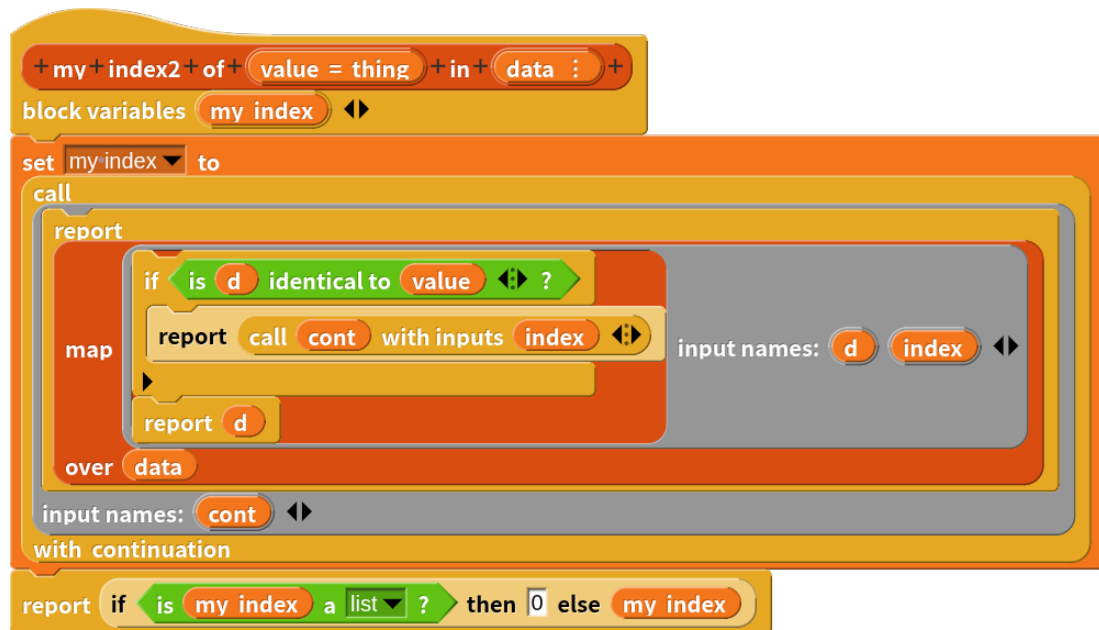
これを実行すると、i の値が 4 になった時に内側のループから脱出し、catch inner の次のスクリプトに進みます。throw outer にすると、外側のループから脱出し、catch outer の次のスクリプトに進みます。ある条件の時に、throw で指定したラベルの付いた catch ブロックの外側に脱出するという仕組みです。

2.3 map からの脱出

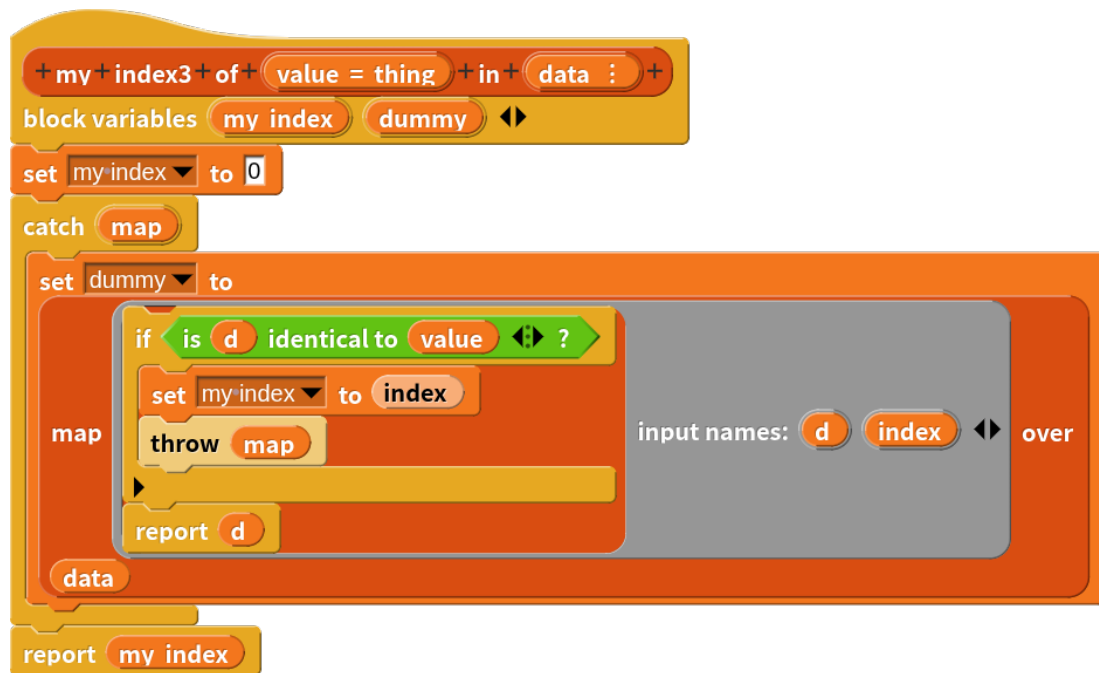
map を使ってリスト操作する場合、基本的にはすべての要素を操作します。index を求めるのに run with continuation を使用すると、要素が見つかった時点で脱出することができます。run with continuation 版では map を使用するにはダミーの変数 dummy が必要です。



call with continuation 版です。map 操作の結果は、見つかった場合はインデックス (number タイプ)、そうでない場合はリストのコピー (list タイプ) になります。結果が list タイプ ならば 0 を返します。

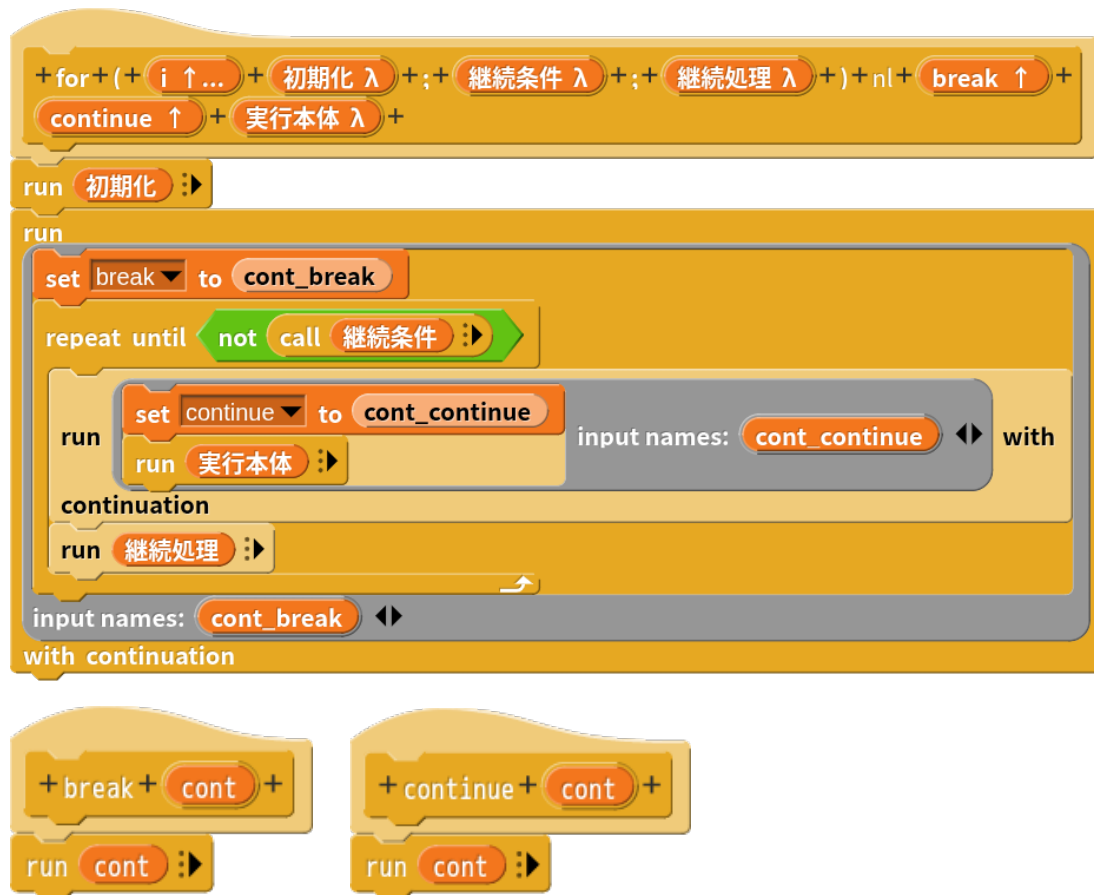


catch throw 版です。

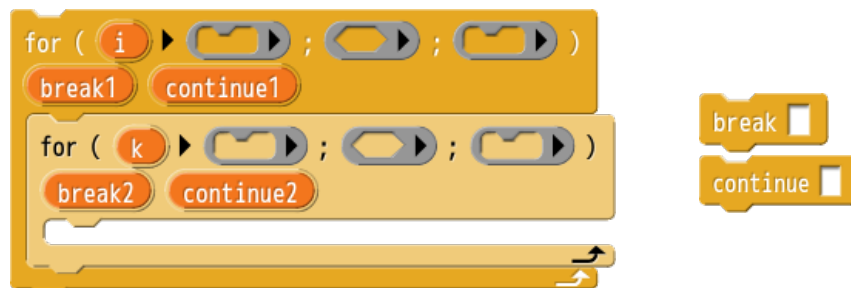


2.4 break と continue

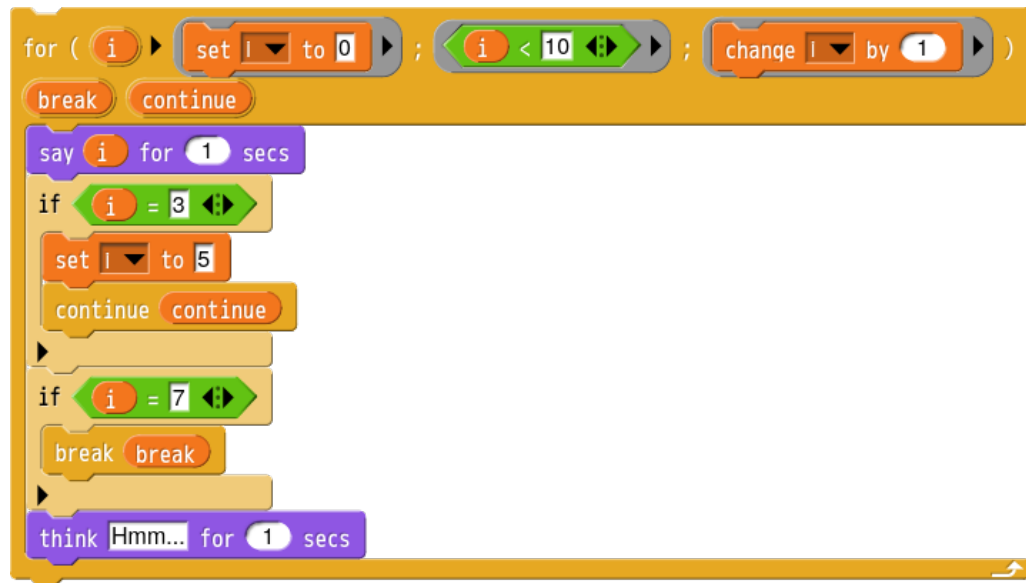
継続の機能を利用すると、C 言語風 for (; ;) ループに break や continue の機能を持たせることができます。break はループから脱出する機能で、continue は「継続処理」へジャンプして次の繰り返しを開始する機能です。紛らわしいですが、ここで入力変数名に使われている「継続」はループ処理を続けるという意味で、continuation に関するものではありません。変数 i は special から variables に設定します。break と continue 定義中の cont は Any type 変数です。



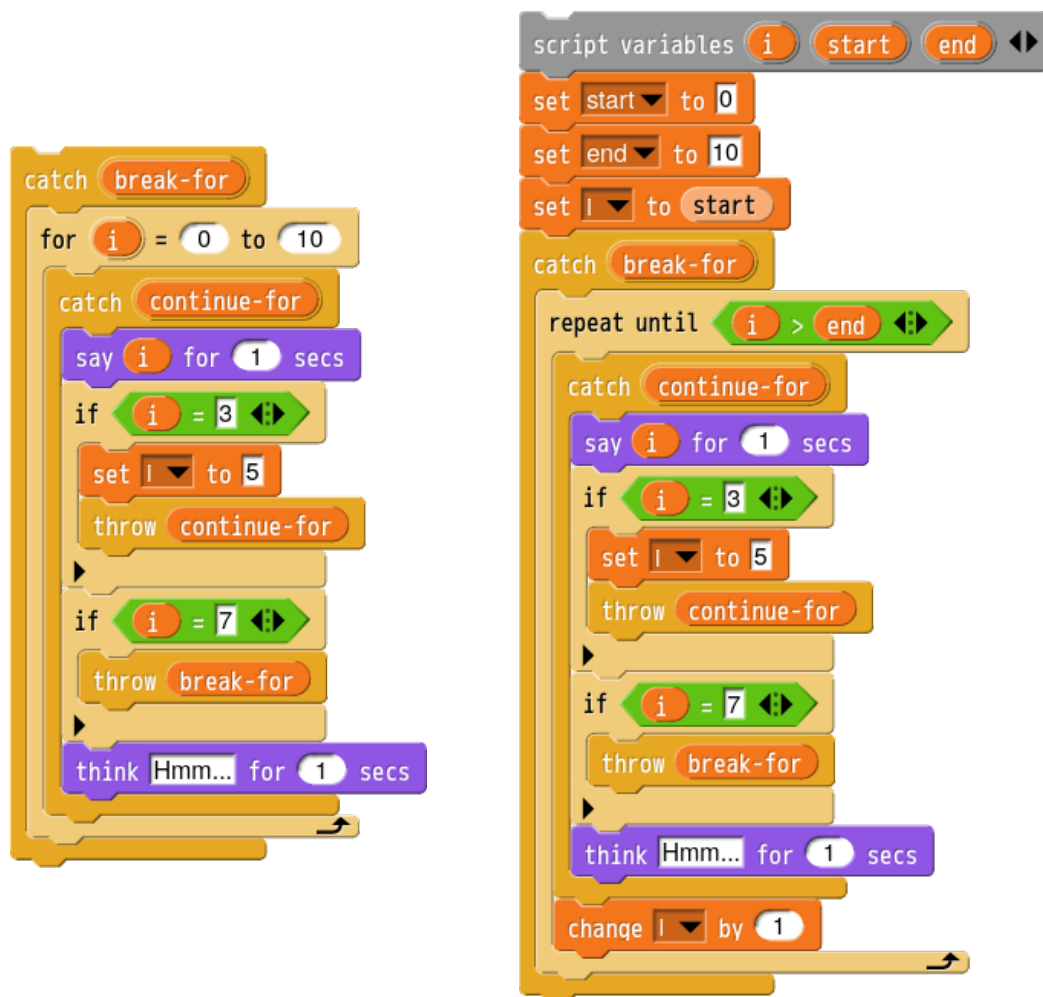
break と continue は、upvar オプションの変数です。for (; ;) ブロックから break, continue 変数をそれぞれ break や continue ブロックにドロップして使用します。二重三重のループの場合は、変数名に数字を追加するなどして区別してください。



次のスクリプトは、「0, Hmm..., 1, Hmm..., 2, Hmm..., 3, 6, Hmm..., 7」と順に表示するものです。3 で値を 5 にして continue、7 で break です。



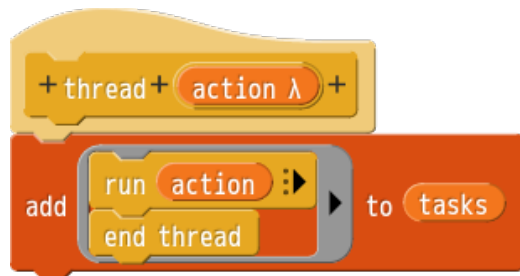
同じことをする、プリミティブの for と repeat until を使った catch と throw 版です。



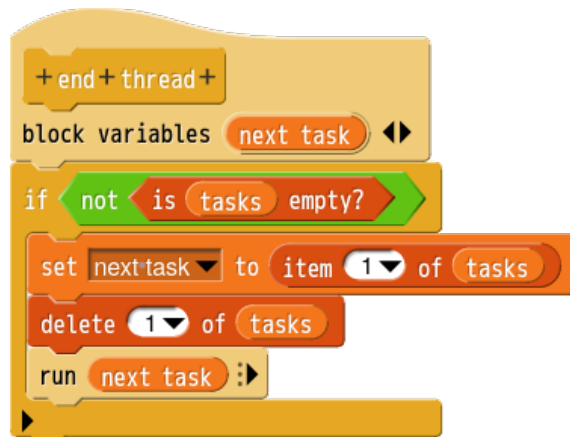
2.5 thread

Snap! では複数の **when clicked** が存在する場合、それぞれのスクリプトを並列に実行しているように見せています。継続を使用すると、一つのスクリプト内にある複数の処理ブロックを並列に実行の順番や処理の移行のタイミングをプログラミングすることができます。並列処理されるそれぞれのスクリプトを thread (スレッド) と言います。

処理するスクリプトの記憶場所として **tasks** の変数を作成しておきます。以下に示す 3 個の定義ブロックはスレッド処理を行うための 3 点セットです。



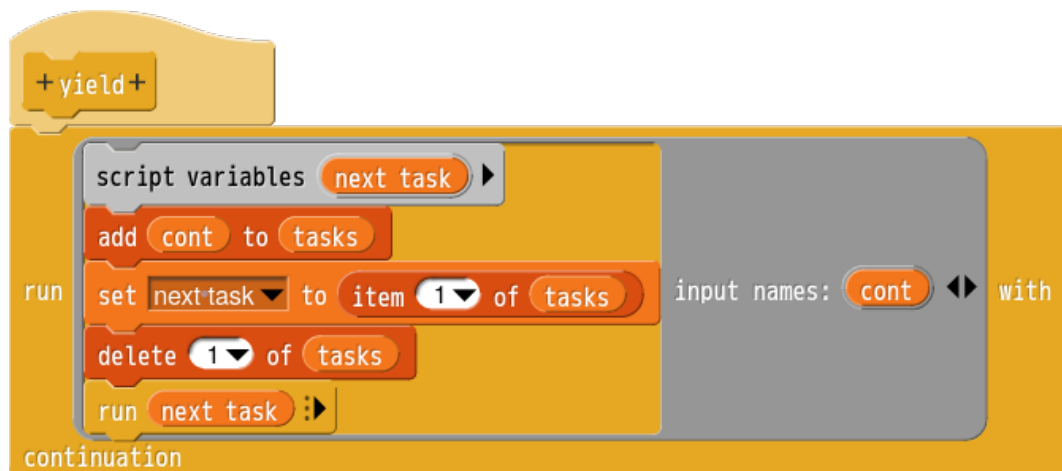
スレッドを設定するブロックです。(設定されたスレッドを tasks リストに加えていきます。) action は Command(C-shape) 型です。



tasks リストが空でなかったら tasks リストの先頭のスクリプトを取り出して実行します。これ以上スレッドが無いことを示します。これによって設定された複数のスレッドの実行が始まります。つまり、これを置かないとスレッドが実行されません。

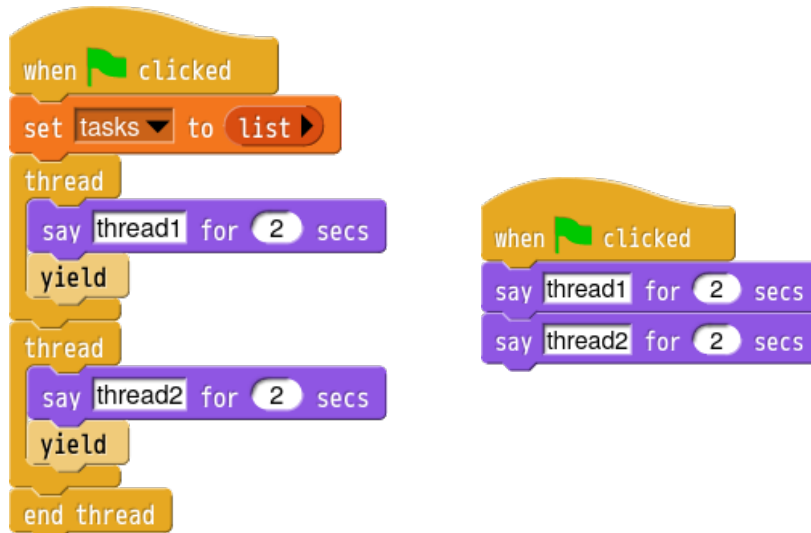


yield ブロックは、継続を tasks リストに加え、tasks リストの先頭のスクリプト (スレッド) を取り出して実行します。つまり、このブロックがこの thread の実行と次の thread への切り替えるタイミングを示します。



thread ブロックでスレッドを設定して (yield ブロックを挿入することで処理の移行のタイミングを指定します)、スレッドの並びの終わりを示す end thread ブロックを置くというプログラミングになります。継続などを意識しなくても定型的な操作でスレッド処理ができます。

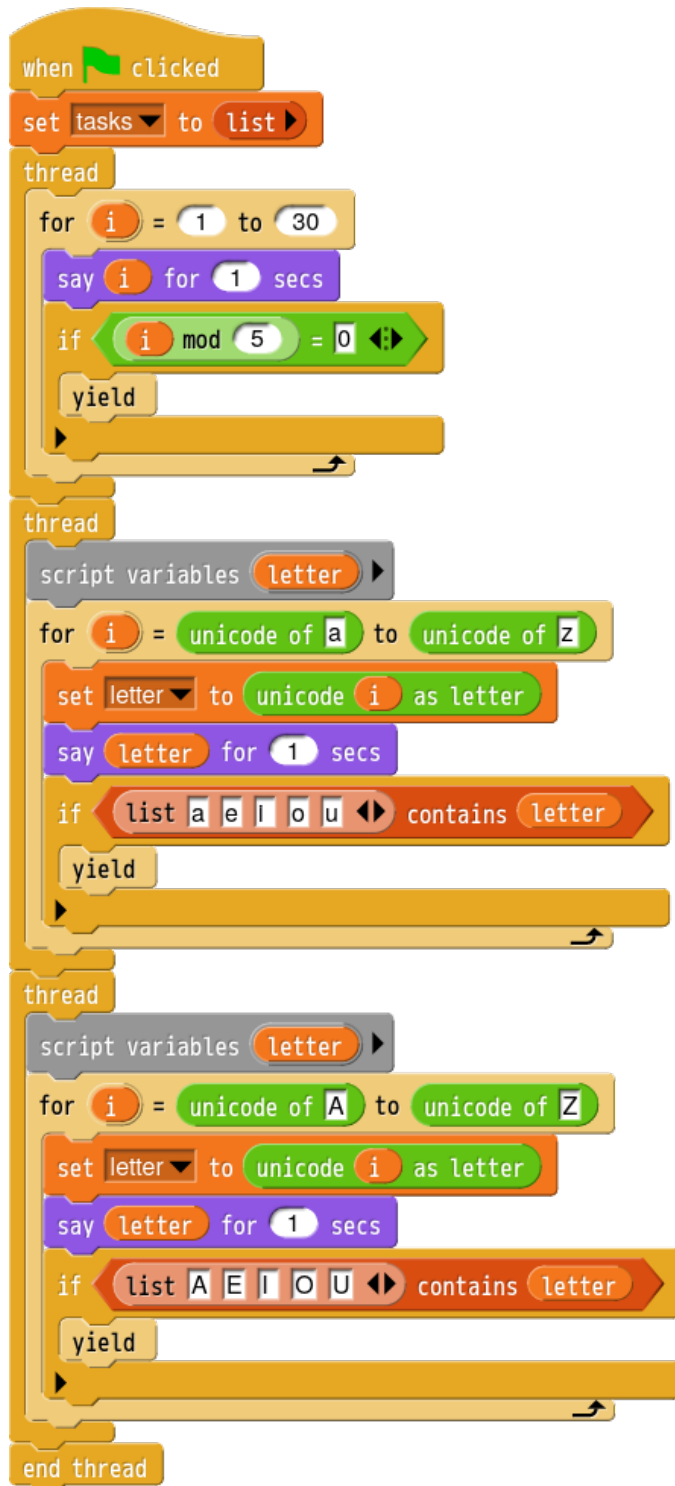
次のスクリプトは、右のスクリプトと同じことなのであまり意味はありませんが、使い方の例です。



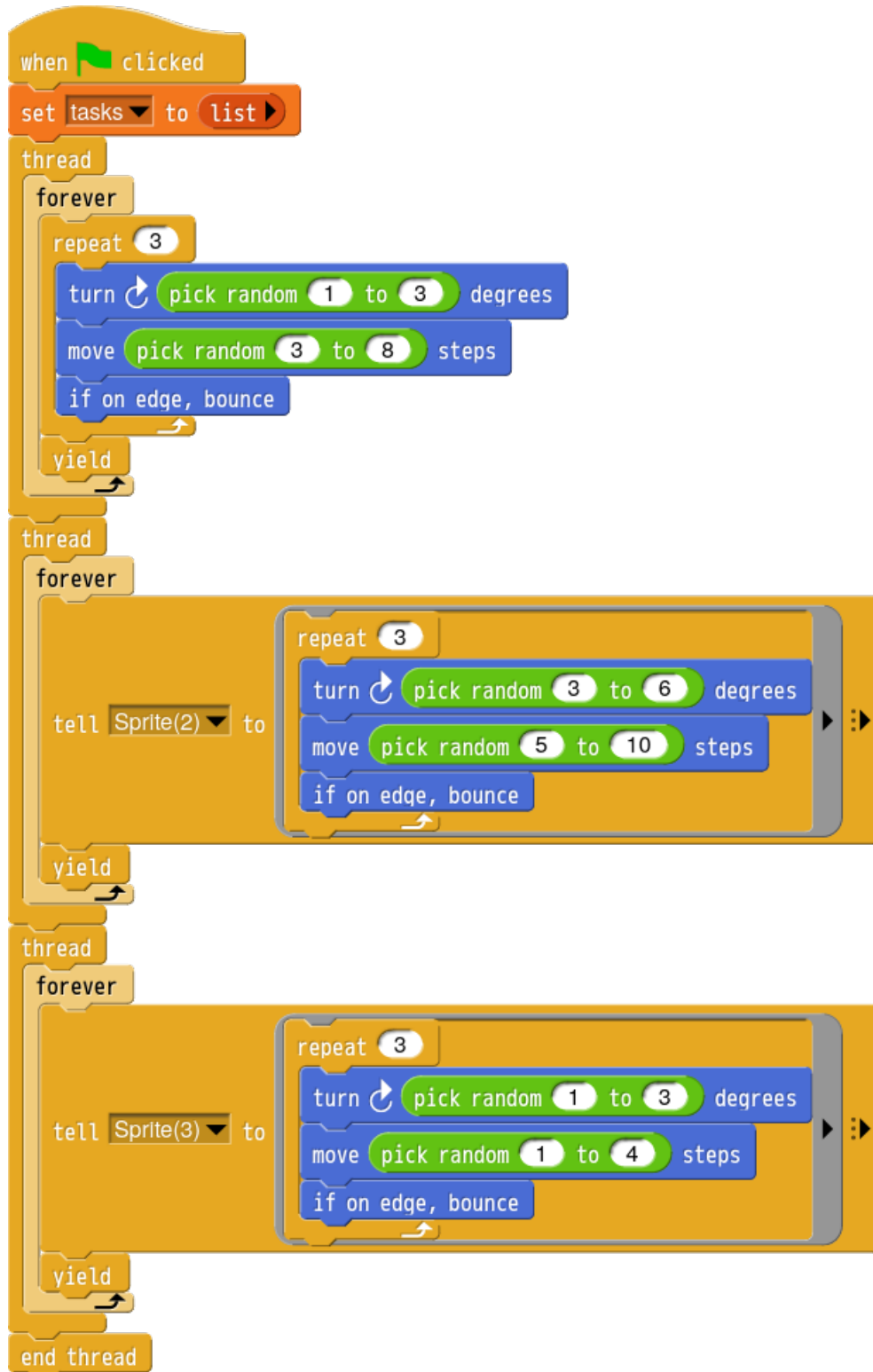
番号を表示するだけの 3 個のスレッドを実行してみます。



マニュアルに掲載されているスクリプトを基にした例です。指定の条件の時に次のスレッドに移行します。最初のスレッドは 1 から 30 までカウントしますが、5 で割り切れる値の時に次のスレッドに移行します。次のスレッドでは a から z までを表示しますが、a, e, i, o, u の時に次のスレッドに移行します。次のスレッドでは A から Z までを表示しますが、A, E, I, O, U の時に次のスレッド（最初のスレッド）に移行します。この操作がそれぞれのスレッドが終了するまで続けられます。



次のスクリプトはスプライトを 3 個使います。それぞれ Sprite, Sprite(2), Sprite(3) という名前になっているとします。スプライトをすこし回転させ前進させることを 3 回行います。thread を使わなくてもそれぞれのスプライトに同じような動きをさせることはできますが、thread を使うと各 thread の実行の順番や処理が切り替わるタイミングが制御できます。このスクリプトは Sprite 用で、他のスプライトにスクリプトは必要ありません。



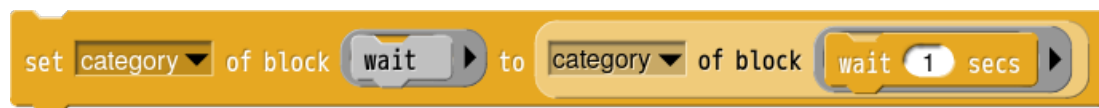
3 カスタムブロックの生成

カスタムブロックを作成するには、[Make a block] を使用しますが、スクリプトで作成することもできます。wait というブロックを Command パレットにある  で作成してみます。



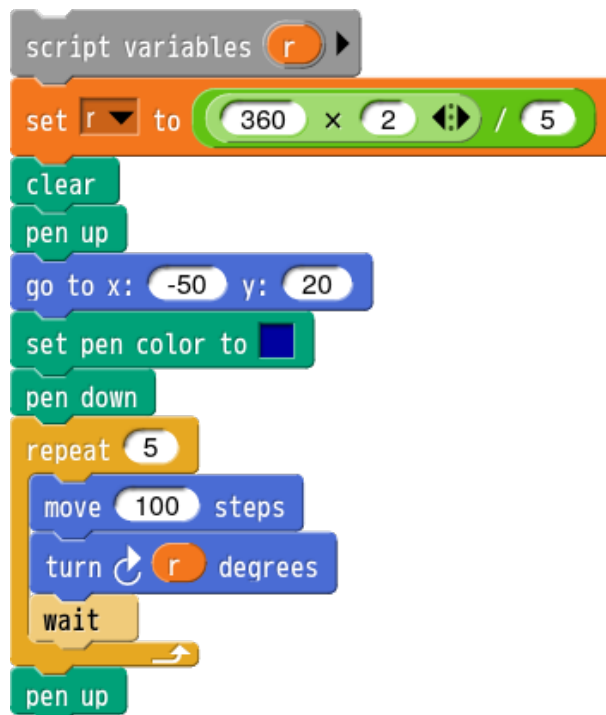
これをクリックすると、Variables パレットのところに  ブロックが出現します。

define ブロック単体でしたので、置き場所 category はデフォルトの Variables のパレット、形 type はデフォルトの Command 型になりました。次のようにすると、置き場所を Command パレットにすることができます。



そうすると、置き場所が変わり  と、Command パレットの色になります。

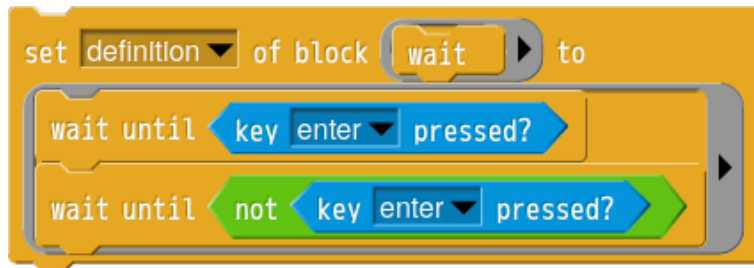
これを使って次のようなスクリプトを実行すると、1 辺を描くごとに 1 秒待つスクリプトになります。



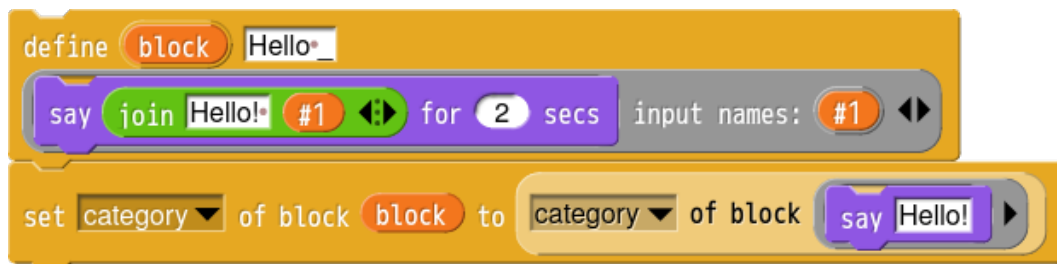
作成したブロックの動作を変更することもできます。次のようにすると、時間よっての wait ではなく pause によっての wait になります。  にすると、wait なしになります。



次のようにすると、enter キー待ちの wait になります。



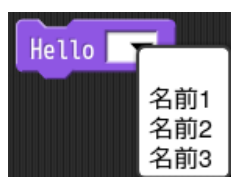
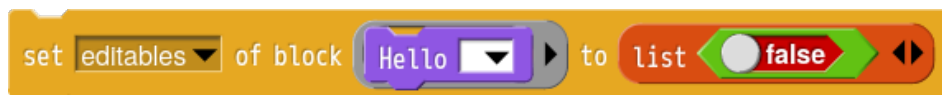
wait ブロックでは、define ブロックで「wait」と名前、ラベルを指定したのでそのような表示になりました。ここに空白で区切った「_」アンダースコアを置くと、そこに入力スロットが配置されることになります。入力スロット 1 個の Hello ブロックを作成します。



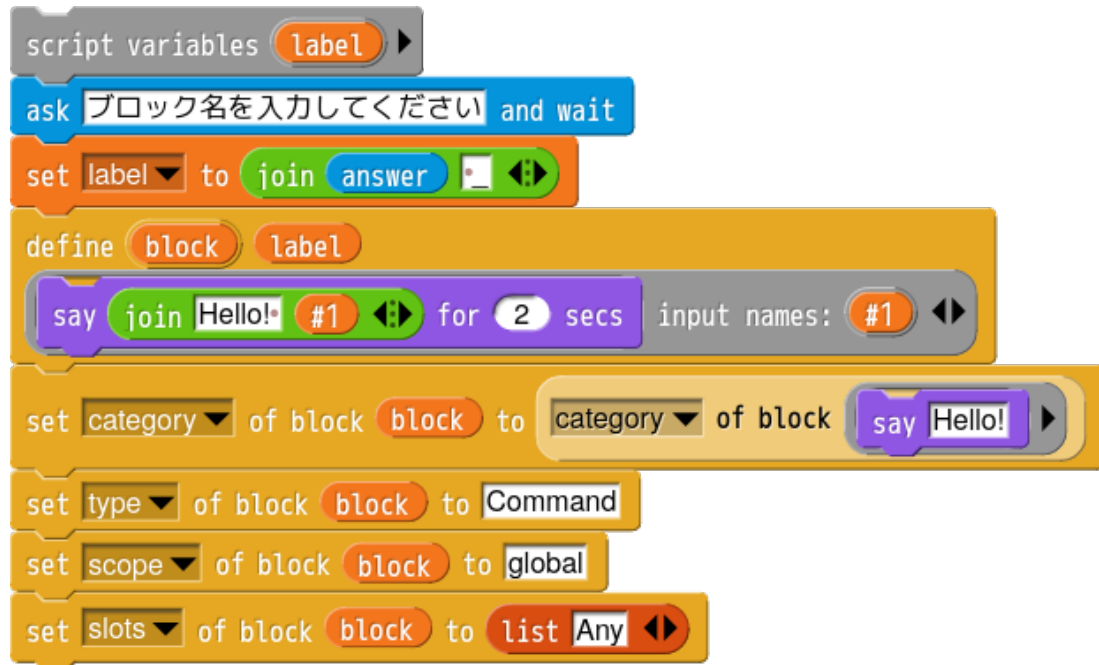
今回は category も設定しているので Looks パレットのところに Hello ブロックが出現します。block は Upvar 変数で、作成される Hello ブロックを表します。set category of block () のところにそれをドロップします。category の指定には、motion, looks, sound, ... と、直接パレット名を指定することもできます。Looks や LOOKS のように大文字での指定も可です。入力スロットから受け取ったものはフォーマルパラメータで参照できます。入力スロットの型はデフォルトが Any になっています。次のようにすると型を Text にすることができます。Number, Command, Text, Any, Reporter, List, Boolean, Predicate が指定できます。(小文字も可)



次のようにすると入力スロットを選択可能にすることができます。さらに、メニュー選択のみにすることもできます。



作成したブロックを  で削除してから次のようなスクリプトを実行すると、入力したテキストを名前、ラベルにしたカスタムブロックを生成することができます。type の指定は、[Make a block] する時に最初に選択するブロックの形の指定 Command, Reporter, Predicate です。



実行すると、

ブロック名を入力してください

Hello!

✓

できたブロック  を実行すると、



となります。

type と slots はデフォルトと同じなので助長ですが、scope では global と local が指定できます。

local 設定にすると、 このスプライトだけで有効なブロックになります。

スクリプトでカスタムブロックを作ることができるので、用途に応じたブロックを自動的に作成することができます。

例えば、[名前、身長、体重] のデータリストを作るならば、

A Scratch 'add' block with 'list' selected. The input field contains '一郎 170 65' and the target is 'data'.

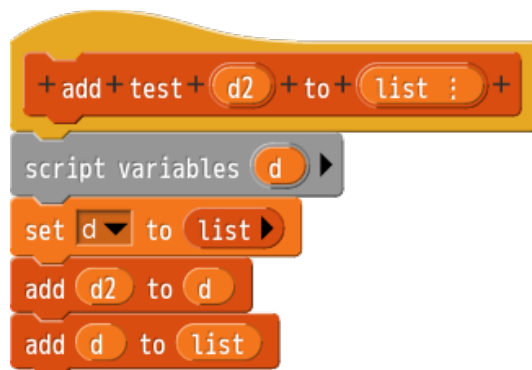
[名前、国語、算数、理科] のテストの成績データリストを作るならば、

A Scratch 'add' block with 'list' selected. The input field contains '次郎 80 75 85' and the target is 'data'.

のようにしてリストを追加していきますが、「addData 名前: 身長: 体重:」や「add 成績表 名前: 国語: 算数: 理科:」のように指定することで必要なブロックを生成することができます。二番目以降のスペースで区切ったデータが入力スロットの項目と位置を表します。

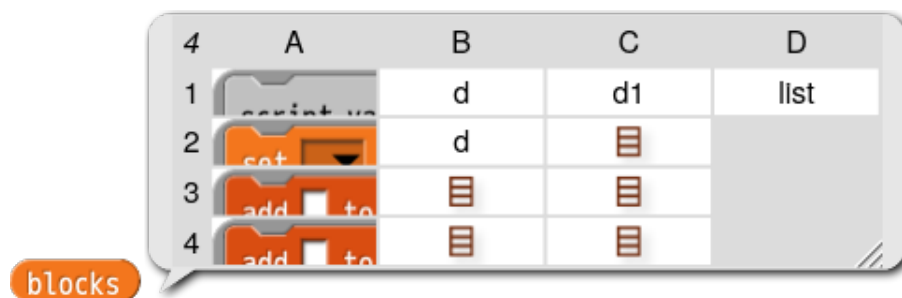
A 'generate add data block' block with the text 'addData 名前: 身長: 体重:'.A Scratch 'addData' block with '名前: 身長: 体重:' in the input field and '目' as the target.A 'generate add data block' block with the text 'add成績表 名前: 国語: 算数: 理科:'.A Scratch 'add成績表' block with '名前: 国語: 算数: 理科:' in the input field and '目' as the target.

基本となるのは一個のデータを追加するものです。

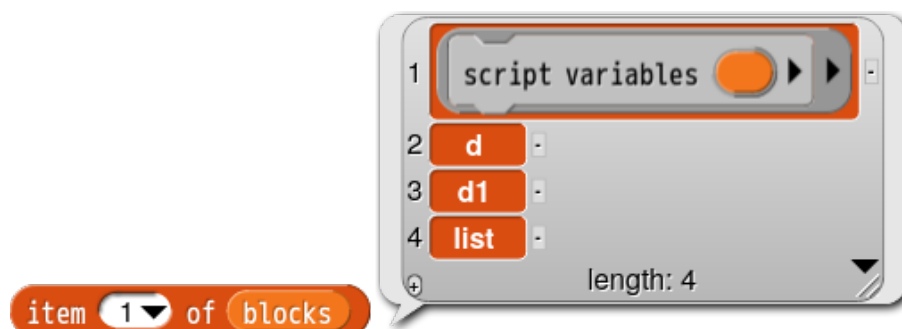
A small Scratch 'add test' block with '目' as the target.A Scratch script starting with a '+ add + test + d2 + to + list : +' block, followed by 'script variables d', 'set d to list', 'add d2 to d', and 'add d to list'.

この定義を definition と split でブロックに分解してみます。

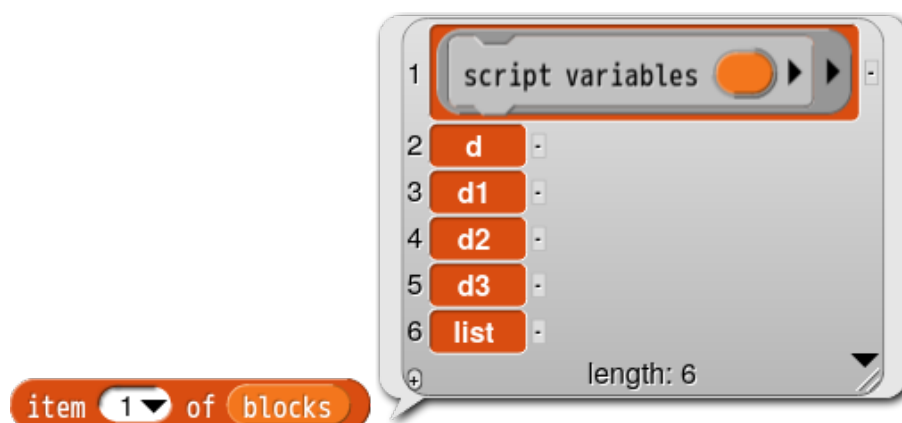
A Scratch 'set blocks to split' block. The input is 'definition of block add test 目' and the target is 'blocks'.



見づらいので細かく見ていきます。1 行目はスクリプト変数 d と入力スロット用の変数 d1 と list が指定されています。

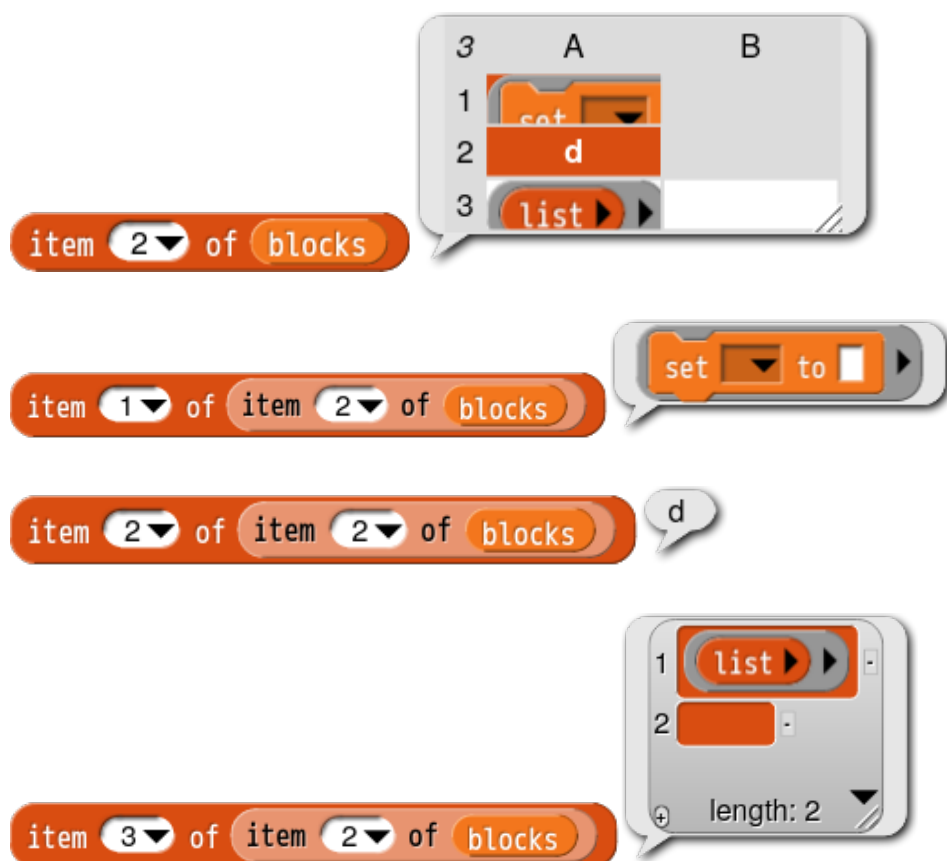


このカスタムブロックは変数名 d1 の入力スロットと、変数名 list のリスト型の入力スロットの二つの入力スロットになっています。もし d2, d3 と入力スロットを増やすならば、ここを

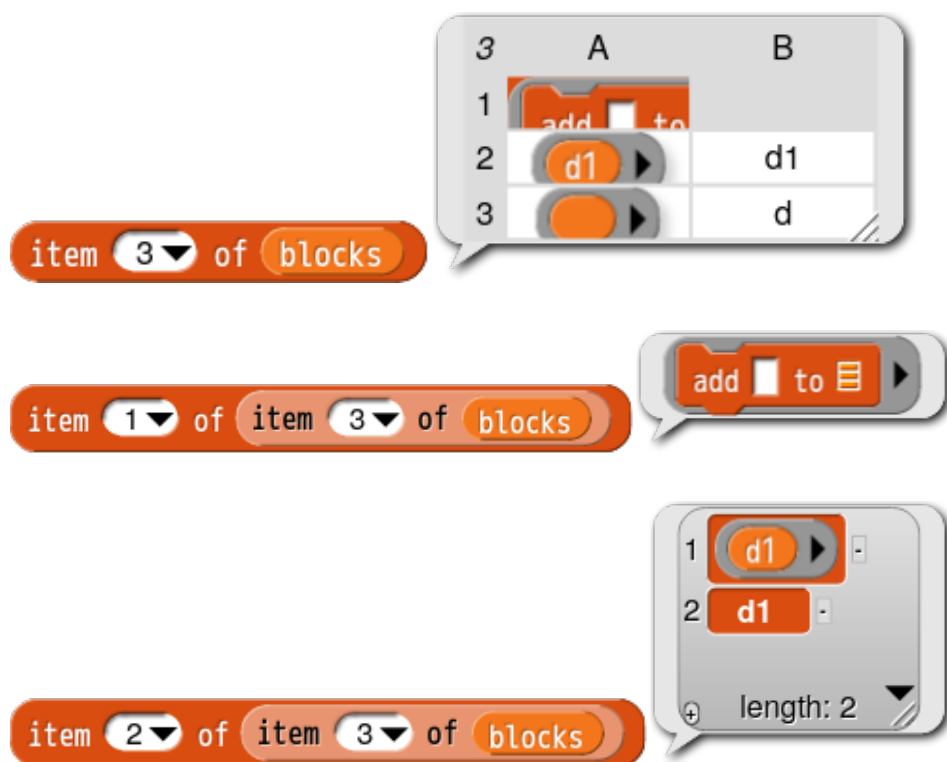


のように、リストに d2, d3 を挿入する必要があります。

2行目は **set d to list** の部分です。



3行目は **add d1 to d** の部分です。



もし d2, d3 と入力スロットを増やすならば、上の

item 3 of item 3 of blocks

add d1 to d

add d2 to d

add d3 to d

を挿入する必要があります。

4 行目は add d to list の部分です。

item 4 of blocks

	A	B
1	add to	
2		d
3	list	list

item 1 of item 4 of blocks

add to

item 2 of item 4 of blocks

1	
2	d

length: 2

item 3 of item 4 of blocks

1	list
2	list

length: 2

これが add test の definition です。これに基づいて自動生成用ブロックを作成します。

ここから、先に示した generate add data block を作っていきます。

generate add data block `addData` 名前: 身長: 体重:

指定されたデータ項目を基にラベル上に入力スロットの位置を配していきます。

名前: _ 身長: _ 体重: _

join input list: map join over split 名前: 身長: 体重: by

基になる `add test` の入力データ個数が 1 なので、そこに 29 ページと 31 ページで示したように、指定された個数になるように追加します。

1 script variables
2 d
3 d1
4 list

1 script variables
2 d
3 d1
4 d2
5 d3
6 list

から への変更と、

add d2 to d
add d3 to d の挿入です。

add d2 to d ブロックを挿入する場合、変数の指定を

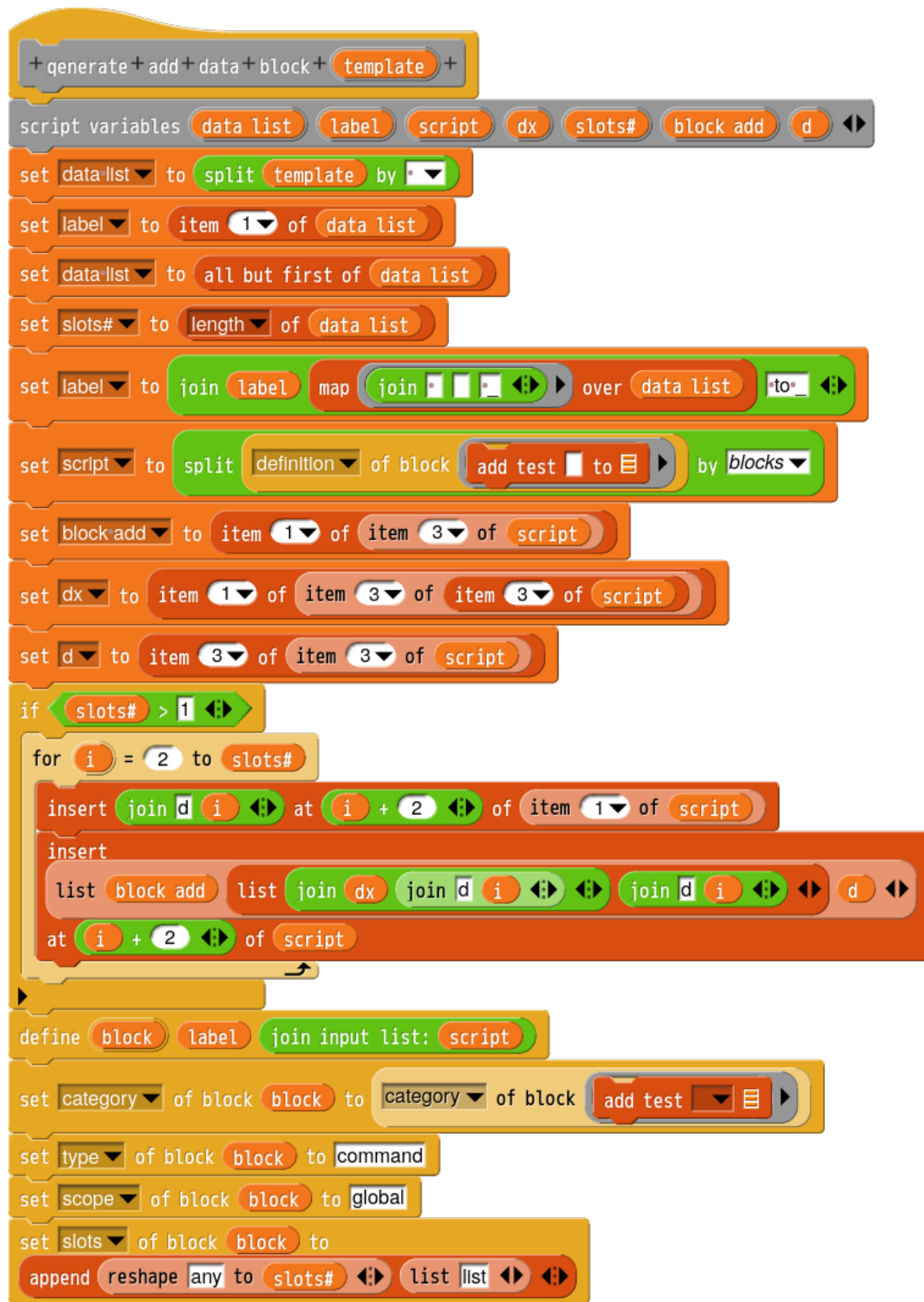
1 d2
2 d2

1
2 d2

としたいのですが、 となってしまう。ですが機能するようです。

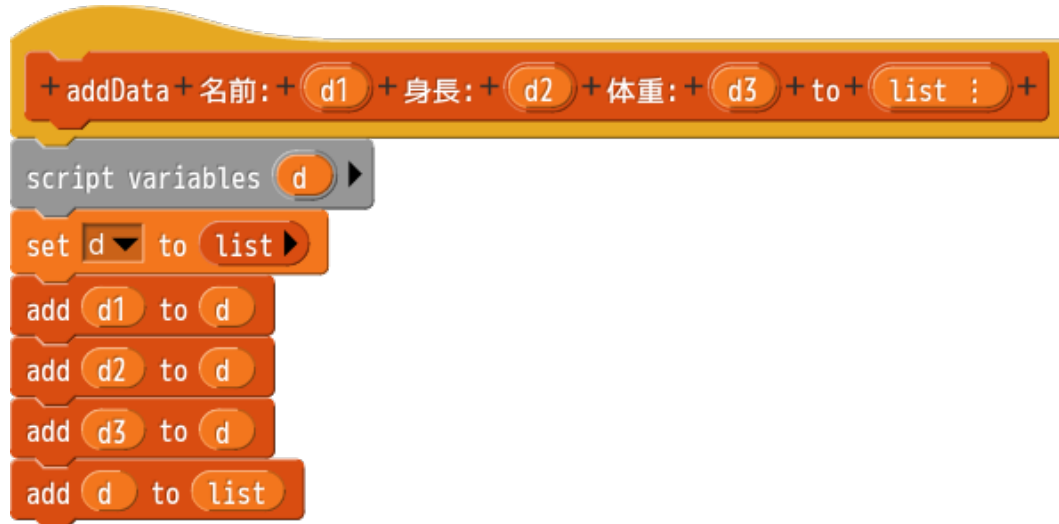
変数 label には入力スロットで指定されたデータからカスタムブロックに表示されるテキストと入力スロット位置がセットされます。

カスタムブロック **add test** の定義をブロックに分解して変数 script にセットし、それに対して修正します。



addData 名前: 身長: 体重: to 

このブロックは generate add data block `addData・名前:・身長:・体重:` で作成したのですが、右クリックでオプションから edit... で Block Editor を起動することができます。そうすると、このように変数名も問題なく設定されているようです。

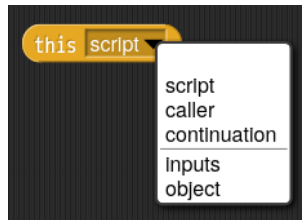


同様に、

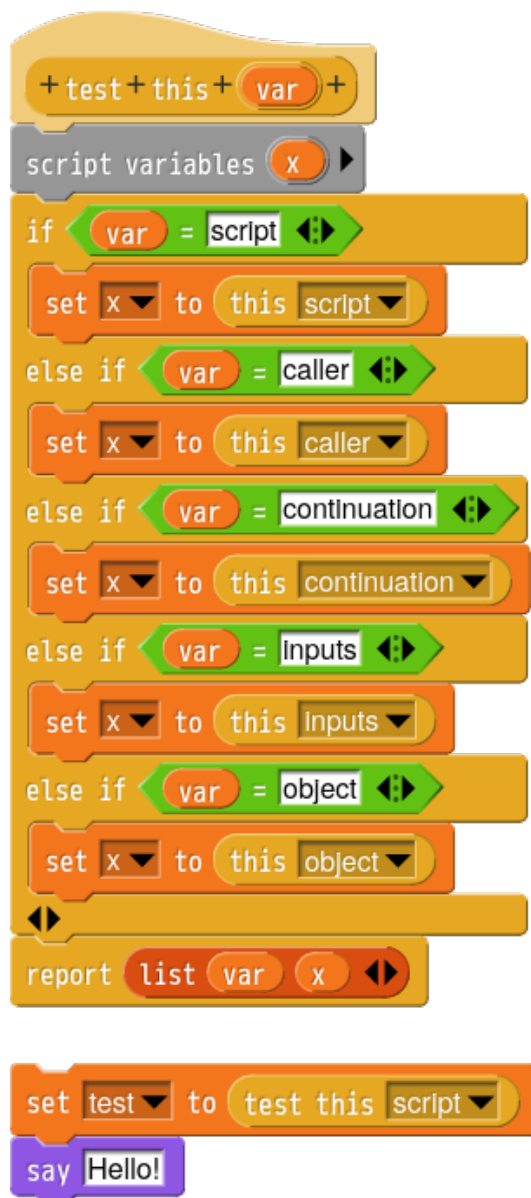


4 this ()

マニュアル (Ver.8) では Macros の項目で this script ブロックを使用しています。

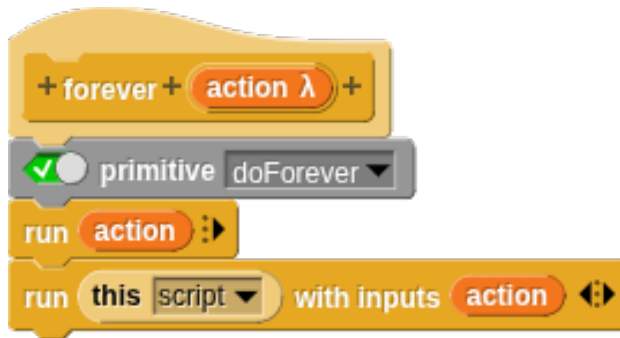


Snap! (Ver.12) 時点では となり、script 以外にも選択項目が増えました。次のスクリプトはこのブロックがリポートするものを示します。選択項目を options... と read-only で指定しています。

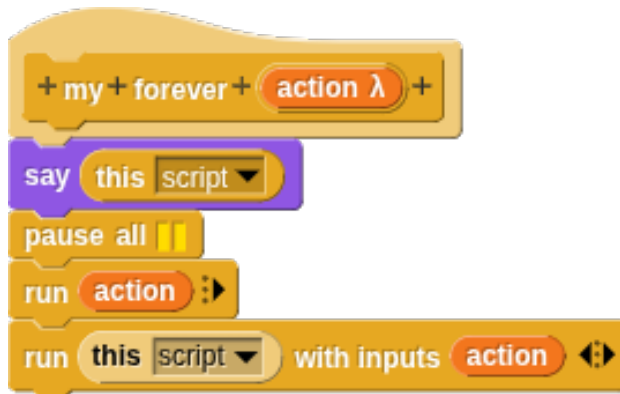


を実行することで変数 test に this () の内容がセットされるのでその機能を確認できます。

this () を使用している例として forever を見てみます。



これを基にして my forever を作成します。



これを使用するスクリプトです。

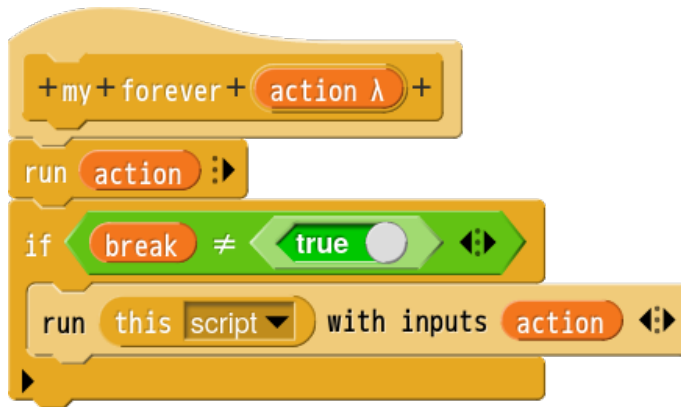


これにより表示される this (script) の内容です。

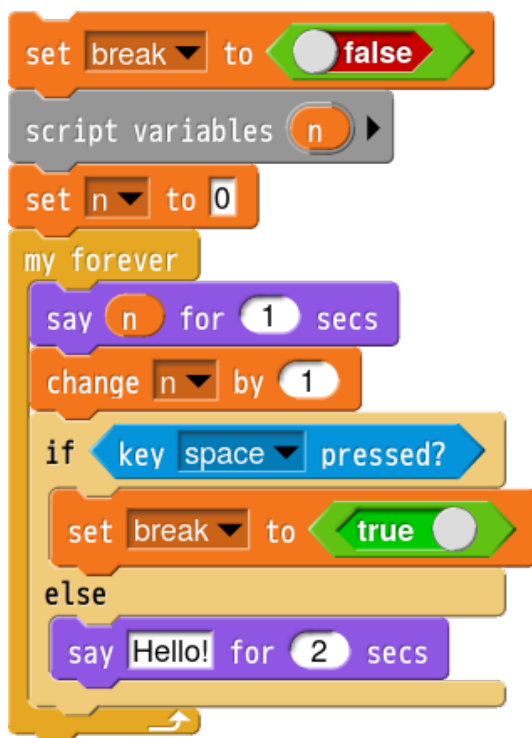


再帰呼び出しのように自分自身を実行することで無限ループを実現しています。

break 変数を作成し、my forever を次のように変更すると、

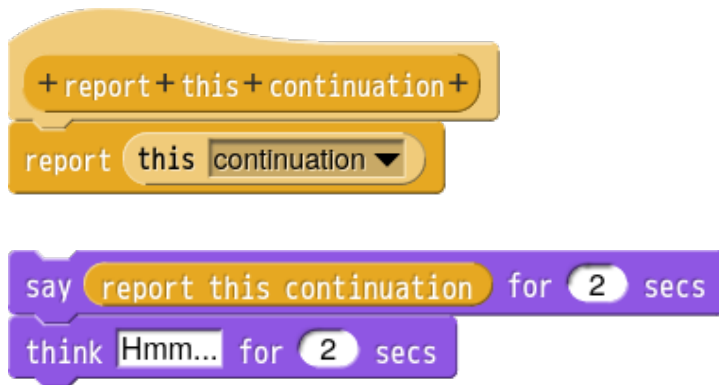


break により無限ループを止めることができます。次のループを行わないというだけなので、break 以降のスクリプトを実行しないような処置が必要です。

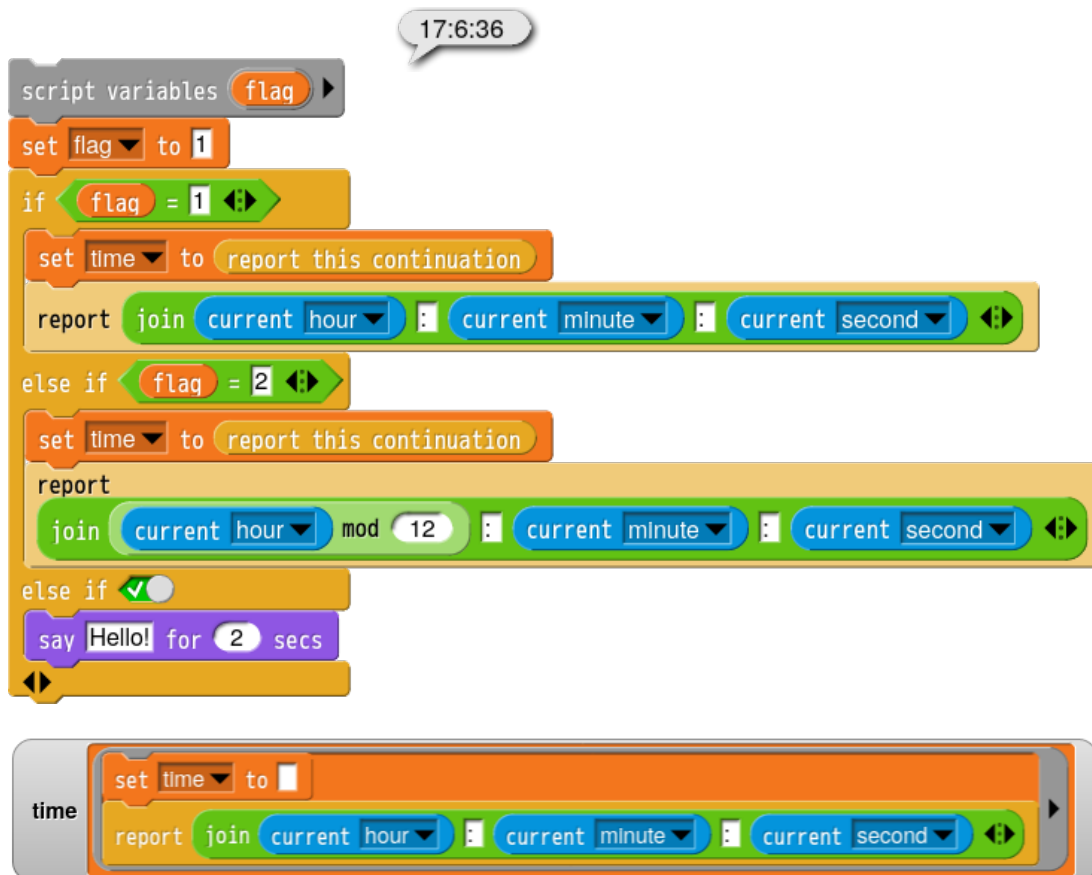


Snap! の利点は Smalltalk のようにある程度プリミティブの定義の内容が見られることです。Libraries にも参考になるものがたくさんあります。

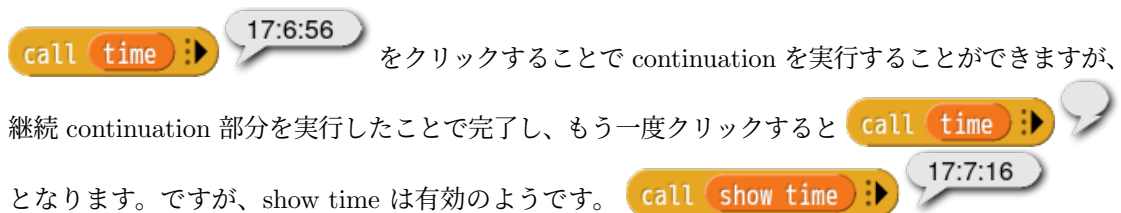
this continuation を使う場合、次のようなブロックが必要になります。



次のようにすると、変数 time に continuation がセットされます。



if ブロック内での continuation はその if の条件の範囲内のスクリプトに限られるようです。ここで、continuation を 'set show time to time' で保存します。



5 再帰呼び出し

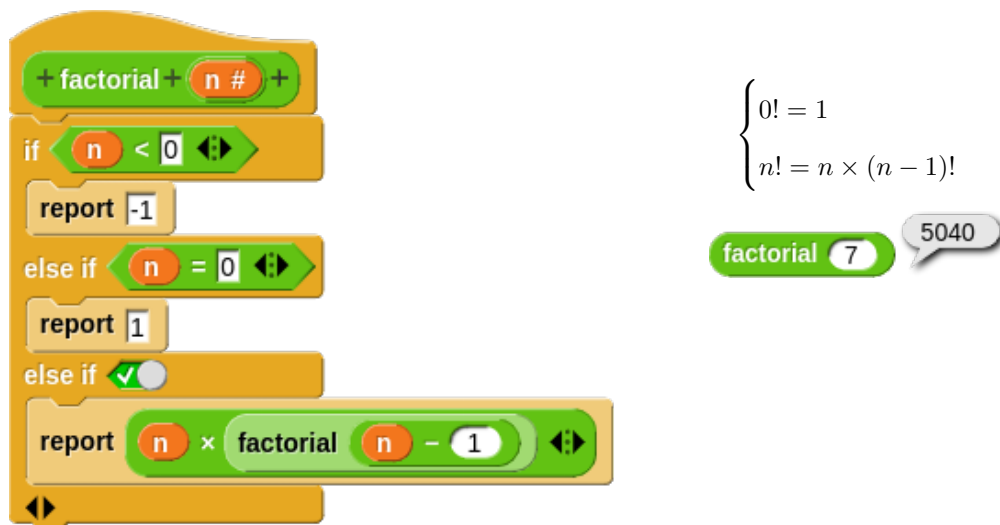
再帰、再帰呼び出し（recursive call）は作成したブロックの中で自分自身を呼び出す（実行する）ものです。関数型プログラミングでは再帰呼び出しが繰り返し処理の手法になっています。

5.1 再帰呼び出しの例

Scratch では値を返せなかったので、階乗やフィボナッチ数列はできませんでした。

5.1.1 階乗

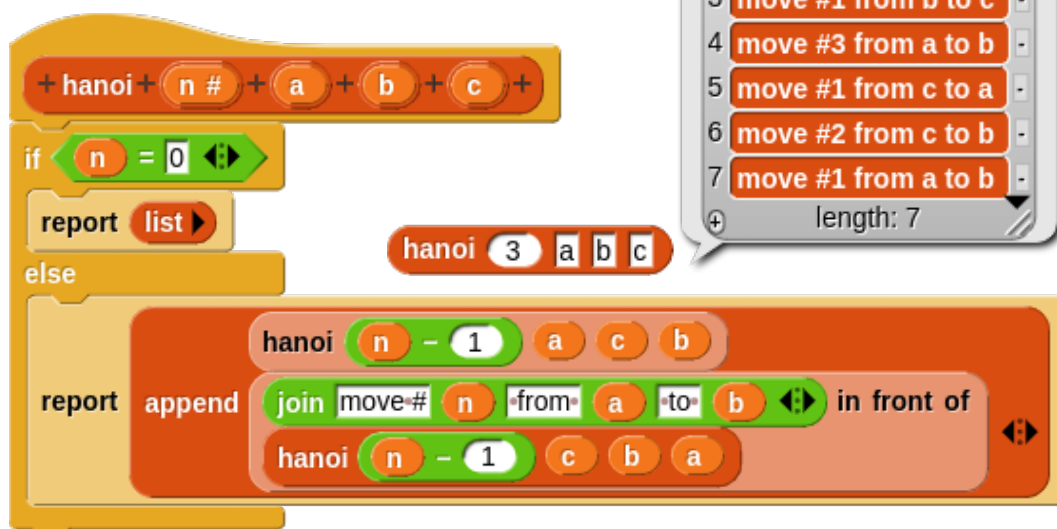
factorial 階乗は再帰呼び出しの例としてよく使用されます。



The image shows a Scratch script for a factorial function. The script starts with a block labeled '+ factorial + n # +'. It then enters an if-else structure. The first condition is 'if n < 0', which leads to 'report -1'. The second condition is 'else if n = 0', which leads to 'report 1'. The third condition is 'else if' (checked), which leads to 'report n x factorial n - 1'. To the right of the script, there is a mathematical definition of factorial:
$$\begin{cases} 0! = 1 \\ n! = n \times (n - 1)! \end{cases}$$
 Below this, there is a 'factorial 7' block with a speech bubble showing the result '5040'.

5.1.2 ハノイの塔

ハノイの塔のパズルも再帰呼び出しの例としてよく使用されます。



The image shows a Scratch script for the Hanoi Tower puzzle. The script starts with a block labeled '+ hanoi + n # + a + b + c +'. It then enters an if-else structure. The first condition is 'if n = 0', which leads to 'report list'. The second condition is 'else', which leads to a 'report append' block. Inside the 'report append' block, there are three sub-blocks: 'hanoi n - 1 a c b', 'join move-# n from a to b in front of', and 'hanoi n - 1 c b a'. To the right of the script, there is a list of moves for the Hanoi Tower puzzle with 3 disks: 1. move #1 from a to b, 2. move #2 from a to c, 3. move #1 from b to c, 4. move #3 from a to b, 5. move #1 from c to a, 6. move #2 from c to b, 7. move #1 from a to b. The list has a length of 7.

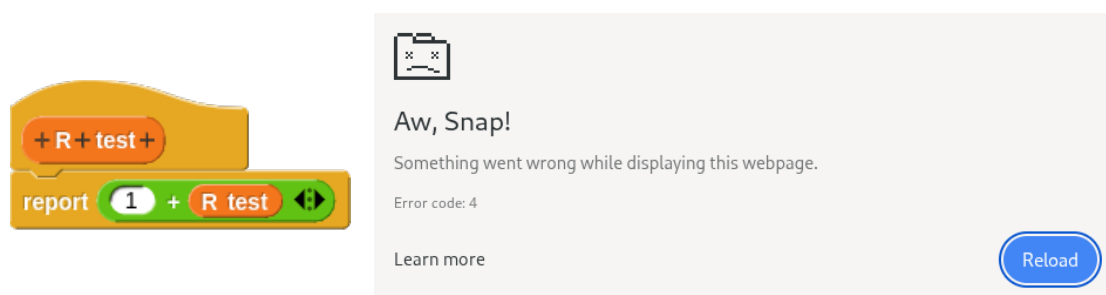
5.2 再帰呼び出しの使用

スクリプトの表示面積を小さくするために、3つのブロックを定義して使用します。

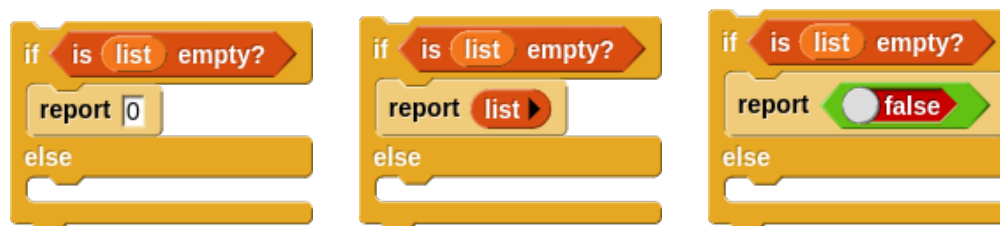


5.2.1 繰り返し

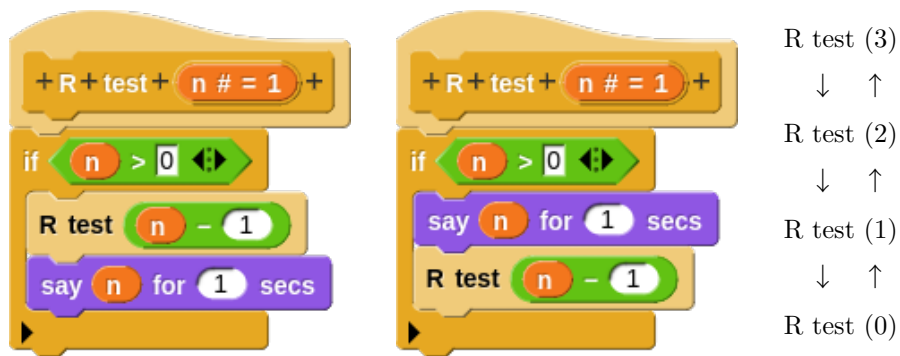
これはただ自分自身を呼び出すものです。(エラーになるので実行しないでください)



定義ブロックはどこかのスクリプトから呼び出されて実行されるわけですが、実行終了後に呼び出し元に帰る必要があります。呼び出し元のスクリプトの実行を継続するための情報を保存しておき、それを取り出してそこに復帰します。上記のスクリプトでは自分自身への無限呼び出しになり、情報を保存するための場所がなくなってしまいます。ただの無限ループというだけの問題ではありません。したがって、再帰呼び出しでは再帰呼び出しを終了するためのスクリプトが必要です。指定の回数繰り返す処理ならば、回数が0が終了条件です。リスト操作では、空リストが終了条件になります。リスト操作で Reporter 型のブロックを作成する場合、数値型では0を、リスト型では空リストを、Predicate 型のブロックを作成する場合は false を原則としてリポートして再帰を終了させます。空リストの場合は、**list** でも同じことです。



次は指定した回数だけ何かを行う定義ブロックです。n = 0 が終了条件になります。n の値を減らしながら以下の矢印(↓)のように自分自身を呼び出していきます。この定義ブロックは0以下だと何もしないで呼び出し元に戻ります。呼び出し元に戻るを繰り返し、一番最初の呼び出し元に戻ったら終了です。再帰呼び出しブロックの位置により n の値はカウントアップにもカウントダウンにもなります。



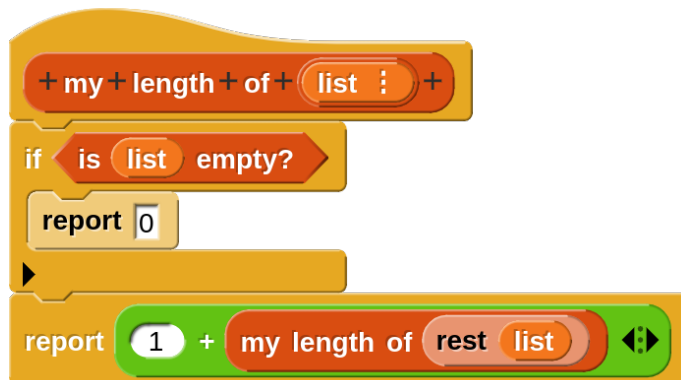
5.2.2 my length

リストの要素数を求める length ブロックを repeat until ブロックを使って作ってみます。要素数が 0 になるまで要素を一つずつ削除しながらカウントすることで求めます。

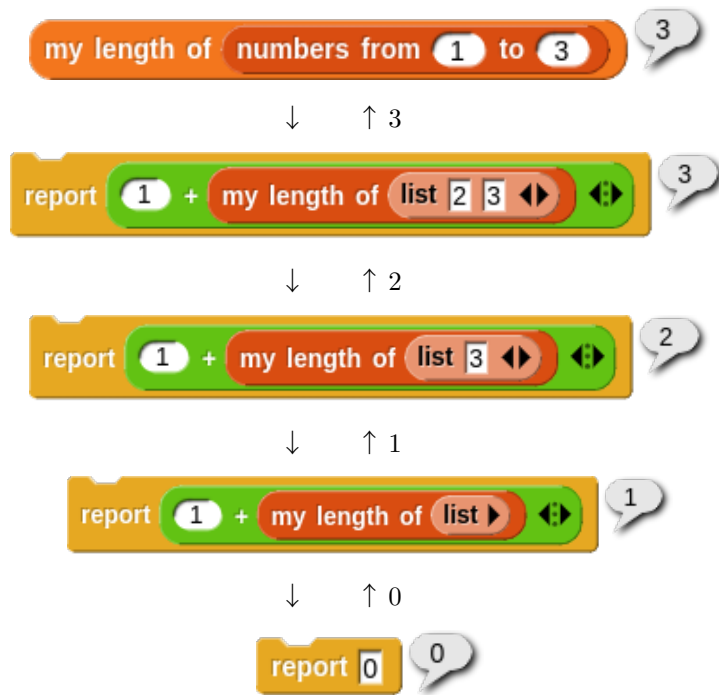


再帰版です。カウント用の変数が無いので理解しにくいですが、report が返す値がカウント用変数の役割を果たしています。`my length of rest list` でリストが空になるまで再帰呼び出しされて、0, 1, 2, ... と、report が返す値 +1 を積み重ねて、結果的に 0 からのカウントアップで要素数を求めることができます。自分自身を呼び出すことを除けば repeat until 版と同じやり方になります。

〔リストの先頭要素を処理する。2 番目以降のリストを引数として自分自身を呼び出す。〕ということをリストが空になるか条件が成立するまで、処理を行うのが再帰処理の基本です。この場合の処理は、レポートされた値（要素数）に 1 を加えているだけです。



実行される様子を見てみます。↓で示す順に再帰呼び出しが実行され、0 と値が確定すると、↑で示す順に返された値に1を加えて呼び出し元に値を返していきます。最終的に値は3になります。



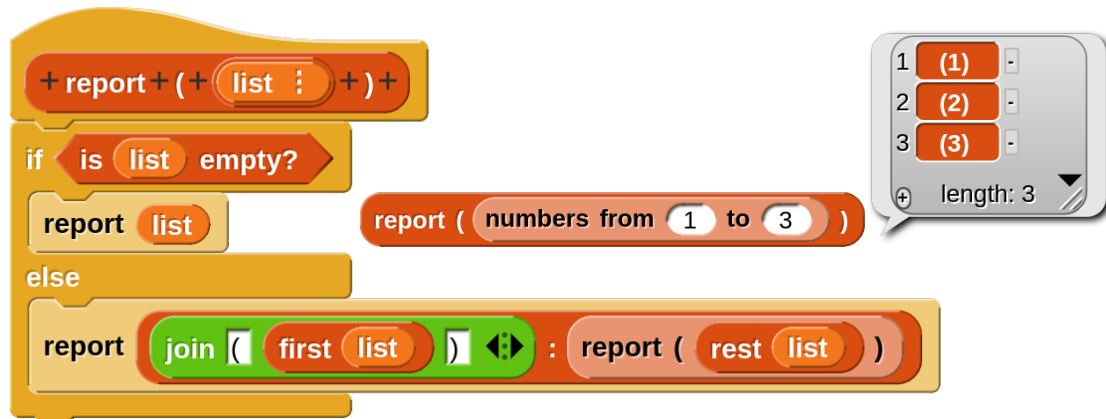
デバッグモードで1ステップ実行してみると、リストが空になると次のように、my length of () が0を返り値としてこの場所に戻ってきます。それぞれの呼び出しで1を加えているのでカウントアップされていきます。



$1 + (1 + (1 + 0)) = 3$ のようにして、返ってきた値に対して加算が行われて終了します。

5.2.3 リストをリポート

リストの先頭の要素に対して操作し、その残りのリストに対して再帰呼び出しをした結果を加えることをリストが空になるまで行います。map を行ったような結果になります。

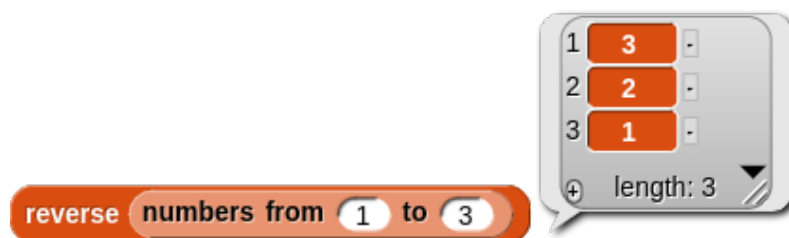


リストの先頭の要素から操作したものを順に加えているのでこのようになりました。

残りのリスト操作の結果に先頭の要素を加えるようにすると、逆順リストが得られます。



append はリスト同士を一つにするものなので、先頭の要素を `list` の中に入れてリストにしています。(ver.11 では要素だけでもよくなりました。)

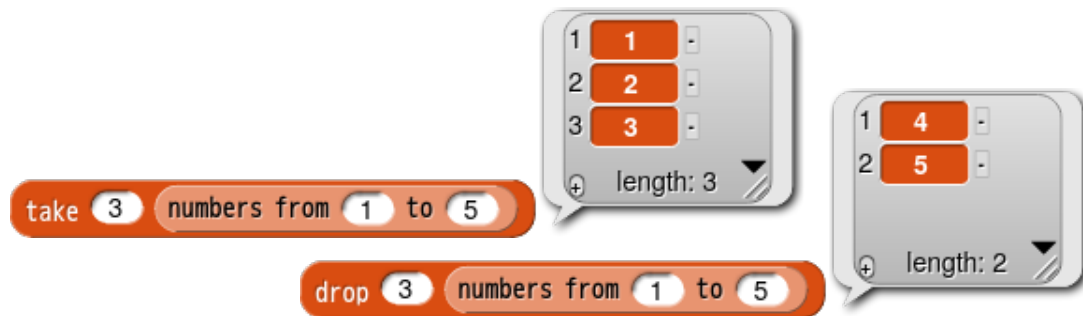


reverse (1 2 3)	↓	[L2]	+	(1)		L3 = (3 2 1)
reverse (2 3)	↓	[L1]	+	(2)	↑	L2 = (3 2)
reverse (3)	↓	[L0]	+	(3)	↑	L1 = (3)
reverse ()				()	↑	L0 = ()

デバッグモードで 1 ステップ実行して眺めてみると、実行の様子が見られておもしろいです。

5.2.4 take と drop

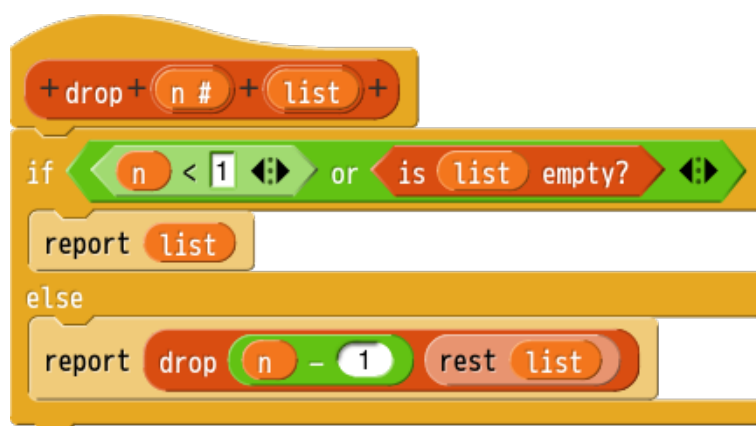
Haskell 言語にはリストの先頭から n 個の要素を取り出す `take` と、リストから先頭の n 個の要素を取り除いたリストを返す `drop` があります。以下のような動作になります。



定義です。take では先頭の要素に n が 0 になるまで次の要素を追加することを再帰を使って繰り返します。 n が 0 以下の時と引数のリストが空の時は空リストを返します。



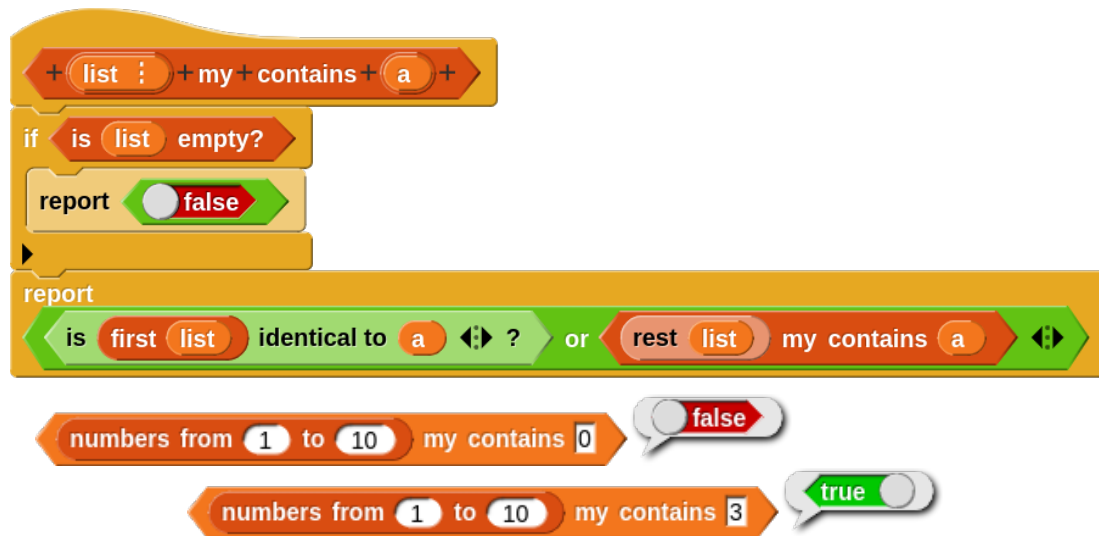
drop では n の値が 0 になるまで n の値を -1 しながら先頭以降のリストに対して再帰を繰り返します。 n が 0 の時に返すのは引数であるリスト、つまり求めるリストです。引数であるリストが空の場合は空リストを返すことになります。



5.2.5 my contains

リストの中に指定の要素が存在するかを求める contains ブロックを作ってみます。

リストの先頭が指定の要素ならば true を返します。そうでないならば、残りのリストに対して同じ操作を繰り返します。



5.2.6 my uniques

リストから重複を取り除く `uniques` of を再帰呼び出しで定義してみます。大文字小文字を区別するようにしています。

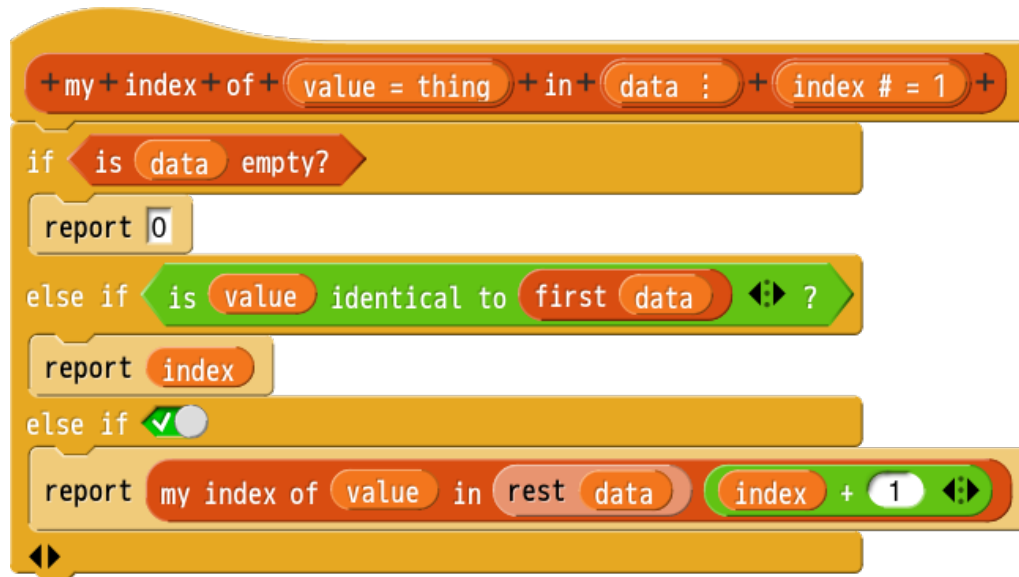


keep を使って先頭の要素と同じでないものを集めることを再帰的に行っています。

因みに、`not is identical to first list ?` を
`not mod first list = 0` にして、`numbers from 2 to 100` を引
数として実行すると 100 までの素数列が求められます。

5.2.7 my index of () in ()

index of thing in の大文字小文字を区別する版を再帰呼び出しで作ってみます。インデックス値を +1しながら走査する必要があるの値を渡していきます。



次のように局所定義ブロックにすると、プリミティブブロックと同じかたちになります。



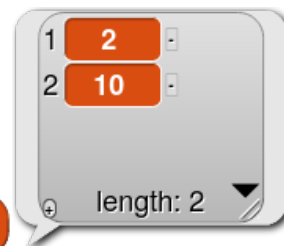
対象文字がヒットしたすべてのインデックスのリストを返すようにしてみます。



```
+my+indexes+of+value = thing+in+data :+
block variables myIndexes
set myIndexes to
if is d empty?
report l
else if is v identical to first d ?
report
append
list i call myIndexes with inputs v rest d i + 1 l
else if
report call myIndexes with inputs v rest d i + 1 l
input names: v d i l
report call myIndexes with inputs value data 1 list
```

The script defines a function `myIndexes` that takes a `value` and a `data` list as inputs. It uses a recursive approach to find all indices of the `value` in the `data` list. The base case is when the data list is empty, returning an empty list `l`. The recursive case checks if the first element of the data list matches the value. If it does, it appends the current index `i` to the list and recursively calls `myIndexes` on the rest of the data list. If it doesn't match, it recursively calls `myIndexes` on the rest of the data list without appending the current index. The script is called with `value = 'A'` and `data = ['aA001122aA']`, returning the list `[2, 10]`.

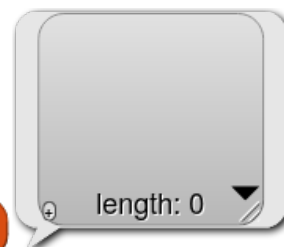
my indexes of A in split aA001122aA by letter



1	2
2	10

length: 2

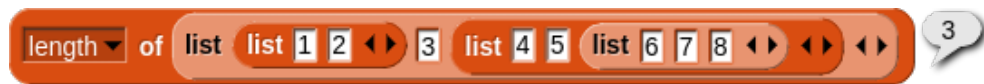
my indexes of x in split aA001122aA by letter



length: 0

5.2.8 リスト要素の巡回

要素にリストを含むリストに対して length を使用すると、内部のリストの分はカウントしません。



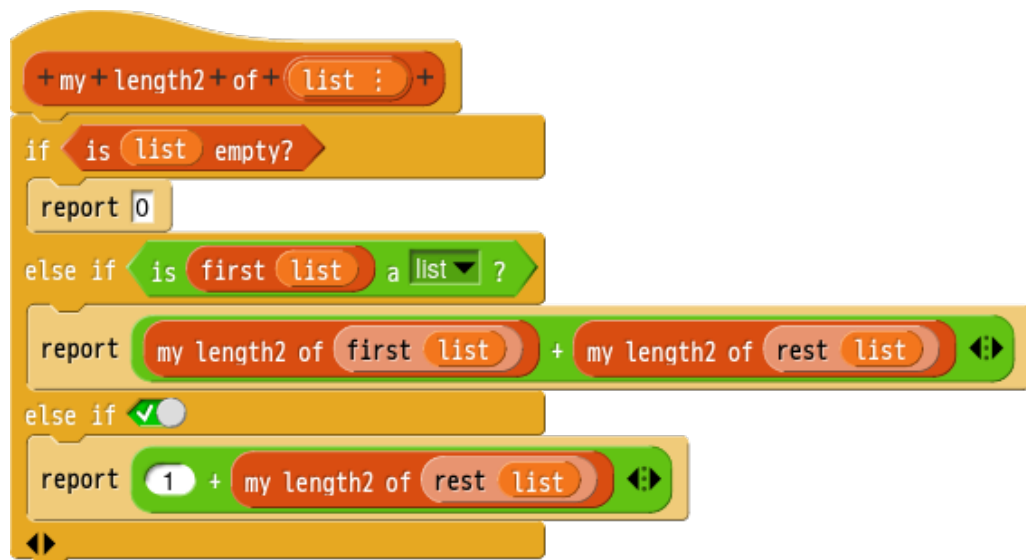
これは my length も同様です。



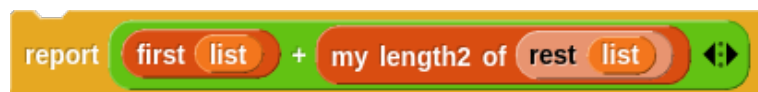
flatten of 後に length でもできますが、再帰を使って内部の要素に対してもアクセスしてみます。

処理の内容は次のようになります。

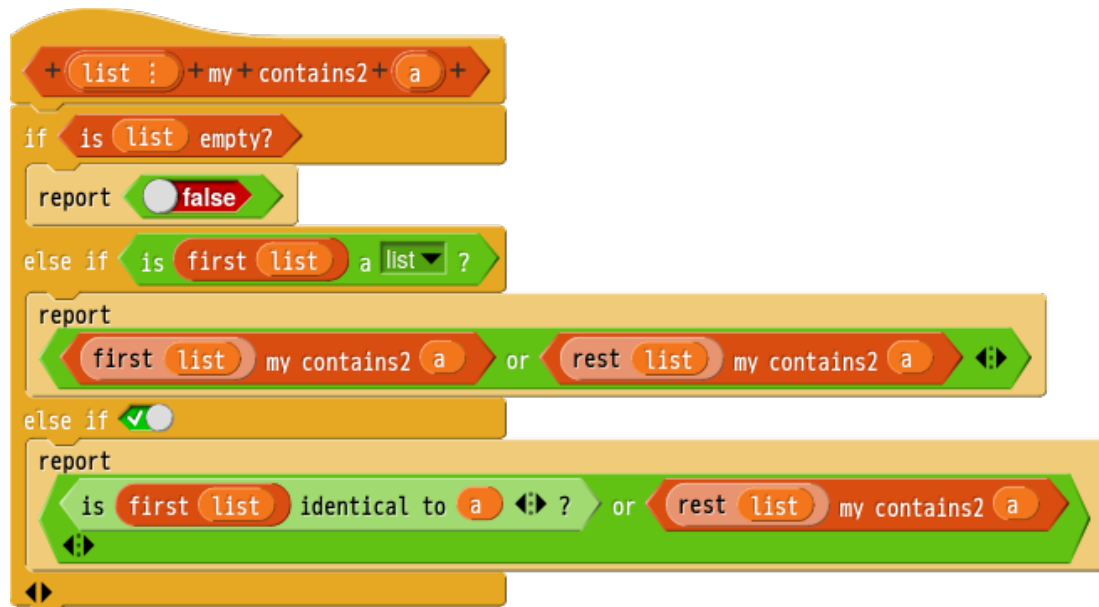
- もしリストが空ならば 0 をレポートする。
- もし先頭の要素がリストならば、そのリストに my length2 をしたものに残りに対して my length2 をしたものを加える。
- そうじゃなかったら、残りに対して my length2 をしたものに 1 を加える。



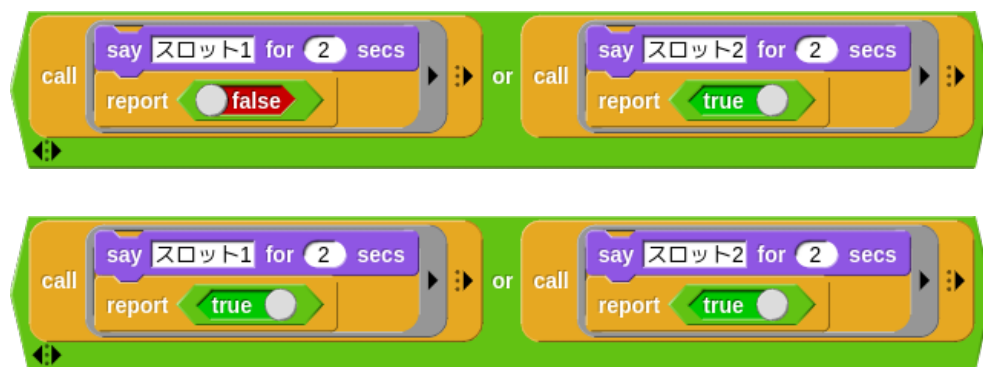
最後の report ブロックで 1 を加えるのではなく、要素の値を加えるようにすると合計を求めることができます。その場合のブロック名は sum of のようなものになると思いますが。



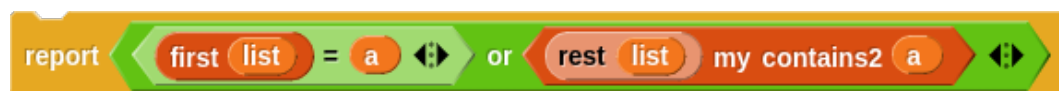
my length2 ブロックを応用すると my contains2 ブロックを作成することができます。true か false かを扱うので演算子は「or」を使用します。なお、is identical to ブロックの代わりに = ブロックを使用すると、マニュアル XI. Metaprogramming A.Reading a block 内の callers of ブロックで使用されている deep contains ブロックになります。



「and」ブロックでは左側のスロットから順にテストして、false ならば残りのスロットのテストは行いません。それに対して、「or」ブロックでは左側のスロットから順にテストして、true ならば残りのスロットのテストは行いません。



そのため、リストの先頭からテストしていった、指定の要素が見つければリストの残りはテストせずにそこで true を返して終了になります。

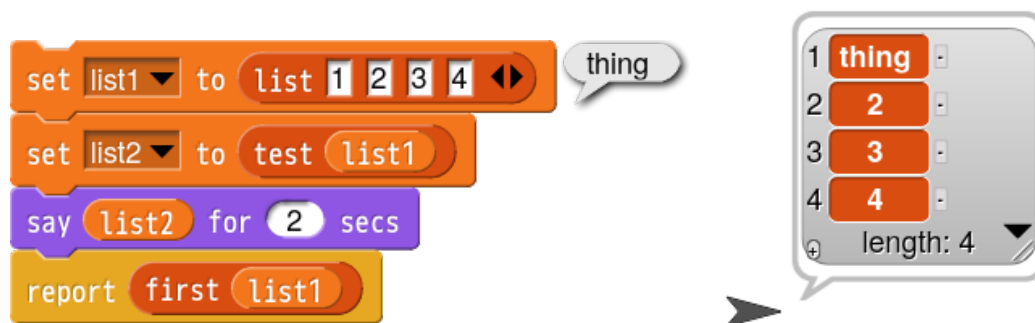


5.2.9 リストのコピー

次のようにリストをコピーしようとしてもリスト自体ではなくリストがどこに記憶されているかがセットされます。変数 list2 は list1 が示す同じリストの記憶場所を参照するので、どちらかの変数を使って変更を加えともう一方も同じ内容をリポートすることになってしまいます。他の言語でもリストや配列ではこのような仕組みになっています。



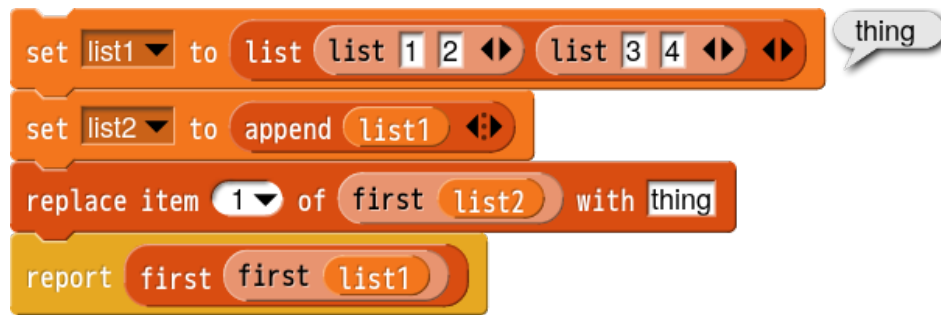
これはブロックの入力スロットにリストをドロップした場合も同じになります。



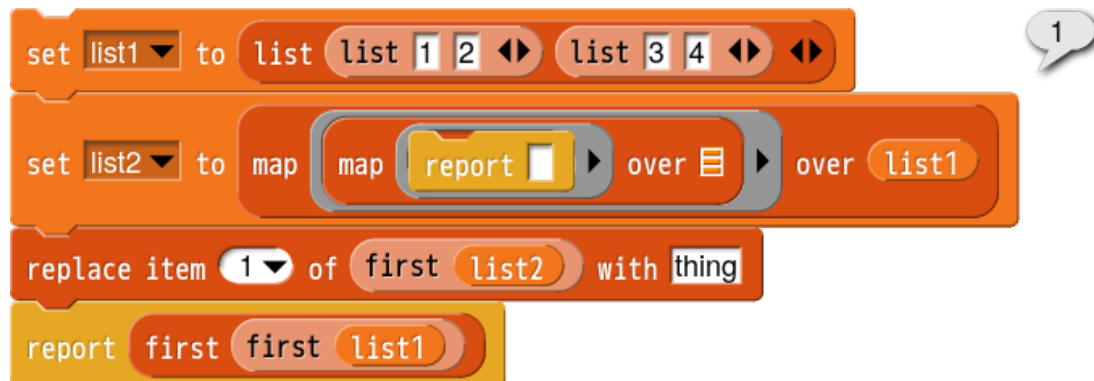
リストを含まないリストならば append でリストのコピーを新たに作成することができます。



しかし、要素にリストが入るとコピーではなくなります。



map を使って 1 層内部までコピーするとうまくいきます。

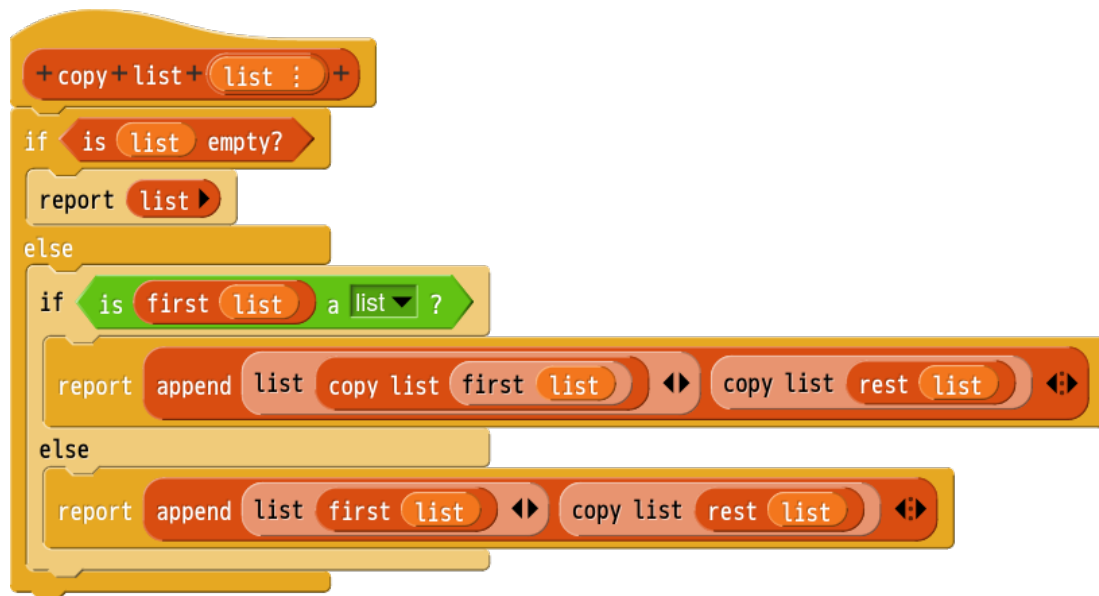


要素のリストの要素にリストが入るとコピーではなくなります。



リストの受け渡しのコピーではなく「参照渡し」（記憶場所のありかを受け渡す）という仕様になっているのは、リストが多く記憶場所を必要とすることが多いですし、コピーに時間がかかることがあるのかもしれない。

append を使うと単純なリストをコピーできることを利用して、再帰的に append して複雑なリストをコピーするブロックです。



5.2.10 指定の要素に対する delete と replace

リストからある要素を取り除いたリストを求めることは keep を使えばできます。しかし、リストの要素がリストの場合はその内部までは対応しません。

3	A	B
1	3	
2	1	2
3	9	

keep items ☐ not ☒ = 2 ☐ from list 2 3 list 1 2 9 2

my length2 を応用した再帰版 delete です。

```

+delete+ a +of+ list : +
if is list empty?
  report list
else if is first list a list ?
  report delete a of first list : delete a of rest list
else if is first list identical to a ?
  report delete a of rest list
else if
  report first list : delete a of rest list

```

1	3	-
1	1	-
2		-
	length: 1	
3	9	-
	length: 3	

delete 2 of list 2 3 list 1 2 9 2

リストを含まないリストの要素の入れ替えは map を使えばできますが、次のように内部のリストまで 2 を 7 に入れ替えることはできません。

5	A	B
1	7	
2	3	
3	1	2
4	9	
5	7	

map if ☐ = 2 then 7 else ☐ over list 2 3 list 1 2 9 2

指定の要素だった場合、それをコピーしないでリストの残りを続行すれば delete になりますが、置き換える要素をコピーすれば replace になります。

```

+replace+ old +with+ new +of+ list : +
if is list empty?
  report list
else if is first list a list ?
  report
    replace old with new of first list :
    replace old with new of rest list
else if is first list identical to old ?
  report new : replace old with new of rest list
else if ✓
  report first list : replace old with new of rest list

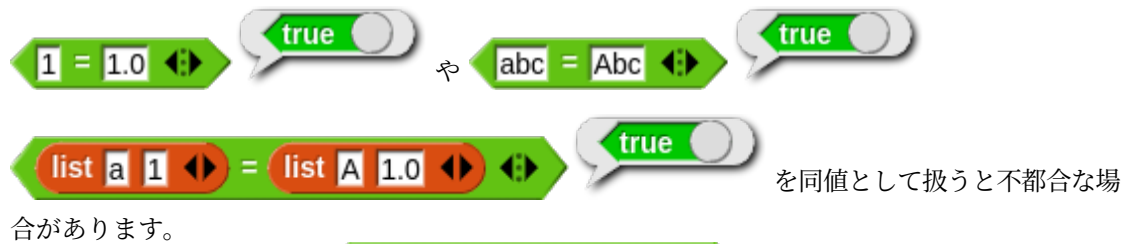
```

5	A	B
1	7	
2	3	
3	1	7
4	9	
5	7	

replace 2 with 7 of list 2 3 list 1 2 9 2

5.3 = と identical

二つの値が同じかどうかを調べる  ですが、



その場合に使えるのが、  です。



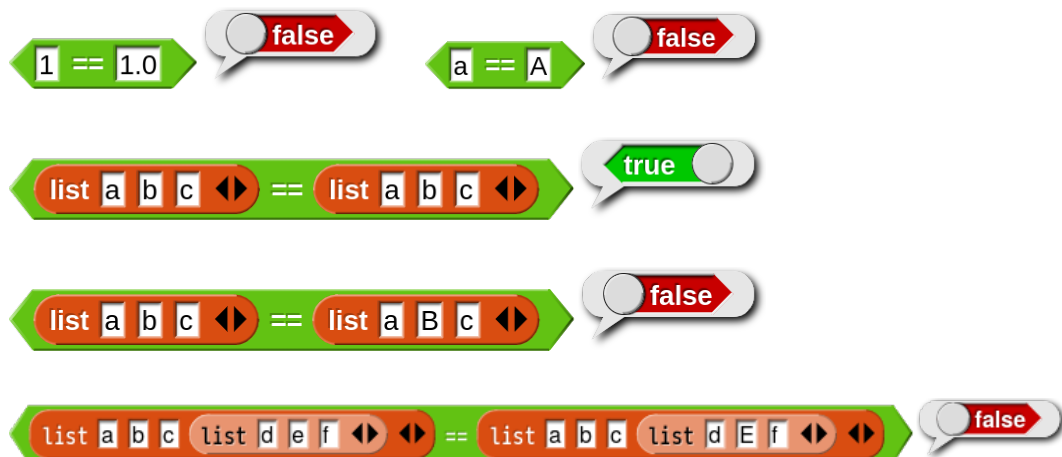
上記は両入力スロットの値についてテストしましたが、リストに対する場合はリストの在り処が同じかどうかのテストをするようです。次は要素値は同じですが、別々に存在するリストです。



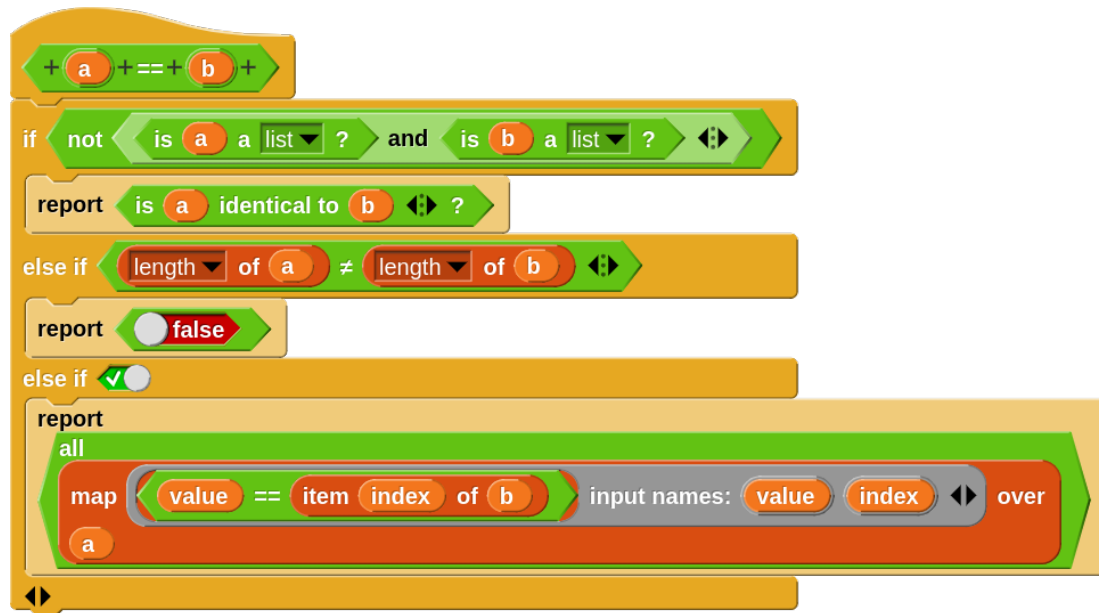
リストの要素を変える場合は元のリストを修正するのではなく、新しくリストを作成してそれを指すようにするようです。a は元のリストを指したままなので `false` になります。

リストに関しても値の同値性だけをチェックするブロックを作ってみます。

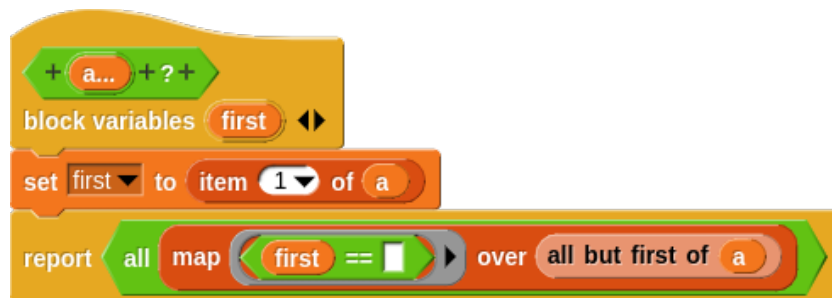
「identical」ではなく「==」としてみました。



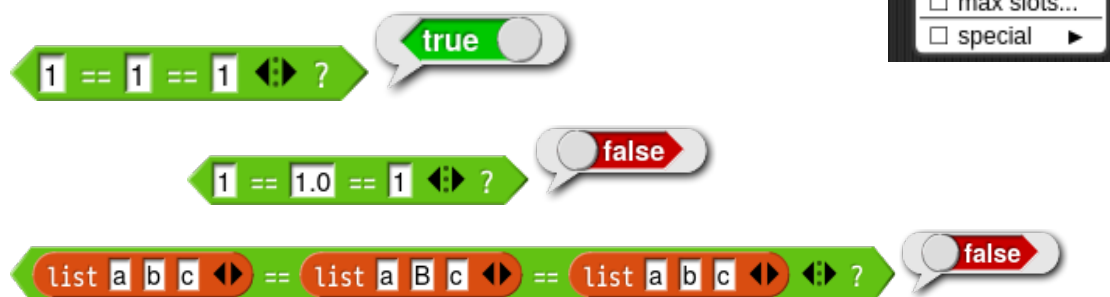
スクリプトです。a と b は Any type 型です。リストの要素にリストを含むものに対処するために再帰呼び出しを使用しています。map を使うとすべての要素をテストすることになります。なお、all は  の右端の三角マークの部分に map をドロップしたものです



is () identical to () では入力スロットを追加できます。  を使って次のようにすると、同じことができます。a を Any type 型 Multiple inputs (value is list of inputs) にします。

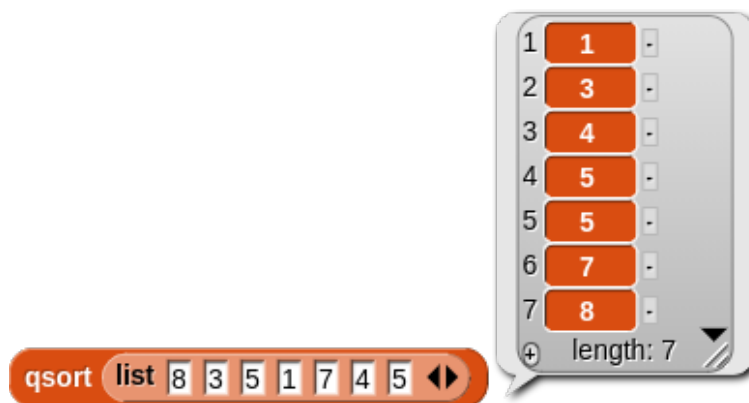
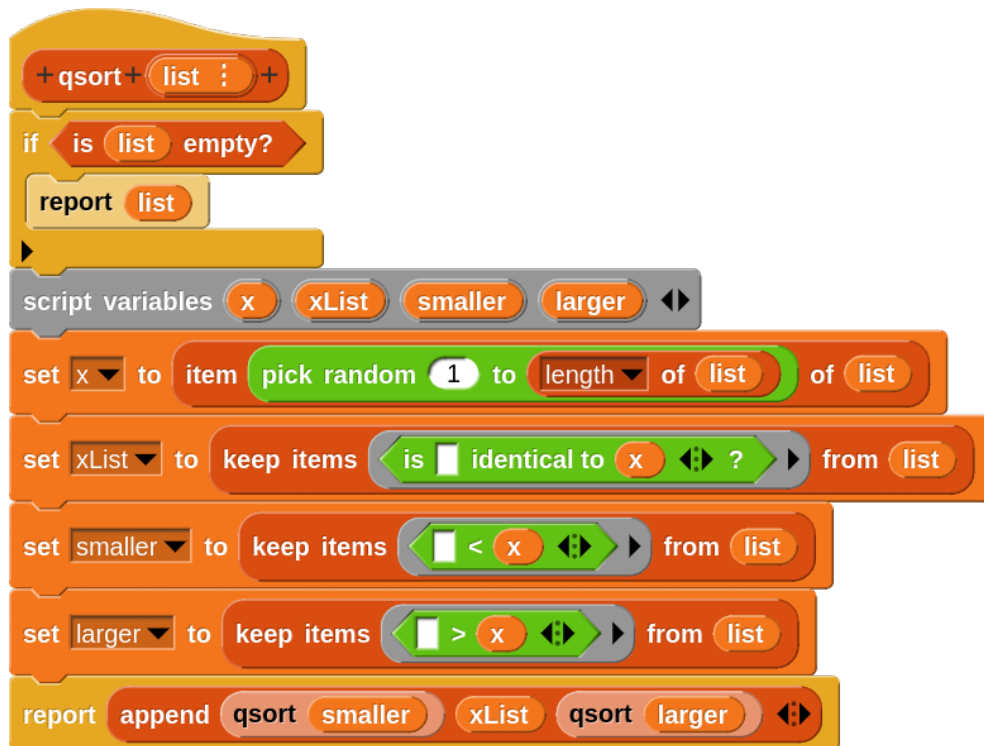


変数の設定で、入力スロットを増やす時に間に挿入する文字列 separator を == に、入力スロット数 0 の場合の文字列 collapse を all == に、初期値としての入力スロット数 initial slots を 2 にします。



5.3.1 クイックソート（整列／並べ替え）

クイックソートのアルゴリズムは有名でよく題材として扱われています。リストの中から任意の値を選び、それよりも小さい値のグループ、その値、大きい値のグループに振り分ければ選択された値の位置付けができます。この操作を小さい値のグループ、大きい値のグループに対して再帰的に繰り返していけば最終的に並べ替えが完了します。任意の値の選び方として random ブロックを使いましたが、先頭の値でも構いません。Snap! には keep ブロックがあるので、リスト要素の入れ替えなどに煩わされず、アルゴリズムをそのまま表したように割と分かりやすいスクリプトが作れます。

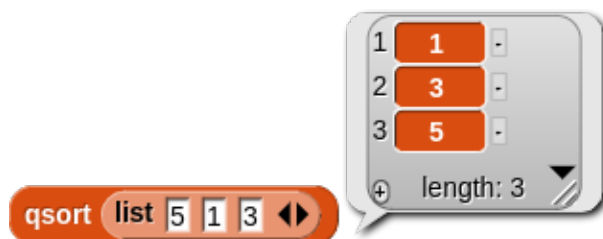


このように自分自身を複数参照することを多重再帰と言います。

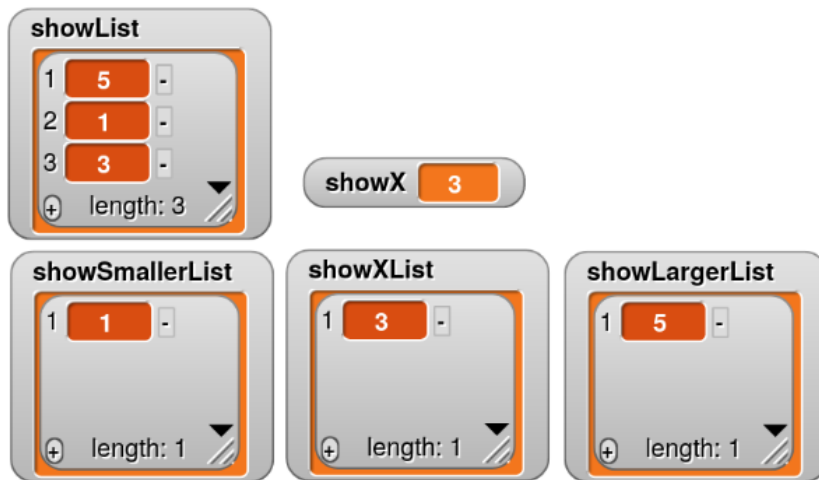
次のように showList, showX, showXList, showSmallerList, showLargerList のグローバル変数を作成し、リストを通して値を表示すると操作の様子が見られます。



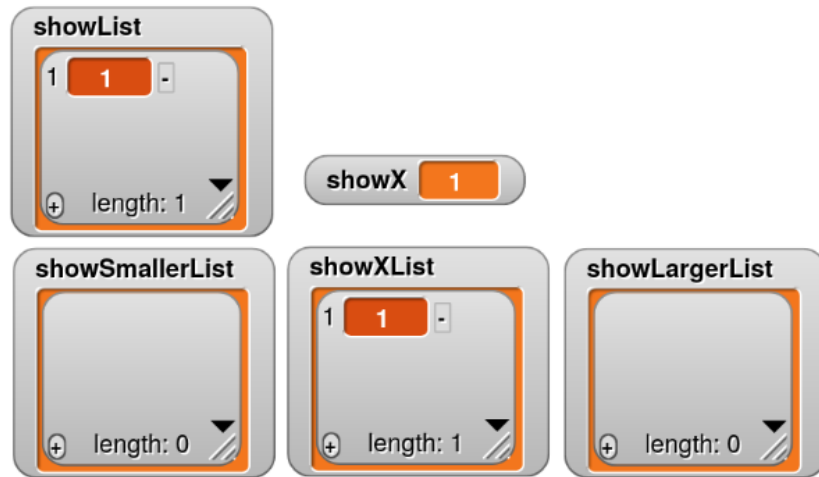
リストの値表示のたびに pause all になります。  をクリックして続行してください。



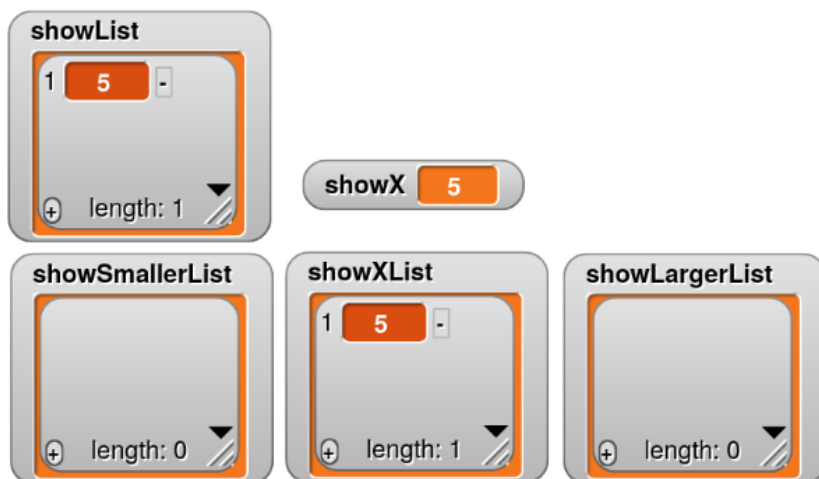
最初に選択された値が 3 だった場合です。



3 より小さい値のグループ処理



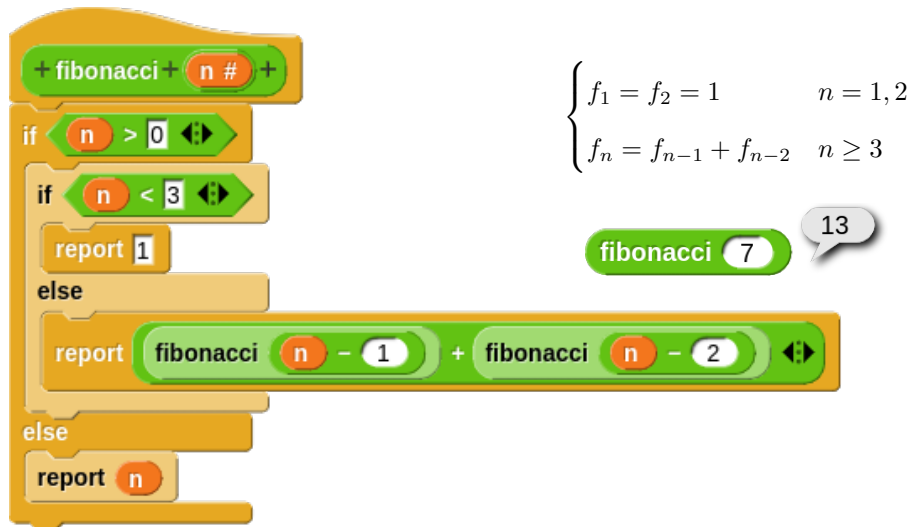
3 より大きい値のグループ処理



この場合要素数は 1 でしたが、それぞれのグループに対して再帰的に処理されます。

5.3.2 フィボナッチ数列

再帰の例としてよく示されるフィボナッチ数列です。

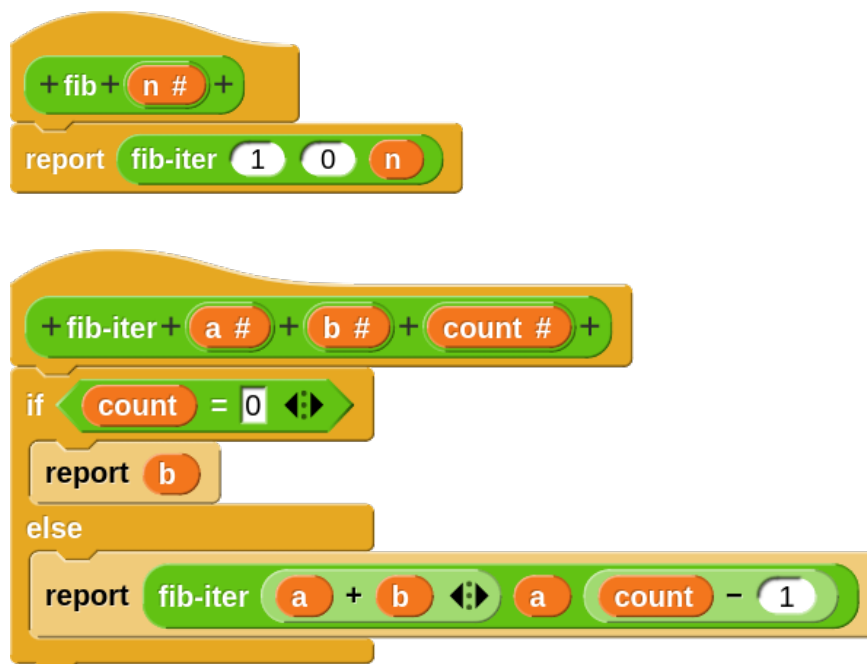


これは $(n - 1)$ と $(n - 2)$ を引数にして 2 度再帰呼び出ししています (多重再帰)。そのため、 n が大きくなると時間がかかってしまいます。

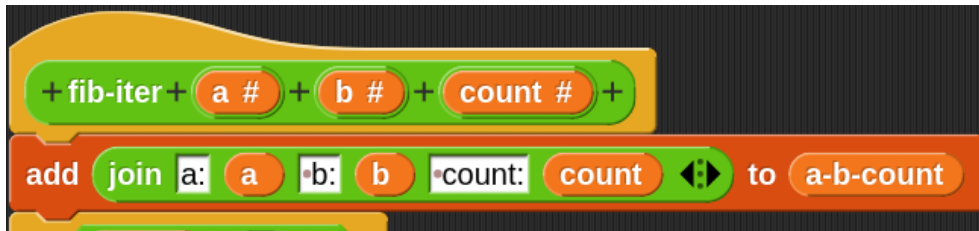
例えば $\text{fib}(6)$ を求める場合は、右のようになりますが、 $\text{fib}(6)$ で必要とする $\text{fib}(4)$ は $\text{fib}(5)$ で処理済みで、 $\text{fib}(5)$ で必要とする $\text{fib}(3)$ は $\text{fib}(4)$ で処理済み ... ということで、処理済みのものはその値を渡してやれば済むのでその分の再帰呼び出しの必要がなくなります。

$$\begin{aligned} \text{fib}(6) &= \text{fib}(5) + \text{fib}(4) \\ \text{fib}(5) &= \text{fib}(4) + \text{fib}(3) \\ \text{fib}(4) &= \text{fib}(3) + \text{fib}(2) \\ \text{fib}(3) &= \text{fib}(2) + \text{fib}(1) \\ \text{fib}(2) &= 1 \\ \text{fib}(1) &= 1 \end{aligned}$$

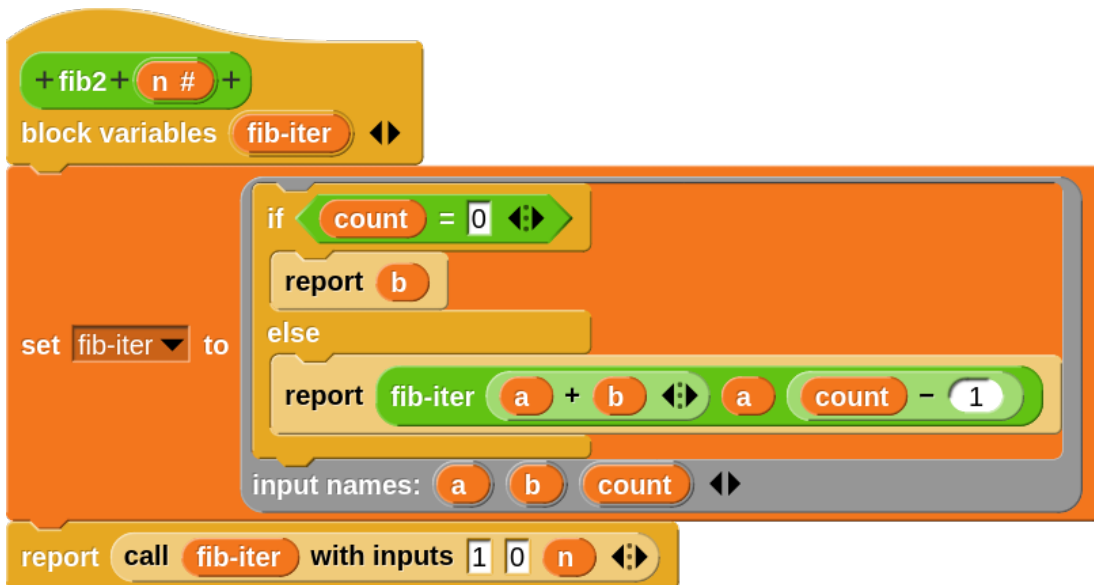
SICP の非公式日本語版翻訳改訂版 真鍋宏史氏訳 の 39 ページに反復プロセス版が載っていました。前二項の値を渡すことで余分な再帰呼び出しを避けています。



次のようにブロックを追加して a, b, count の値を表示させると動作が分かります。

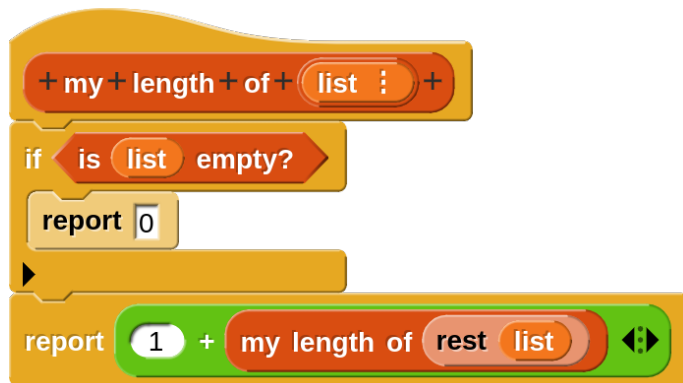


fib-iter はここでしか使わないので、本体内部で変数にセットすると局所定義ブロックにすることができます。



5.3.3 末尾再帰

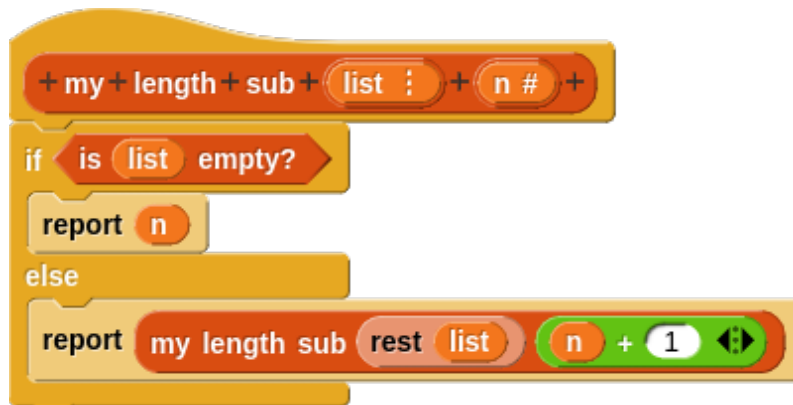
41 ページで my length を扱いました。



この定義ブロックでは、計算式に再帰呼出しが含まれていて、再帰呼出しによる値が確定しないと計算することができません。リストが空になると確定した 0 の値を使って順々に計算した値を戻しながら一番最初のところまで戻って、ようやく値 3 を得ることになります。my length of (1 2 3) を $L(1\ 2\ 3)$ と表してみます。

$$\begin{aligned}
 L(1\ 2\ 3) &= 1 + L(2\ 3) \\
 &= 1 + 1 + L(3) \\
 &= 1 + 1 + 1 + L() \\
 &= 1 + 1 + 1 + 0 \\
 &= 1 + 1 + 1 \\
 &= 1 + 2 \\
 &= 3
 \end{aligned}$$

これに対して、次のようにすると再帰呼出しだけを行うという形にすることができます。引数 $(n + 1)$ を渡していくことでカウントしています。

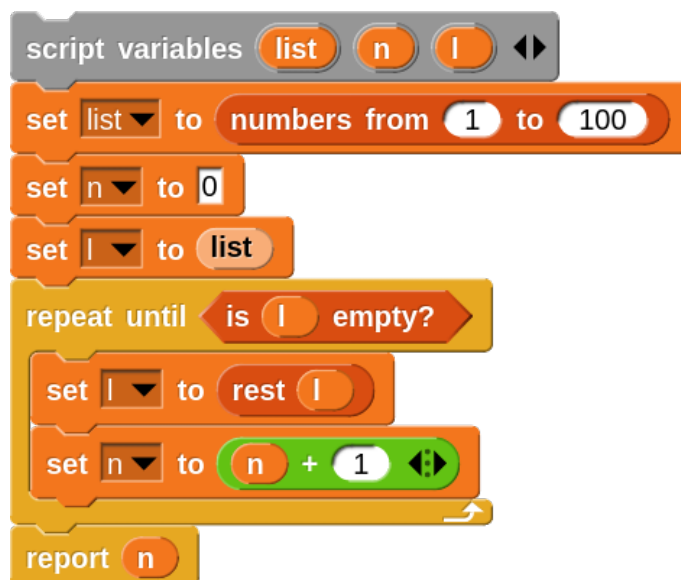


これ呼び出す本体定義は次のようになります。引数 `n` の値を 0 にすることで、カウンターの値を初期化しています。



```
my length3 of (1 2 3)
  my length sub (1 2 3)(0)
    my length sub (2 3)(1)
      my length sub (3)(2)
        my length sub ()(3)
          3
        3
      3
    3
  3
```

このように、一番最後の再帰呼出しの返り値がそのまま定義ブロックの値になり、再帰呼び出しから戻る時は単に値をそのまま渡すだけです。このような処理の最後を単独の再帰呼び出しにする形を末尾再帰といいます。末尾再帰のスクリプトは再帰を使わない反復のスクリプトに変換することができます。



Scheme や Haskell などでは、末尾再帰はループに最適化されるので高速になりますが、Snap! ではさほど変わらないようなので末尾再起にこだわる必要もないようです。

6 高階関数

関数の引数や戻り値に関数を指定できるものを高階関数と言います。リスト操作に使用する `map`, `keep`, `find`, `combine` ブロックは入力スロットにリングがあり、引数に関数型ブロックを指定する高階関数型のブロックです。

6.1 高階関数型ブロックの基本

半径を入力して、円の面積をレポートする関数型ブロックです。



たとえば、`円の面積 2 + 5` や `list 円の面積 1 円の面積 2` のように入力に関数型ブロックを使用しても、それは結果としての値を使用しているだけで高階関数型とは言えません。つまり、それぞれ `12.56 + 5` や `list 3.14 12.56` の意味になります。

`map` のような高階関数型ブロックの場合は、リスト要素の処理方法（スクリプトブロック）を指定するという意味になります。



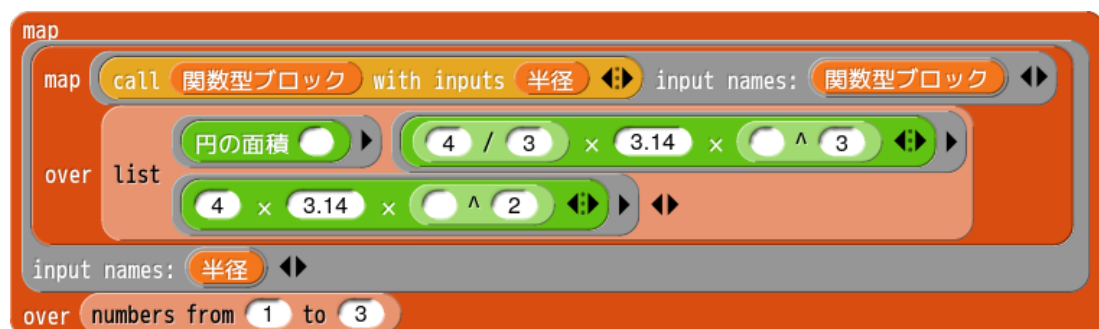
リストを半径とする球の体積を求めます。



リストを半径とする球の表面積を求めます。

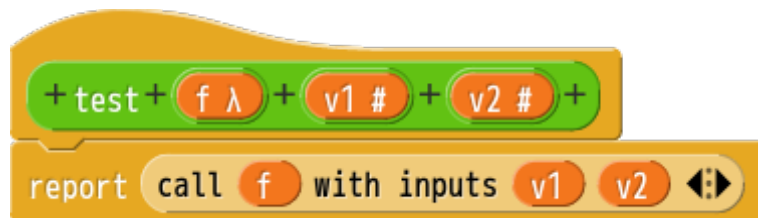


`map` を二重にするとこんな使い方もできなくはありません。



Snap! のユーザー定義ブロックでは高階関数型のものを作成することができます。入力スロットでスクリプトブロックを受け取ることができますし、スクリプトブロックを ringify リングで囲ってやればそのスクリプトブロック自体を戻り値としてレポートすることができます。

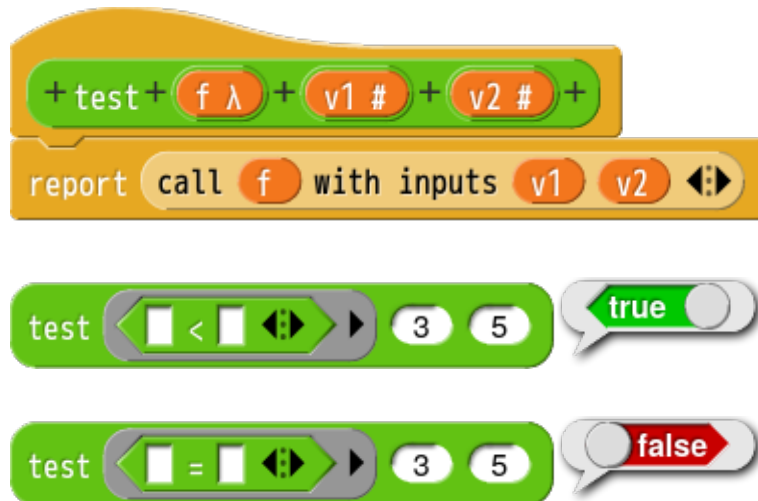
次のブロック定義 test は二つの引数を必要とする関数型ブロックを実行して値を返すものです。入力 f は Reporter 型です。v1 と v2 の入力は Number 型です。関数型ブロックの実行は call ブロックを使用します。これが高階関数型の定義ブロックの基本となります。



入力スロット f に関数型ブロックを入れて処理を指定します。



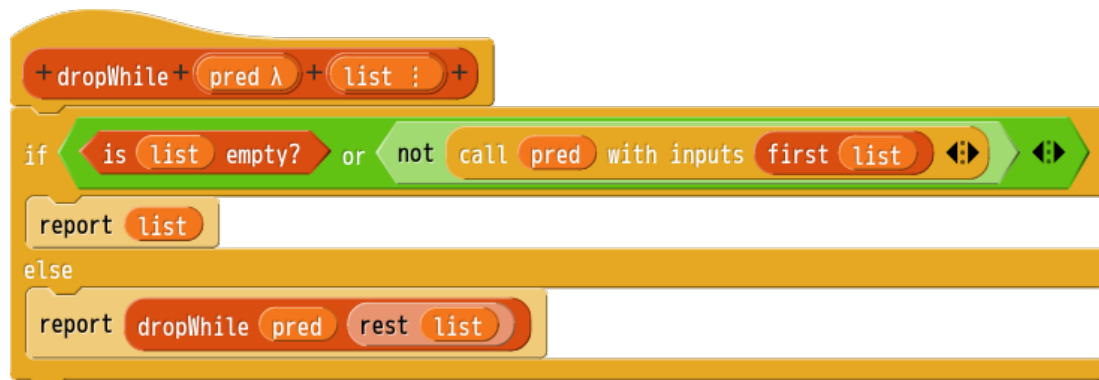
入力 f を Predicate 型にすると、次のように変更されます。(ブロック定義自体の見た目は同じですが)



高階関数型として入力に使用するタイプは Reporter, Any(unevaluated), Predicate, Boolean(unevaluated) が利用できます。実は、Any type や Boolean(T/F) でも入力を ringify してやれば同じ機能になります。

6.2 高階関数型の take と drop

44 ページで要素の個数を指定する take と drop を定義しました。高階関数型ブロックの例として、リストの先頭から条件に一致する要素だけを take や drop するブロック takeWhile と dropWhile を作成してみます。不一致の要素が見つかった時点で操作は終了です。条件をチェックするブロックを受け取る変数 pred は Predicate 型です。



6.3 操作を指定するリストの巡回

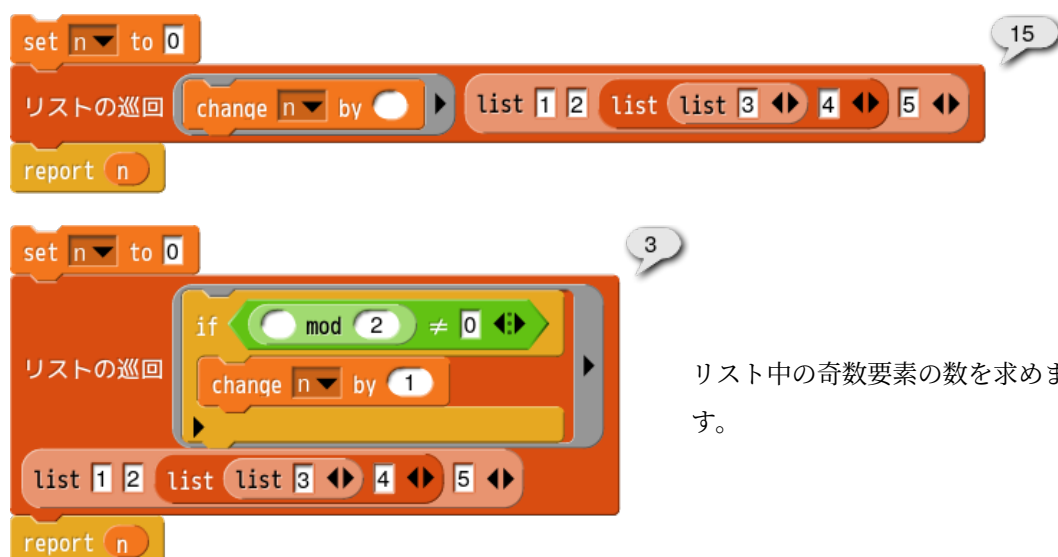
48 ページで「リスト要素の巡回」を扱いました。引数で操作を指定すれば同じ定義ブロックでいろいろな用途に使えます。定義ブロック自体は Command 型で、引数 proc は Command(inline) 型です。proc は引数の一つ使用します。



リストの要素数を求めます。引数としてリストの要素が用意されますが、[change (n) by (1)] と (1) でふさがっているので要素は使用されずに 1 でのカウントになります。

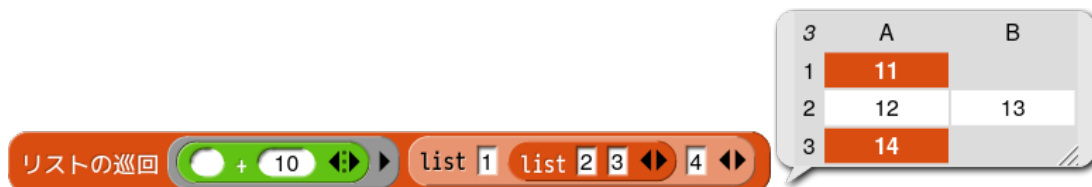
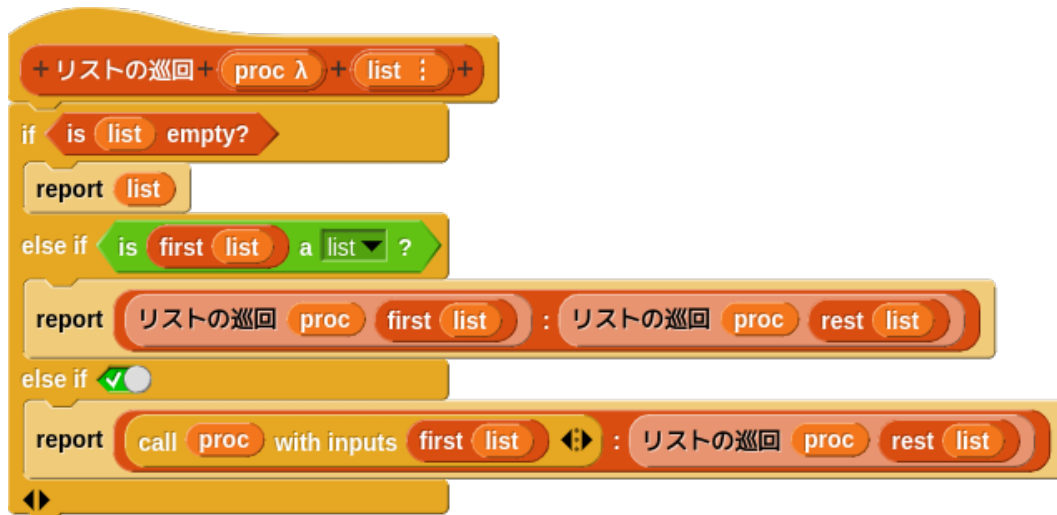


リストの要素の値の合計値を求めます。上とは違い [change (n) by ()] と引数のスロットにリストの要素が入り加算されます。



リスト中の奇数要素の数を求めます。

リポーター版です。引数 proc は Reporter 型です。



意味のある例ではありませんが、要素が奇数の時は 1 を加えます。そして各要素を 2 倍します。



6.4 foldl, foldr

Haskell 言語には foldl と foldr というリスト操作関数があります。

これは、二つの引数をとる関数 proc (変数型は Reporter) と、初期値 init と list の合計 3 つを引数として取る関数です。初期値 init と item (list の先頭要素) の二つの値を引数として proc が何らかの処理をします。得られた値を新たな init とします。その init と item (list の次の要素) を引数として proc を実行 ... というのを list の終わりまで行います。init の値が最終値になります。foldl はリストを左側から順に操作し、foldr はリストを右側から順に操作するものです。

foldl から見ていきます。



操作内容は、 のようになります。

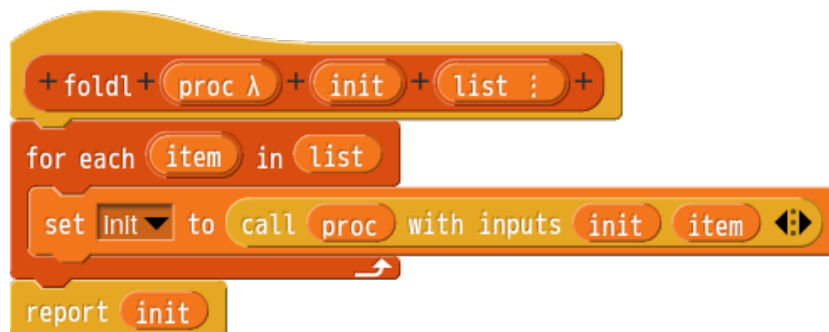
フォーマルパラメータを使用すると次のようになります。



※ フォーマルパラメータの順序は右下の表のように init, item の順になります。(＃1, 2 から名前の変更をしています)

init	+	item	値
0	+	1	1
1	+	2	3
3	+	3	6
6			6

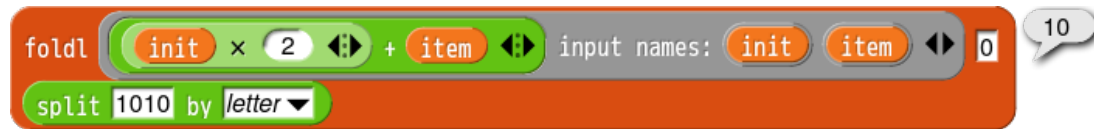
非再帰版だとこうなります。



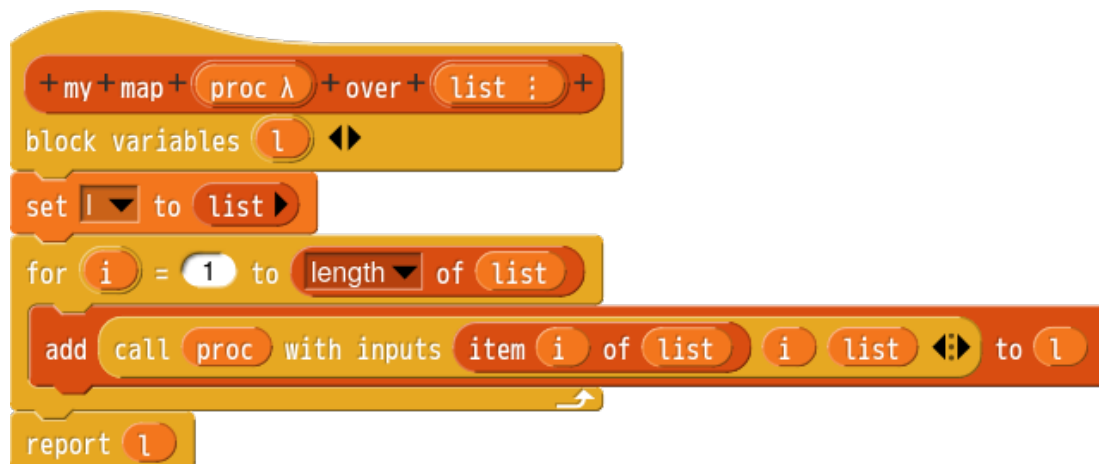
次のようにすると、リストの要素の値は使用しないで要素数 length を求めることになります。



次のようにすると文字列の二進数（1010）を十進数に変換します。



map ブロックを再帰呼び出しを使用しないで作成してみます。引数 proc は Reporter 型です。プリミティブのもののように value, index, list 値を利用可です。



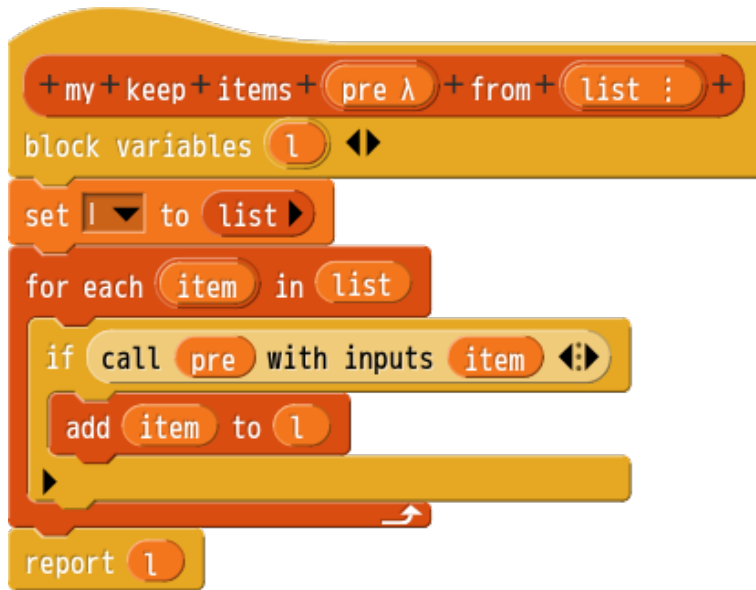
再帰呼び出しを使用して作成してみます。value, index, list 値は利用できません。



foldl 版です。



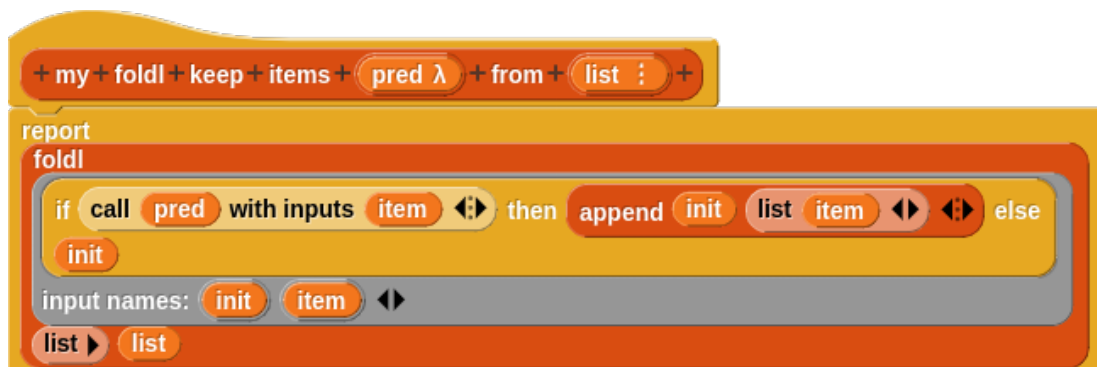
keep ブロックを再帰呼び出しを使用しないで作成してみます。引数 pred は、Predicate 型です。



再帰呼び出しを使用して作成してみます。



foldl 版です。



foldr です。

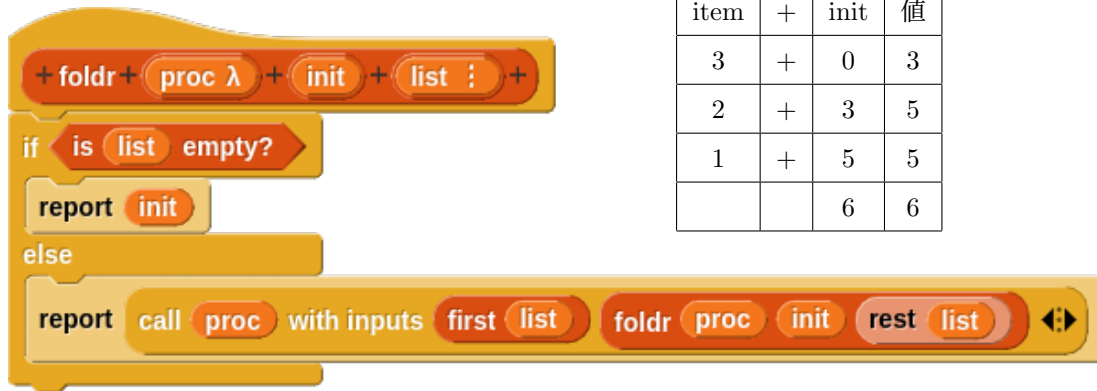


操作内容は、 のようになります。

フォーマルパラメータを使用すると次のようになります。



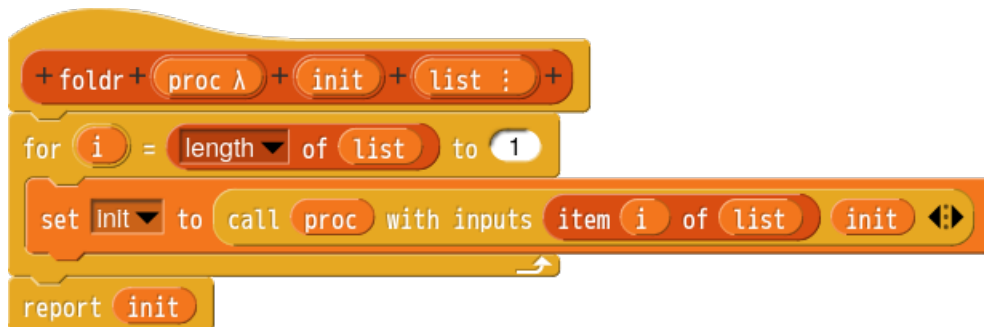
※ フォーマルパラメータの順序は右下の表のように、foldl とは違う item, init の順になります。



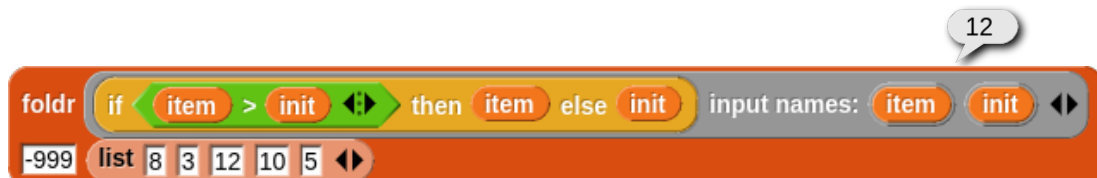
foldr

item	+	init	値
3	+	0	3
2	+	3	5
1	+	5	5
		6	6

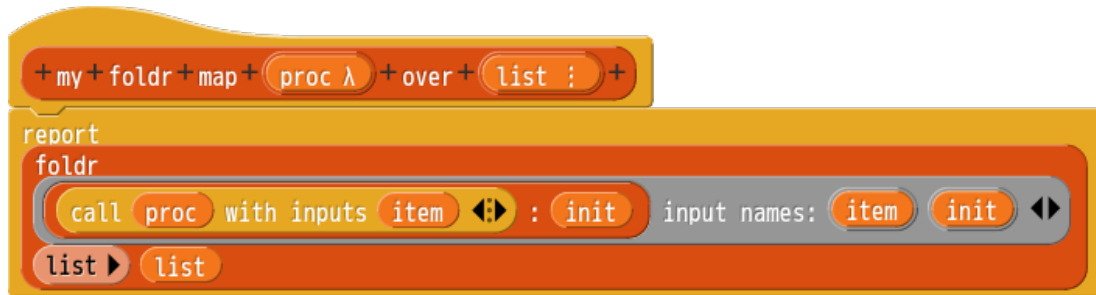
非再帰版だとこうなります。



次のようにすると最大値を求めることができます。



foldr を使っても foldl と同じようなことができます。



foldr がリストを右側から操作することを利用した append です。

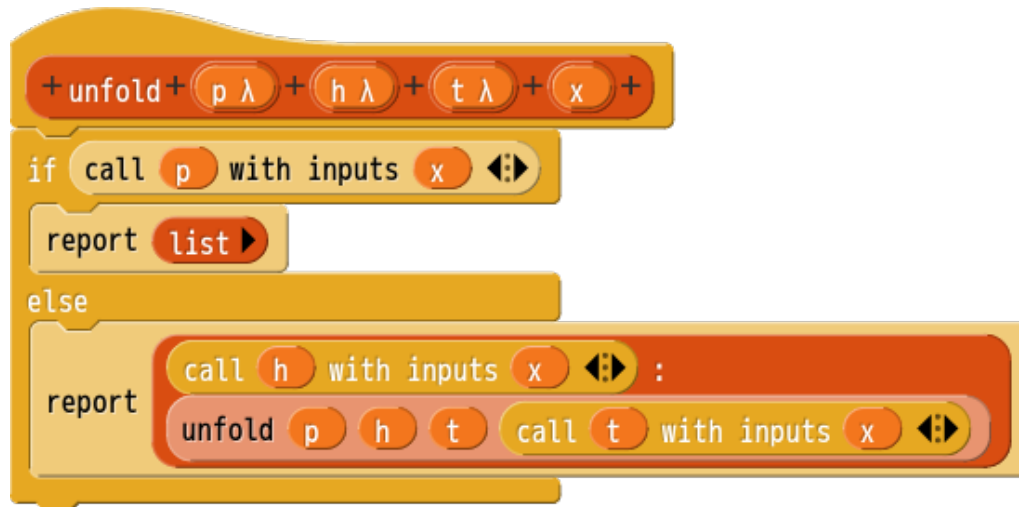


foldl で作成する場合には、リストを逆順にする操作が必要になります。



6.5 unfold

fold は、リストに対して操作を行い値を求めるものでした。逆に、ある値に操作を加えてリストを作成するのが unfold です。



p は、Predicate 型で処理（再帰呼び出し）の終了条件を指定するものです。h は、指定された値 x に操作を加えてリストの要素となる値を取り出す Reporter 型のブロックです。t は、次の要素を求めるために x を処理するための Reporter 型ブロックです。unfold はリストの要素を左から並べていきます。p, h, t はそれぞれ関数型のブロックなので、unfold は高階関数型のブロックです。

二進数をリストで表したもののから十進数に変換するには、foldl を使用して次のようにできます。



逆の操作は unfold を使用すると、次のようにできます。



値を 2 で割って割ると商と余りが求められます。余りが最下位桁の値になります。商は次の桁を求めるための値になります。この操作を商が 0 になるまで行います。リストは逆順で作られるので reverse する必要があります。

リストの要素を右から並べていく `unfoldr` を作成するならば、次のようになります。

The script defines a function `unfoldr` that takes a lambda expression `p λ + h λ + t λ + x +` as input. It uses an `if` block to check the result of `call p with inputs x`. If it's truthy, it reports the `list`. Otherwise, it reports an `append` block containing `unfold p h t call t with inputs x` and `list call h with inputs x`.

Below the script, the `unfoldr` block is shown with arguments: a counter starting at 0, a `mod 2` operation, a `floor of / 2` operation, and the value 10. A monitor displays the resulting list:

1	1
2	0
3	1
4	0

 with a length of 4.

`unfold` を利用して `map` を作成することもできます。

The script defines a function `my unfold map` that takes a lambda expression `f λ + list :` as input. It reports an `unfold` block with arguments: `is empty?`, `call f with inputs first`, and `rest`.

Below the script, the `my unfold map` block is shown with arguments: a `+ 2` operation and `numbers from 1 to 5`. A monitor displays the resulting list:

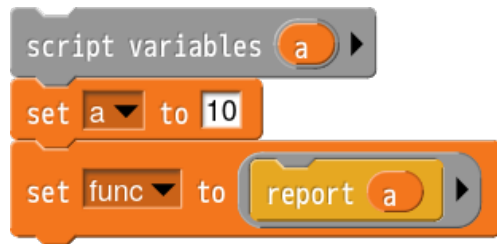
1	3
2	4
3	5
4	6
5	7

 with a length of 5.

6.6 クロージャー

関数の外側で定義された変数を関数内部で使用する関数をクロージャーと言います。その変数を使用できる環境が関数に取り込まれているイメージです。

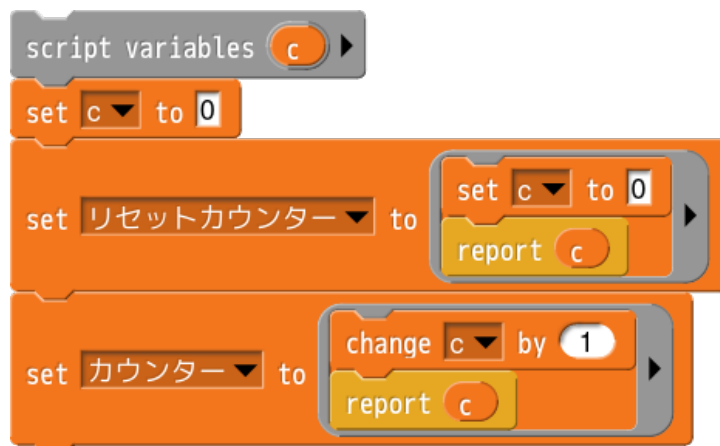
func というグローバル変数を作成し、次のように関数ブロックにします。



すると、func には **func** が設定されます。

a はスクリプト変数なので **a** が示すように、上記の
スクリプト外ではアクセスできないはずです。しかし、**call func** のように a の
値をレポートします。上記のスクリプトを削除しても a はクロージャー func により存在し続
けます。

これを利用すると、次のようにカウンターを作成することができます。



変数 c にはクロージャーを通してしかアクセス
できないので、不用意に変更されない安全な仕
組みになります。



における `value x 2` の部分もクロージャーです。value は関数の外部の変数にあたります。

複数の引数が必要な関数を、一つの引数だけ受け取って処理をすることを連ねて行うやり方があります。そのように 1 引数の関数化することをカーリー化と言います。カーリー化された関数は関数を返す高階関数です。

ある文字を文字列に加えるには、join ブロックを使って次のようにできます。



カーリー化したブロック join-c は入力スロットから文字列を受け取り ringify されたブロックを返します。そのブロックは実行されると加える文字を引数として受け取り、結果として合成された文字列を返します。ここで、join では外部の変数 input を参照しているので結果的に join-c もクロージャーになります。



リポーターブロックを実行するには call ブロックを使用しますが、call ブロックにはリングが装備されています。そこに ringify されたブロックを渡されると次のようになってしまいます。



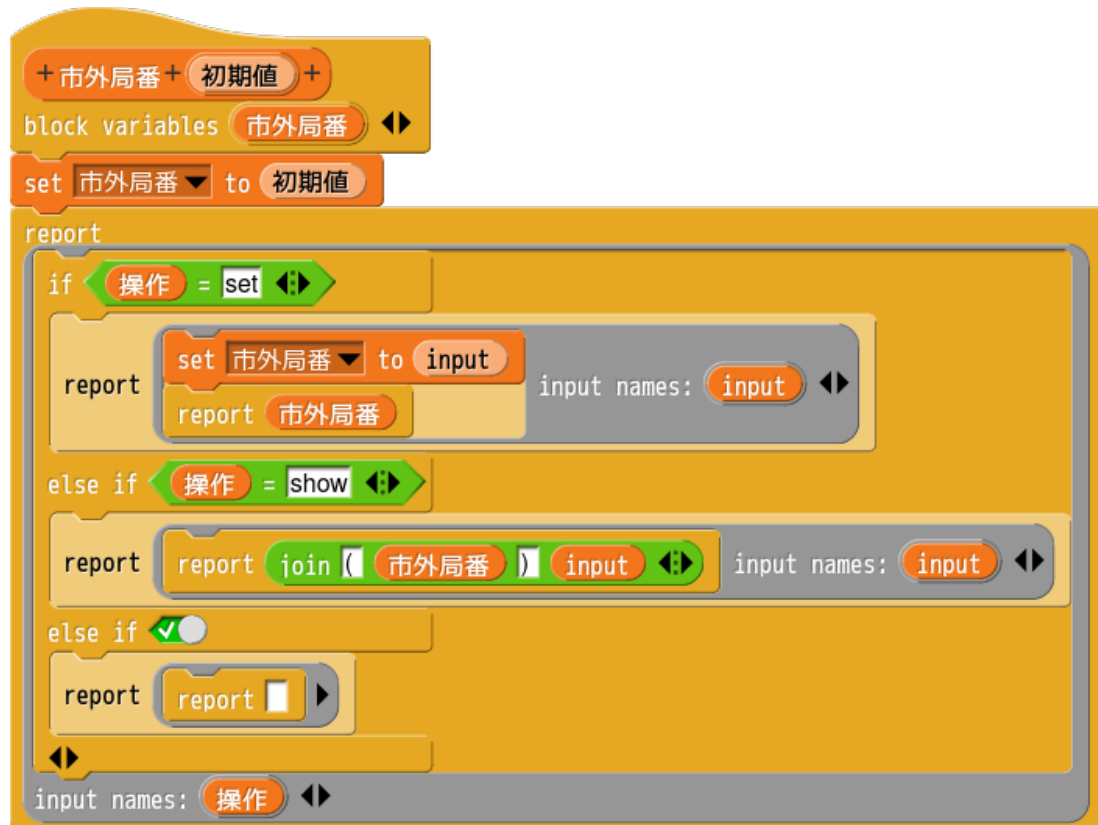
join-c ブロック部分を右クリックして unringify する必要があります。



のようにすると、部分適用することもできます。



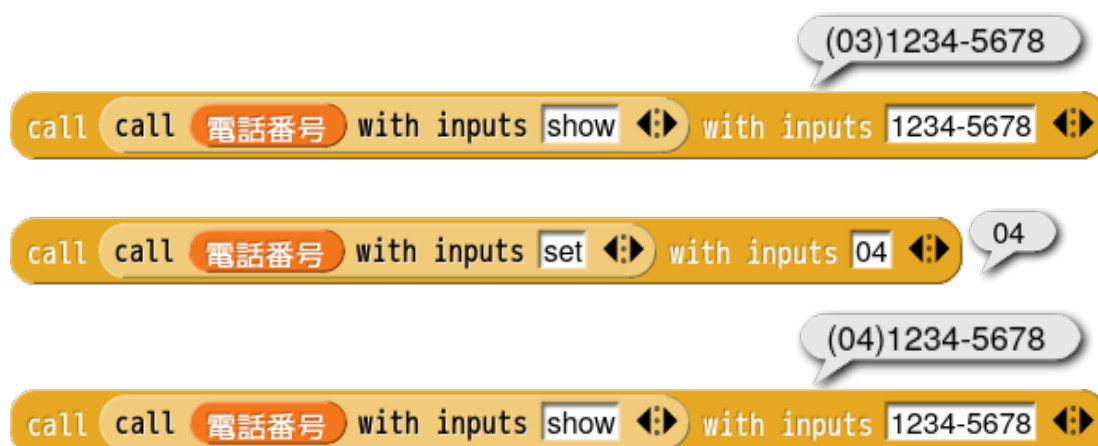
次のようにすると、クロージャーク内のローカル変数「市外局番」に対して変更ができます。
report ブロックの中に操作に応じてそれぞれに report ブロックがあります。report ブロックが返すのは ringify されたブロックなので、これを実行するには二重の call ブロックが必要です。



変数「電話番号」をクロージャークにします。

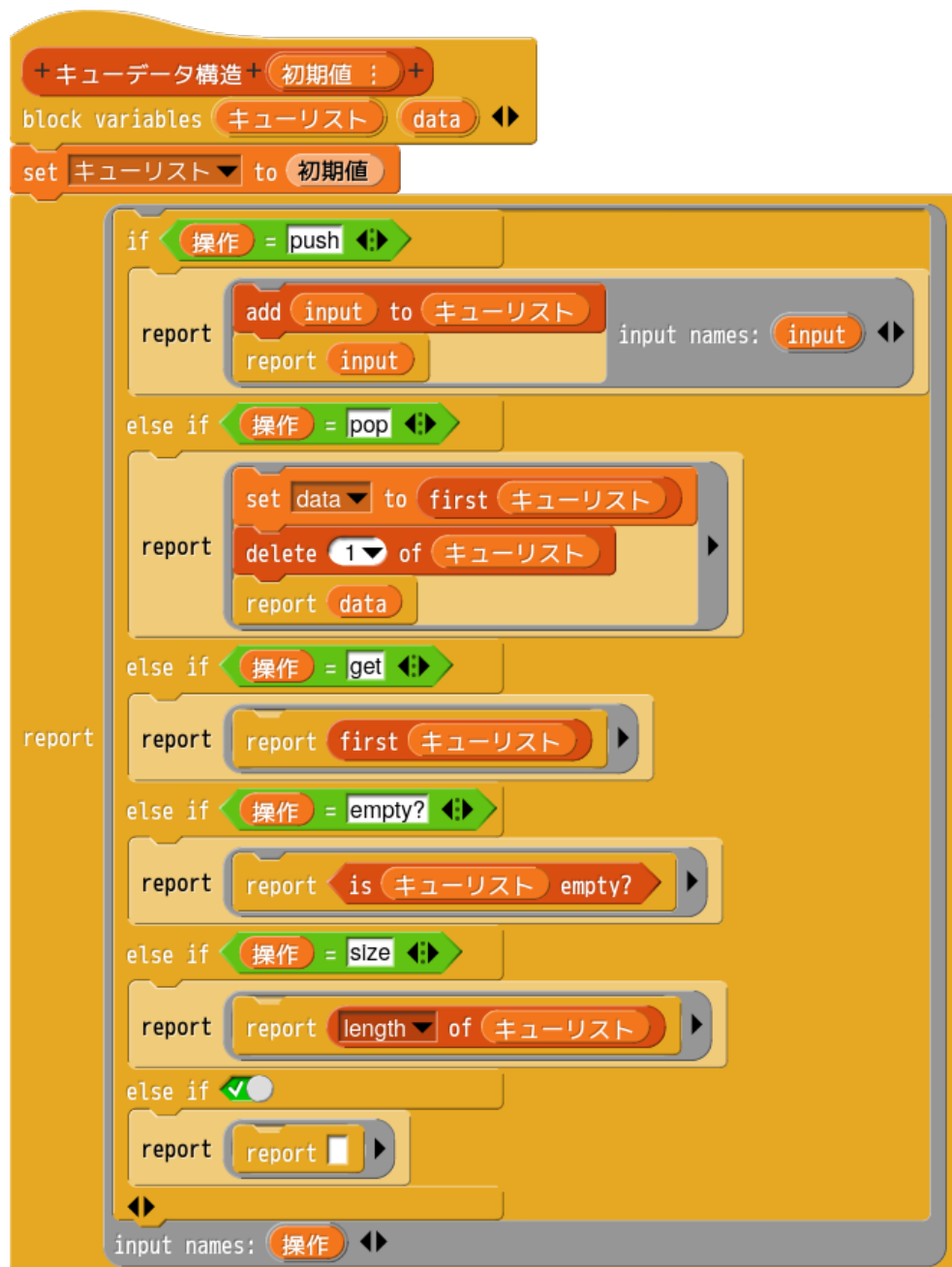


そうすると、「電話番号」を操作することができるようになります。



応用すると、キューやスタック、辞書などのデータ構造を作成することができます。

キューは待ち行列です。データの格納 (push) は列の後尾から、取り出し (pop) は列の先頭からになります。先頭データの参照 (get)、キューが空かどうか (empty?)、キューのサイズ (size) の機能を加えます。



変数「キュー」をクロージャーにします。



call call キュー with inputs push with inputs 2 2

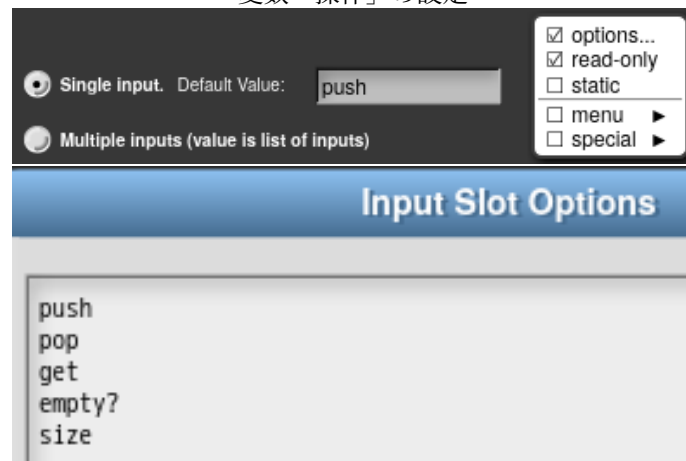
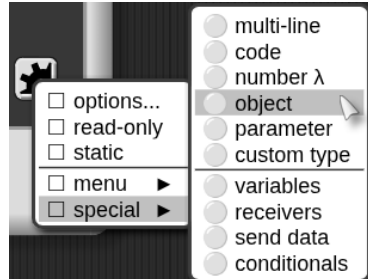
call call キュー with inputs pop with inputs 1

操作用ブロックを作成します。操作は options...(push, pop, get, empty?, size)、read-only で指定するようにします。Default value は push です。変数「データ構造」を special -> object に設定し、キューデータ構造やスタックデータ構造を持つ変数を指定して使えるようにします。

+コレクション+ データ構造 >> + 操作 = push + data +
report call call データ構造 with inputs 操作 with inputs data

変数「操作」の設定

変数「データ構造」の設定



set キュー to キューデータ構造 list

コレクション キュー push 1 1

コレクション キュー push 2 2

コレクション キュー push 3 3

三番目の入力スロットは空になっていますが、値を入れたとしても使用されません。

コレクション キュー pop 1 1

コレクション キュー pop 2 2

コレクション キュー pop 3

コレクション キュー pop

コレクション キュー empty? true

キューを使って九九のドリルを作ってみます。2～9の問題をシャフルしてばらばらの順番にします。シャフルしなければ昇順になりますし、リバース又は9～2にすれば降順の問題になります。問題をキューにセットします。「popした間に正解できなければその問をキューにpushする」をキューが空になるまで繰り返します。その段をクリアしないと終わりません。間違えた問には「!」を付けます。

```
when clicked
  script variables
  set 段 to 3
  set c to 1
  set 段問題 to キューデータ構造 shuffled of numbers from 2 to 9
  repeat until コレクション 段問題 empty?
    set 残り to コレクション 段問題 size
    set 問 to コレクション 段問題 pop
    set 答え to 段 x 問
    set チェック to if c > 8 then ! else 
    ask join ( 残り ) チェック 段 *x 問 !=? and wait
    if answer = 答え
      say よくできました for 2 secs
    else
      say よくかんがえよう for 2 secs
      say join 段 *x 問 != 答え です for 2 secs
    set dummy to コレクション 段問題 push 問
  change c by 1
```

スタックはデータを積み上げるイメージで、取り出しは積み上げたデータのトップからになります。使用できる機能はキューと同じにしています。

+スタックデータ構造+

block variables スタックリスト data

set スタックリスト to list

if 操作 = push

report

add input to スタックリスト

report input

input names: input

else if 操作 = pop

set data to item last of スタックリスト

report

delete last of スタックリスト

report data

else if 操作 = get

report

report item last of スタックリスト

else if 操作 = empty?

report

report is スタックリスト empty?

else if 操作 = size

report

report length of スタックリスト

else if

report

report

input names: 操作

set スタック to スタックデータ構造

コレクション スタック push 1

コレクション スタック push 2

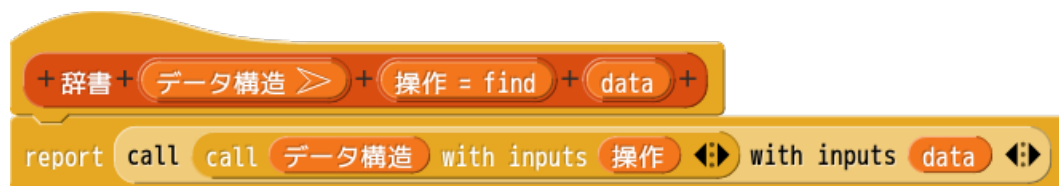
コレクション スタック push 3

三番目の入力スロットは空になっていますが、値を入れたとしても使用されません。

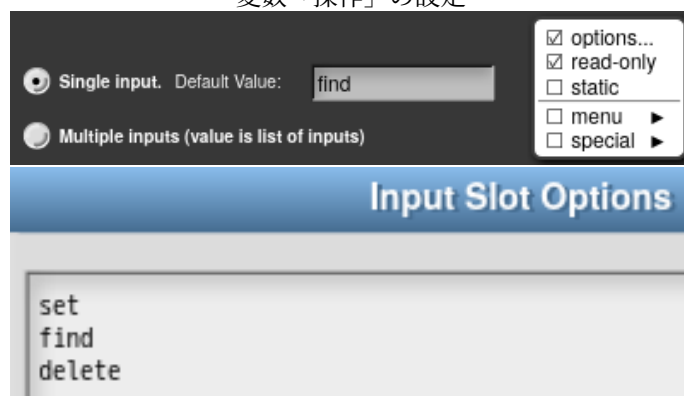


辞書は、キーとデータが対になったものを格納します。連想リストという言い方もあります。検索と削除はキーの値を指定して行います。

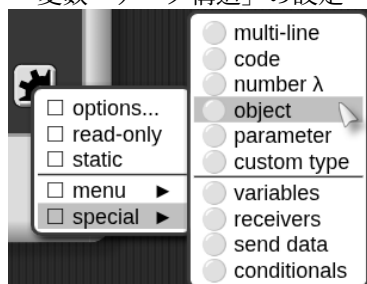
紙面の都合で、操作用の定義ブロックを先に示します。変数「操作」は options...(set, find, delete)、read-only で指定するようにします。Default value は find です。変数「データ構造」を special -> object に設定し、辞書データ構造を持つ変数を指定して使えるようにします。



変数「操作」の設定



変数「データ構造」の設定



+ 辞書データ構造 + 初期値 : +

block variables 辞書リスト index data

set 辞書リスト to 初期値

report

if 操作 = set

report

if is input a list?

set index to

index of find first item first 目 = first input in 辞書リスト in 辞書リスト

if index > 0

delete index of 辞書リスト

insert input at 1 of 辞書リスト

report input

else

report

input names: input

else if 操作 = find

report

set data to find first item first 目 = input in 辞書リスト

report if data = then else item 2 of data

input names: input

else if 操作 = delete

report

set index to

index of find first item first 目 = input in 辞書リスト in 辞書リスト

if index > 0

set data to item index of 辞書リスト

delete index of 辞書リスト

report data

else

report

input names: input

else if

report report

input names: 操作

set 辞書 to 辞書データ構造 list

1 one
2 1
length: 2

辞書 辞書 set list one 1

1 two
2 2
length: 2

辞書 辞書 set list two 2

1 three
2 3
length: 2

辞書 辞書 set list three 3

辞書 辞書 find two 2

1 two
2 2
length: 2

辞書 辞書 delete two

辞書 辞書 find two

キュー、スタック、辞書の使用例として「逆ポーランド記法計算」を示します。

[参考文献]

『プログラミング言語 AWK』 第2版 発行：オライリー・ジャパン
Alfred V. Aho
Brian W. Kernighan 著 千住 治郎 訳
Peter J. Weinberger

一般的な「中置記法」による計算式 $(1+2) \times 3$ は「逆ポーランド記法」（「後置記法」）で記述すると、`1 2 + 3 *` になります。カッコを必要としません。要素が数値の時はスタックに積み、演算子の時は必要な個数の数値をスタックから取り出して、演算の結果をスタックに積みます。この操作を式の終わりまで繰り返すと演算結果がスタック上に求められます。（キーボードには「 $\times$ 」がないので「*」を使用します。）「変数=」でスタックから変数に、「変数」で値がスタックに置かれます。変数の管理は辞書を使用し、初期値として PI を登録しています。

逆ポーランド記法計算 `list 1 2 + 3 *` 9

逆ポーランド記法計算 `list 2 r= PI r 2 ^ *` 12.56

逆ポーランド記法計算 `list 1 +` 引数不足エラー

逆ポーランド記法計算 `list 1 2 3 +` 引数過多エラー

逆ポーランド記法計算 `list 1 2 @` 演算子エラー (@)

演算子テーブル設定用の辞書リストです。指定できるのは入力スロットが二つのものだけです。演算子をキーとして検索するとそれを実行するための演算ブロックが得られます。

+演算子リスト2+

report

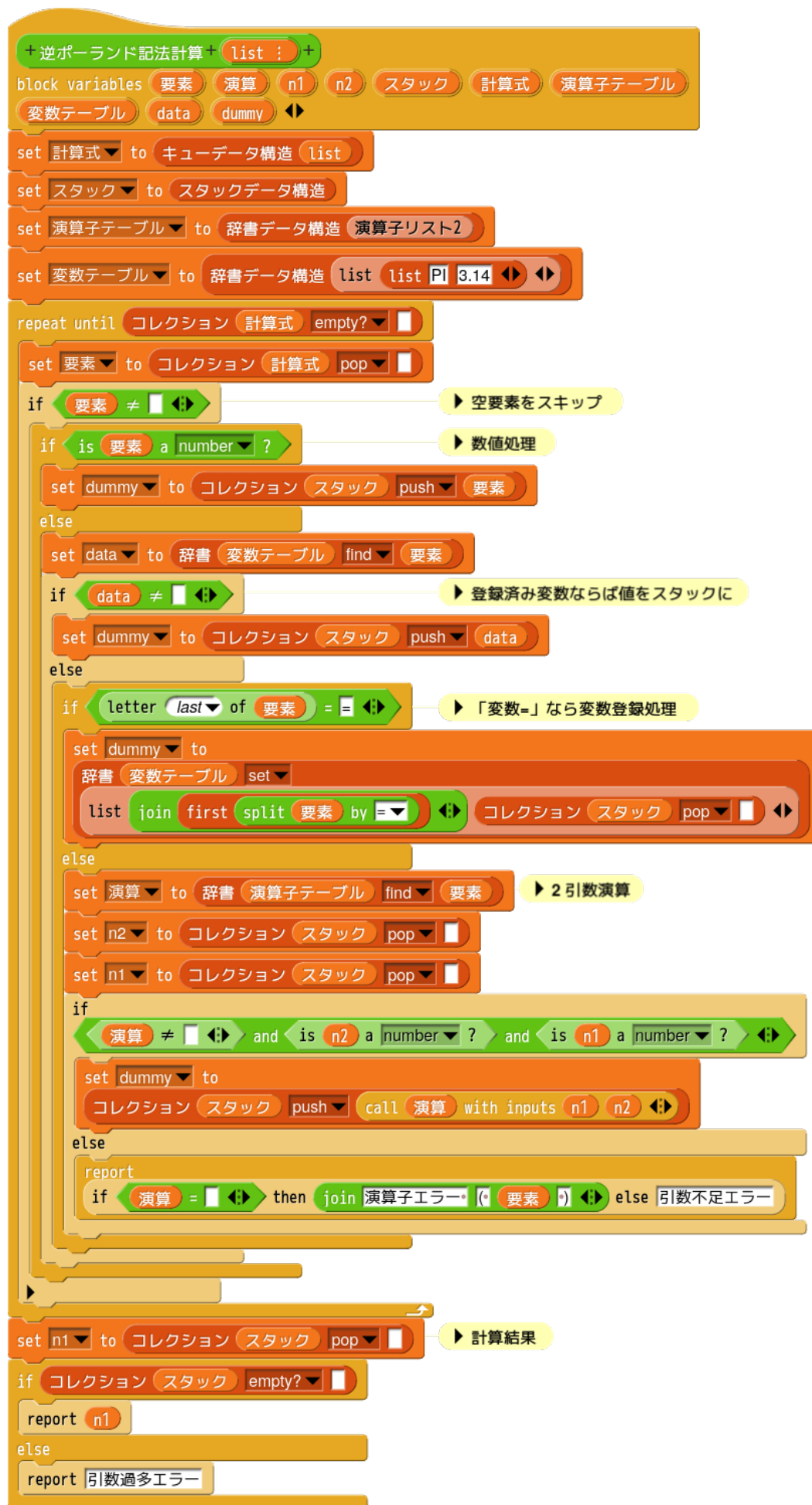
list + list -

list * list /

list ^ list mod

list min list max

本体の定義です。紙面の都合で機能限定版です。後で示す foldl 版のような機能を持たせることもできます。



foldl を使用すると、その仕様をキューやスタックの機能として利用することができます。foldl に与えられたリストからはキューのように要素が供給されます。init は演算の結果で更新されるものですが、これをスタックのように使用して、数値や数値を取り出して得た演算結果を積んでいきます。

FORTH 言語の DUP、SWAP、OVER、ROT 機能を追加し、1 引数の演算もできるようにしています。ただし、細かいエラー処理はできていません。辞書リストにはスタック処理用の辞書、引数 1 用の辞書、引数 2 用の辞書、変数用の辞書のリストがセットされます。



SWAP はスタックトップから一番目と二番目を入れ替えます。



DUP はスタックトップの値をコピーします。



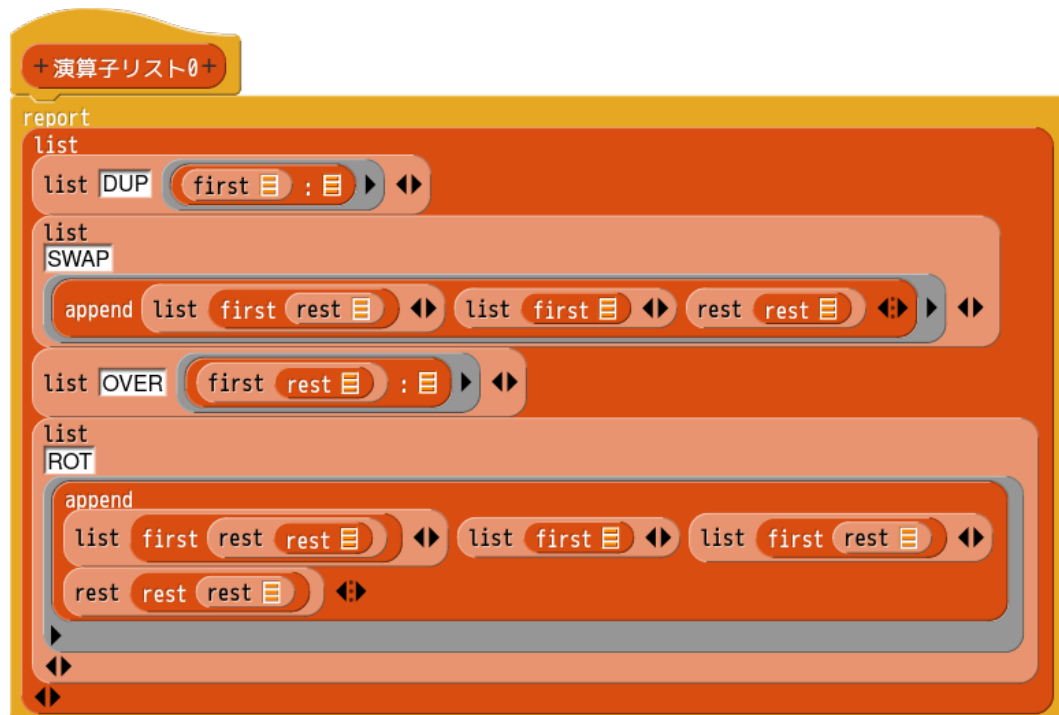
OVER はスタック二番目の値をコピーします。 3 4 コピー



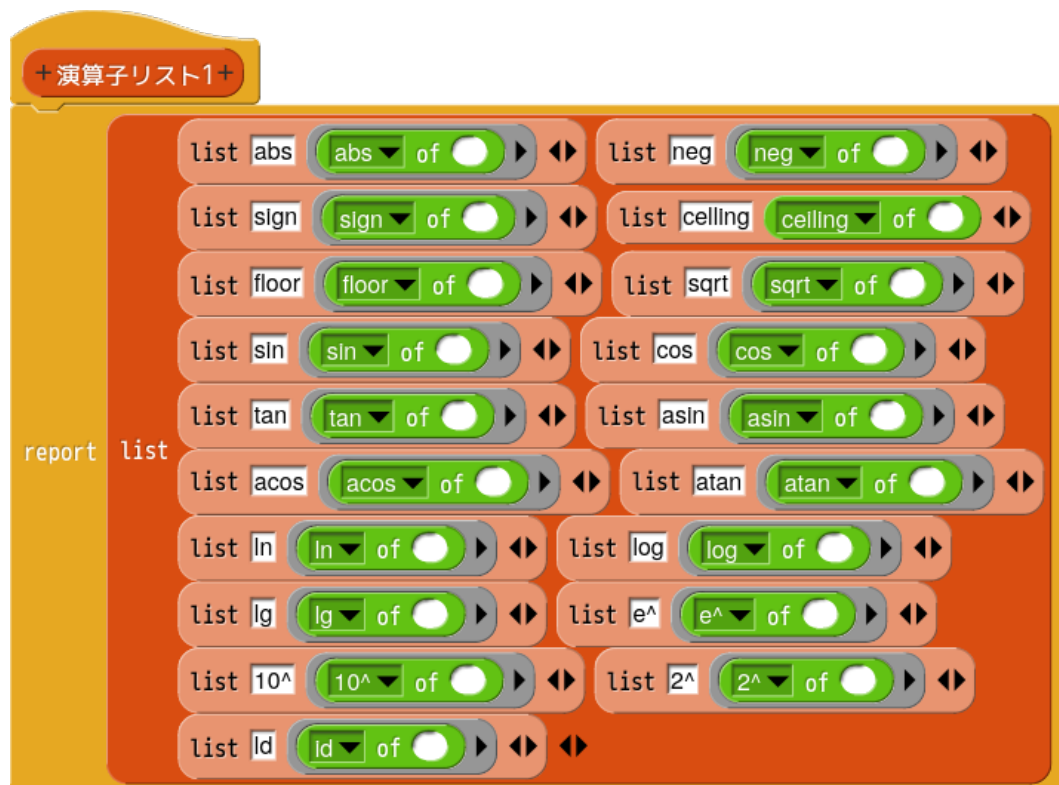
ROT はスタックトップ 3 の要素を循環して入れ替えます。 3 <- 4 <- 5 移動



スタック演算用辞書設定ブロックです。



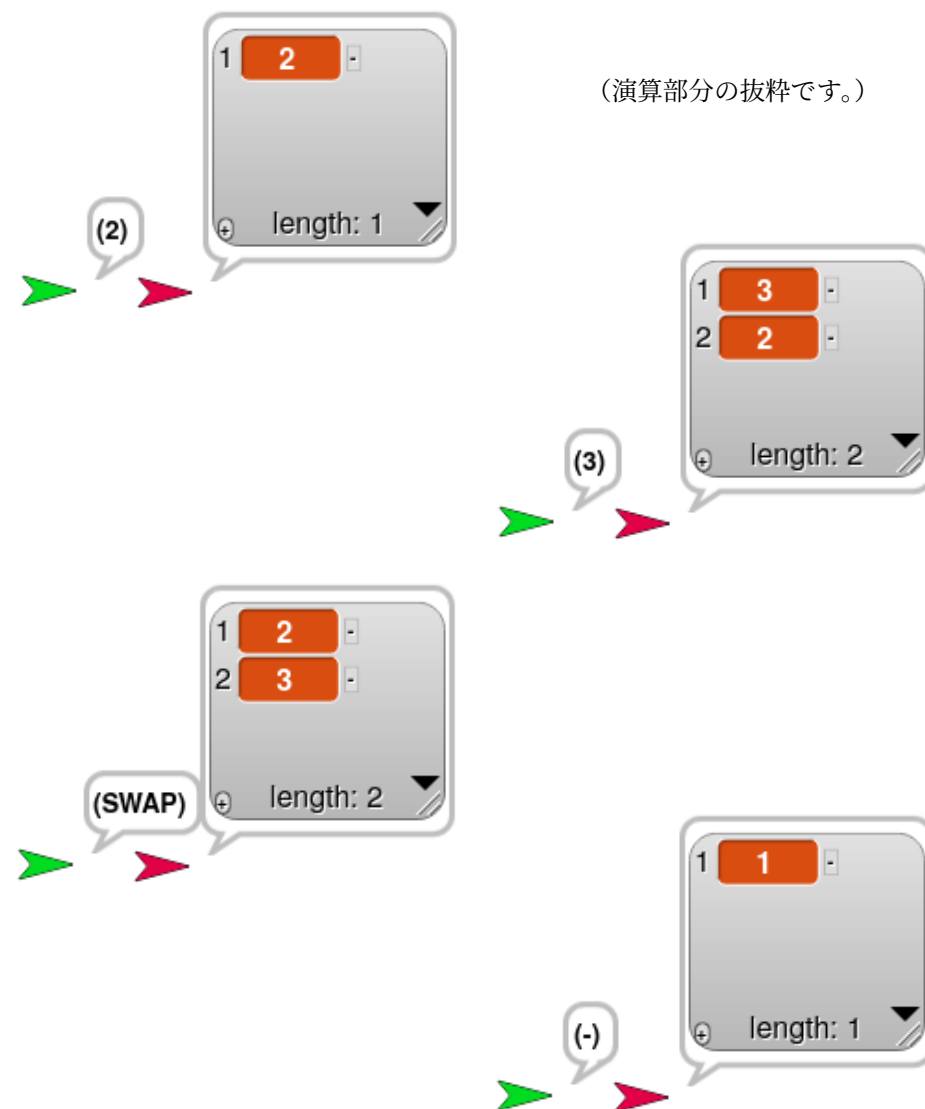
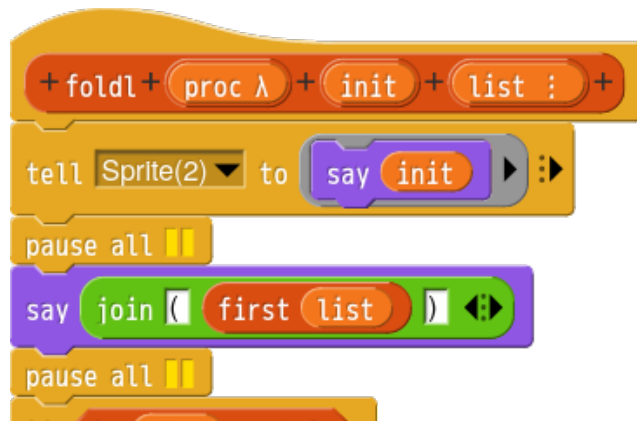
1 引数演算用辞書設定ブロックです。



foldl に指定する計算処理を行うブロックです。



Sprite(2) を追加して、次のように foldl 定義に say を挿入すると、処理中の式の要素やスタックとして使用している init の様子が見られます。式の実行は Sprite 側から行います。



索引

: ブロック, 40

all, 56

append, 50

break, 19

catch, 17

category, 25

collapse, 56

columns, 6

continuation, 14

continue, 19

deep contains, 49

define, 25

definition, 28

delete, 53

delete block, 27

drop, 44

dropWhile, 66

editables, 26

end thread, 21

factorial, 39

first ブロック, 40

for ブロック, 19

identical, 55

initial slots, 56

inner product, 4

iteration-composition, 17

keep, 45, 53, 57, 71

map, 51, 54, 56, 70

menues, 26

multiple inputs, 56

outer product, 11

pipe, 10, 12, 13

qsort, 57

replace, 54

rest ブロック, 40

separator, 56

slots, 26

take, 44

takeWhile, 66

this, 35

thread, 21

throw, 17

type, 27

unfold, 74

unique, 45

with continuation, 14

yield, 21

階乗, 39

回転行列, 5

カーリー化, 77

逆ポーランド記法, 86

キュー, 79

行列の積, 4

クイックソート, 57

クロージャー, 76

継続, 14

高階関数, 64

再帰呼び出し, 39

座標変換, 5

参照渡し, 51

辞書, 83

スタック, 82

スレッド, 21

素数, 12, 45

多重再帰, 57, 60

単位行列, 11

転置行列, 6

ハノイの塔, 39

フィボナッチ数列, 60

部分適用, 77

末尾再帰, 62

リスト要素の巡回, 48, 67